
Workgroup:	Network Working Group
Internet-Draft:	draft-whelan-chio-protocol-00
Published:	2 October 2026
Intended Status:	Standards Track
Expires:	5 April 2027
Author:	C. Whelan <i>Backbay Industries</i>

The Chio Protocol: Kernel Primitives for Agentic Operating Systems

Abstract

Chio is a kernel for agentic operating systems. It provides shared primitives for delegated authority, resource accounting, governed execution, and verifiable records. This document specifies the Chio protocol: the signed objects and exchanges through which agents, kernels, and supporting services use those primitives.

The protocol defines capability tokens, receipts, budgets and metering, governed transactions, and signed checkpoints. A capability token binds time-bounded authority to a subject key. The authenticated attenuation profile binds delegated authority to the parent scope and checks the child against that scope. Governed transactions require declared intent and approval; checkpoints commit to batches of receipts.

Agents access the kernel through a framed native transport or a binding to the Model Context Protocol. HTTP interfaces support issuance, delegation, receipt query, and revocation. The kernel establishes authority before execution, accounts for committed resources, controls the release of results, and preserves verifiable evidence across retries and recovery. These functions provide a common execution foundation for agents, applications, and workflows.

Status of This Memo

This Internet-Draft is submitted in full conformance with the provisions of BCP 78 and BCP 79.

Internet-Drafts are working documents of the Internet Engineering Task Force (IETF). Note that other groups may also distribute working documents as Internet-Drafts. The list of current Internet-Drafts is at <https://datatracker.ietf.org/drafts/current/>.

Internet-Drafts are draft documents valid for a maximum of six months and may be updated, replaced, or obsoleted by other documents at any time. It is inappropriate to use Internet-Drafts as reference material or to cite them other than as "work in progress."

This Internet-Draft will expire on 5 April 2027.

Copyright Notice

Copyright (c) 2026 IETF Trust and the persons identified as the document authors. All rights reserved.

This document is subject to BCP 78 and the IETF Trust's Legal Provisions Relating to IETF Documents (<https://trustee.ietf.org/license-info>) in effect on the date of publication of this document. Please review these documents carefully, as they describe your rights and restrictions with respect to this document. Code Components extracted from this document must include Revised BSD License text as described in Section 4.e of the Trust Legal Provisions and are provided without warranty as described in the Revised BSD License.

Table of Contents

1. Introduction	7
1.1. Design Goals	8
1.2. Scope	9
1.3. Document Organization	9
2. Conventions and Terminology	9
3. Protocol Overview	11
3.1. Roles and Trust Boundaries	11
3.2. Conformance and Feature Selection	12
3.3. Kernel Execution	12
3.4. Durable Execution Lifecycle	13
3.5. Protocol Surfaces	14
4. Encoding and Cryptography	15
4.1. Canonical JSON	15
4.2. Hashing	16
4.3. Signature Suites	16
4.4. Key and Signature Encodings	17
4.4.1. Bounded Cryptographic Decoding	17
4.5. Artifact Signing Inputs	18
4.6. Chio Key Identifiers	18

5. Capability Tokens	18
5.1. Capability Structure	18
5.2. Scope and Grants	19
5.2.1. Tool Grants	20
5.2.2. Resource Grants	20
5.2.3. Prompt Grants	20
5.3. Constraints	20
5.4. Sender Constraint	21
5.5. Delegation	22
5.6. Attenuation	22
5.6.1. Authenticated Security Context	23
5.7. Chain Binding	24
5.8. Verification	25
5.9. Feature Negotiation	25
6. Receipts	26
6.1. Receipt Structure	26
6.2. Receipt Kinds and Boundaries	28
6.3. Decisions	28
6.4. Content Binding	29
6.5. Receipt Identifier	29
6.6. Signing	29
6.7. Receipt Lineage	30
6.8. Child Receipts	30
6.9. Verification	31
7. Budgets and Metering	32
7.1. Monetary Amounts	32
7.2. Grant Ceilings	32
7.3. Budget Holds	33
7.4. Delegated Budget Shares	34
7.5. Aggregate Invocation Budgets	34

7.6. Guarantee Levels	37
7.7. Authoritative Spend	37
7.7.1. Operation-Owned Nonces	39
7.7.2. Caller Execution and Spend Evidence	39
7.8. Metering Metadata	40
8. Governed Transactions	44
8.1. Applying the Checks	45
8.2. Governed Intent	45
8.2.1. Approval of Exact Arguments	48
8.2.2. Intent Hash	49
8.2.3. Intent Checks	49
8.2.4. Metered Billing	50
8.3. Intent Constraints	51
8.4. Approval Tokens	52
8.5. Threshold Approval	55
8.5.1. Proposal	55
8.5.2. Verifying a Threshold Set	57
8.5.3. Kernel-Issued Proposals	58
8.6. Provenance Classes	59
8.7. Call-Chain Continuation	60
8.8. Governed Context in Receipts	62
9. Receipt Checkpoints	64
9.1. Checkpoint Statement	64
9.2. Merkle Trees	66
9.3. Predecessor and Consistency Proofs	66
9.4. Inclusion Proofs	69
9.5. Checkpoint Verification	71
9.5.1. Retention and Evidence Availability	71
9.6. Claim Limits	71

10. Native Transport	72
10.1. Native Framing	72
10.2. Native Messages	73
10.3. Tool Call Results	76
10.4. Native Errors	78
10.5. Receiver Behavior	79
11. Hosted MCP Binding	80
11.1. Endpoints and Headers	80
11.2. Session Admission	82
11.3. Initialization	83
11.4. Session States	84
11.5. Notification Stream and Replay	85
11.6. Tool Calls	85
11.7. Model Metadata	89
12. Trust-Control Interface	89
12.1. Capability Issuance	90
12.2. Delegated Issuance	91
12.3. Receipt Query	93
12.3.1. Paging	94
12.4. Revocation	95
13. Versioning and Negotiation	96
14. Error Model	96
14.1. Error Registry	97
14.2. Surface Mappings	97
15. Security Considerations	97
15.1. Assets and Trust Boundaries	97
15.2. Transport Security	98
15.3. Token Theft and Replay	99
15.4. Kernel Impersonation and Receipt Trust	99
15.4.1. Signing Roles and Key Lifecycle	100

15.5. Delegation Abuse	100
15.6. Signature Algorithm Downgrade	100
15.7. Canonicalization	101
15.8. Runtime Enforcement Profiles	101
15.8.1. Tool Server Confinement	101
15.8.2. Credential Custody	101
15.8.3. Information-Flow Control	102
15.8.4. Active Response	102
15.9. Resource Exhaustion	103
15.10. Budgets and Approvals	103
15.11. Checkpoint Equivocation	103
15.12. Provenance Inflation	103
15.13. Attestation	103
15.14. Time and Deadlines	104
16. Privacy Considerations	104
17. IANA Considerations	105
17.1. Chio Error Codes	105
17.2. Chio Signature Suites	106
17.3. Chio Native Message Types	107
17.4. Chio Signed Artifact Schemas	107
18. Implementation Status	108
19. References	108
19.1. Normative References	108
19.2. Informative References	110
Appendix A. Examples	112
A.1. Capability Token	112
A.2. Tool Call Request Frame	115
A.3. Receipt	116
Appendix B. Test Vectors	117
B.1. Canonical JSON	117

B.2. SHA-256 Digests	119
B.3. Ed25519 Signatures	119
B.4. Capability Token Verification	121
B.5. Receipt Verification	123
Appendix C. Relationship to Other Work	126
C.1. Agent Communication Protocols	126
C.2. Delegated Authorization	126
C.3. Capability Tokens	126
C.4. Workload Identity and Attestation	127
C.5. Signed Evidence and Transparency	127
C.6. Verifiable Credentials	127
C.7. Candidate Companion Documents	127
Appendix D. Document History	127
Acknowledgments	128
Author's Address	128

1. Introduction

Agentic operating systems need a shared execution core beneath their agents, applications, and workflows. That core establishes the authority an agent can exercise, the constraints on delegated work, the accounting of resource use, and the records that other parties can verify. These responsibilities span individual models, tools, and application frameworks.

Chio supplies this foundation as a kernel. Capability-based authority, local policy evaluation, budgets, governed transactions, and signed records form a common execution model on which agentic systems can build. This document specifies the protocol primitives and exchanges that make those functions interoperable across kernel implementations.

Agents read and write files, query services, send messages, and spend money on behalf of people and organizations. When an agent directly holds broad tool credentials, delegating a task can also delegate more authority than it needs. Unless the deployment records an authenticated authorization decision and result binding, a third party lacks that evidence of what was authorized and what happened.

The kernel separates an agent's requested operation from the authority to execute it. A native agent sends each kernel-mediated call with a capability token: a statement, signed by an issuer, that a subject key may call named tools under stated constraints and limits until a stated time. The kernel verifies the token, evaluates local policy, dispatches the call to the tool server only if both permit it, and records the decision and result in a signed receipt. Execution is a stateful kernel operation: admission, resource reservation, dispatch, output release, and finalization have distinct authority and failure semantics. The kernel preserves those distinctions through concurrency, process failure, and restart ([Section 3.4](#)).

Chio's authenticated delegation profiles enforce narrowing of authority. A holder can request or arrange trusted-issuer signing of a narrower token for another key. The resulting token signature is made by a trusted issuer; a delegation-link signature alone does not make an arbitrary holder a trusted token issuer. Each delegation link is signed. In the authenticated attenuation profile, scope hashes and a subset witness bind the child scope to the issuer-resolved parent scope. A kernel enforcing that profile rejects a claimed parent scope that is not bound to the chain. Plain delegated v1 tokens do not themselves require these hashes or witnesses; deployments requiring authenticated attenuation enforce the profile in [Section 5.7](#).

This document specifies the objects and exchanges that independent implementations need in order to interoperate: capability tokens, receipts, their canonical encoding and signatures, the native transport between agent and kernel, a binding to the Model Context Protocol [[MCP](#)], and the HTTP interface of the trust-control service. It also specifies three extensions that deployments use to reserve spend exposure and to audit history: budgets and metering, governed transactions, and receipt checkpoints.

1.1. Design Goals

Shared kernel primitives: Agents and applications use a common model of authority, constrained execution, and signed records across the protocol's transports.

Explicit authority: Every mediated call names the capability token that authorizes it. The kernel grants no authority that a presented token does not carry.

Narrowing delegation: An authenticated attenuation profile rejects authority beyond its trusted parent in the checked dimensions: tools, operations, constraints, invocation counts, costs, or time.

Fail-closed evaluation: A failure to verify a token, a delegation chain, a proof, or a policy result denies the call.

Durable execution and evidence: The kernel binds authorization, resource accounting, and signed outcomes to the same operation. Recovery preserves committed authority and prevents an uncertain operation from silently becoming a new execution.

Portable verification: A party with the receipt and an independently trusted kernel key can verify its integrity without contacting the kernel. Additional claims, such as inclusion in a particular log or execution by a particular provider, require evidence appropriate to that claim.

Transport independence: The native transport and the MCP binding use the same signed capability and receipt formats. Native replies and hosted MCP tool results carry receipts for kernel outcomes; authorized consumers can also query retained receipts.

Algorithm agility: Keys and signatures identify their own algorithm, so classical and post-quantum hybrid signature suites can coexist in one deployment.

1.2. Scope

This document specifies the kernel's signed artifacts and protocol exchanges. Implementations choose the runtime architecture and host isolation mechanisms that support those exchanges.

This document does not specify:

- a policy language or a set of guards. Each kernel evaluates its own local policy and records the results in receipts;
- how tool servers implement tools;
- a replacement for the Model Context Protocol, the Agent2Agent protocol [\[A2A\]](#), or other agent communication protocols. Protocol bindings connect those interfaces to the kernel's authority and execution model;
- an OAuth authorization server;
- payment settlement, standing relationships between operators, or portable identity credentials for agents. [Appendix C](#) lists these as candidate companion documents.

1.3. Document Organization

[Section 3](#) describes the roles and walks through one native kernel execution. [Section 4](#) through [Section 9](#) define the signed objects: the encoding and signature rules, capability tokens, receipts, budgets, governed transactions, and checkpoints. [Section 10](#) through [Section 12](#) define the exchanges: the native transport, the MCP binding, and the trust-control interface. [Section 13](#) and [Section 14](#) define version negotiation and the error model. The remaining sections cover security, privacy, IANA registrations, and implementation status. [Appendix A](#) and [Appendix B](#) give examples and test vectors generated from the reference implementation.

2. Conventions and Terminology

The key words "MUST", "MUST NOT", "REQUIRED", "SHALL", "SHALL NOT", "SHOULD", "SHOULD NOT", "RECOMMENDED", "NOT RECOMMENDED", "MAY", and "OPTIONAL" in this document are to be interpreted as described in BCP 14 [\[RFC2119\]](#) [\[RFC8174\]](#) when, and only when, they appear in all capitals, as shown here.

This document uses the following terms.

Agent: A software actor that requests tool calls. The kernel does not trust an agent; an agent acts only through the capability tokens it presents.

Kernel: The trusted execution core of an agentic system. It verifies capability authority, evaluates local policy, applies the configured budget and governance checks, dispatches permitted calls to tool servers, and durably records signed outcomes. Its execution state survives a restart without recreating consumed authority.

Tool server: A service that implements tools, resources, or prompts, to which a kernel dispatches permitted calls.

Trust-control service: The service that issues and revokes capability tokens and answers queries over stored receipts.

Capability: The authority to call a set of tools, read a set of resources, or use a set of prompts, under stated constraints and limits.

Capability token: A signed, time-bounded JSON object that grants a capability to a subject key ([Section 5](#)).

Issuer: The key that signs a capability token.

Subject: The key to which a capability token is bound. Only the holder of the subject key can exercise the token when the token requires a sender proof.

Grant: One entry in a capability token's scope: a tool grant, a resource grant, or a prompt grant.

Delegation: Issuing a capability token from the authority of an existing token.

Delegation link: One signed step in a capability token's delegation chain.

Attenuation: A narrowing of authority along one or more dimensions: the grants themselves, their operations and constraints, invocation counts, cost ceilings, budget shares, or time.

Guard: A check that the kernel evaluates before dispatch or before releasing output, as part of its local policy.

Receipt: A signed record of a call or operation, carrying a kernel decision, trace observation, or advisory evaluation ([Section 6](#)).

Operation: One logical execution, including its admission, reservations, dispatch, recovery, and finalization. A transport request identifier correlates an exchange; it is not by itself a durable operation identifier.

Dispatch commitment: The durable transition that consumes an operation's dispatch authority and captures its associated resource exposure. Commitment does not prove that an external effect occurred.

Output release: The authorized transfer of a result across a kernel trust boundary, after the applicable output checks and durable release conditions.

Execution profile: The agreed set of enforcement and evidence requirements for a call. A profile cannot weaken the artifact verification rules in this document.

Decision: The outcome that a receipt records for a mediated call: `allow`, `deny`, `cancelled`, or `incomplete`.

Checkpoint: A signed statement by a kernel that commits to a batch of receipts in a Merkle tree ([Section 9](#)).

Code-formatted words such as `expires_at` are protocol element names and values, which are case-sensitive. JSON is defined in [[RFC8259](#)].

3. Protocol Overview

3.1. Roles and Trust Boundaries

[Figure 1](#) shows the roles and the four flows between them.

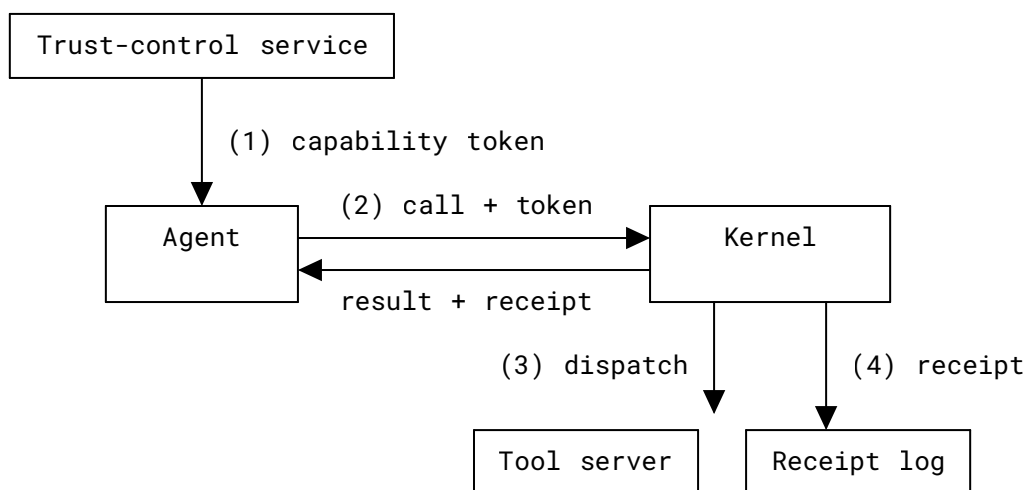


Figure 1: Roles and Flows

The trust-control service issues a capability token to the agent (1). The agent sends a call, with the token, to the kernel (2). The kernel verifies the token, evaluates policy, and, if both permit the call, dispatches it to the tool server (3). The kernel signs a receipt for the receipt-bearing outcome and records it (4). The illustrated native exchange returns that receipt with the result. The hosted MCP binding returns the receipt in result metadata ([Section 11.6](#)). Authorized consumers can query retained evidence through trust-control.

The kernel is trusted to mediate the call path. It holds the key that signs receipts and the configuration of trusted issuer keys. The agent is untrusted: it can present any token it holds, and the kernel verifies each one. Preventing calls that bypass the kernel depends on deployment isolation and credential controls ([Section 15.8.1](#)). A tool server role does not itself confer token-issuing authority. The trust-control service is trusted to issue and revoke tokens under the operator's policy.

3.2. Conformance and Feature Selection

This document distinguishes artifact verification from kernel execution. An artifact verifier implements the encoding, signature, and semantic checks for the artifact types it accepts. It **MUST** identify unsupported schemas, algorithms, and required features as unsupported; it **MUST NOT** report them as successfully verified. A valid signature authenticates signed bytes under a key. Trust in that key and acceptance of the artifact's authority are separate checks.

A conforming execution kernel implements capability verification, policy enforcement, receipts, and the durable lifecycle in [Section 3.4](#). Budgets, governed transactions, authenticated attenuation, and checkpoints are feature profiles. A kernel that accepts a call requiring a feature **MUST** enforce that feature's complete requirements. Otherwise, it **MUST** reject the call before dispatch. It **MUST NOT** discard a constraint or substitute an advisory check for an enforcement requirement.

Peers **MUST** establish the required features, signature policy, trusted identities, and transport binding before accepting execution requests. The native v1 binding uses authenticated deployment configuration; its wire version alone does not negotiate these features. The MCP binding uses its initialization exchange and trusted server policy. An agent-supplied feature list cannot lower that policy. A change that requires a new wire member or value **MUST** follow [Section 13](#).

Confinement, credential custody, information-flow control, and active response extend the kernel's execution boundary. [Section 15.8](#) defines their security requirements when selected by an execution profile. Their platform mechanisms and companion artifact formats are not additional wire formats implicitly negotiated by this document. An implementation **MUST** state which profiles it implements. Support for an artifact library or protocol adapter alone is not conformance as an execution kernel.

3.3. Kernel Execution

[Figure 2](#) shows one call over the native transport ([Section 10](#)).

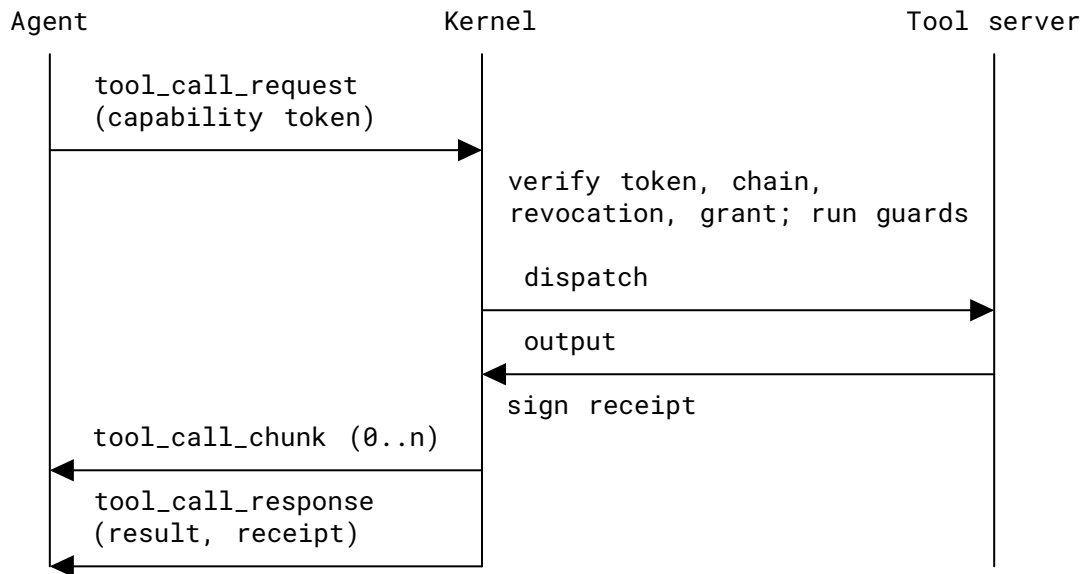


Figure 2: One Native Kernel Execution

The figure shows a successful execution. A kernel can buffer output until finalization, or use a streaming release policy that enforces the output requirements below. Chunk boundaries describe delivery, not the time at which the tool produced the data.

A native call proceeds as follows:

1. The agent sends a `tool_call_request` with its capability token, target, and parameters.
2. The kernel verifies the token, chain, validity interval, revocation state, matching grant, required proofs, and local policy. The native v1 request has no sender-proof field; a call requiring an explicit sender proof is rejected on this binding.
3. The kernel reserves applicable resources and resolves required approvals. A pending approval does not authorize dispatch.
4. The kernel commits dispatch authority and calls the tool server. It evaluates the returned output under the selected release policy.
5. The kernel durably finalizes the operation and its receipt, then returns the authorized result. [Section 3.4](#) specifies the failure and recovery cases; [Section 10.3](#) defines the wire outcomes.

3.4. Durable Execution Lifecycle

The kernel **MUST** bind the authenticated subject, capability digest, normalized target, parameters digest, selected policy, and required resource reservations to one logical operation. A retry, approval, or recovery message **MUST NOT** replace that binding with caller-supplied state. Concurrent processes sharing an authority domain **MUST** serialize consumption of its single-use authority and resource limits.

Before dispatch, the kernel **MUST** verify current authority and acquire the required reservations. A reservation, successful evaluation, pending approval, or execution nonce is not permission for an independent caller to perform an external effect. Dispatch authority **MUST** be committed durably before a tool is permitted to execute. The commit **MUST** bind the operation to its executor and resource state. A stale process or expired ownership lease **MUST NOT** authorize a new transition.

Before commitment, the kernel can cancel an operation and release its unconsumed reservations. After commitment, loss of a response or absence of evidence of an effect **MUST NOT** be treated as proof of non-execution. The kernel **MUST** preserve committed exposure until authenticated evidence and the applicable accounting rules authorize reconciliation. It **MUST NOT** automatically redispach a committed operation, refund its exposure, or mint replacement single-use authority because a process restarted or a deadline elapsed. An uncertain effect is recorded as incomplete.

Output **MUST** pass the applicable guards before crossing the agent-facing boundary. When a policy requires the complete result, the kernel **MUST** buffer it within configured limits before release. A streaming profile **MUST** define the unit of inspection and release; an unchecked prefix cannot be exposed and retroactively described as blocked. The kernel **MUST** durably record the release authorization and retain the evidence needed to finalize the operation before releasing protected output. A release record does not prove that the client received the bytes.

Finalization **MUST** bind the original operation, accounting outcome, and receipt. Recovery **MUST** reuse that binding and any already-finalized receipt; it **MUST NOT** silently sign a different account of the same terminal outcome. If safe recovery cannot establish ownership or recover required state, the affected execution domain **MUST** remain unavailable. A missing or corrupt authority store **MUST NOT** be replaced with an empty store that restores consumed authority.

Failures before sufficient authenticated context exists can produce only a transport error. A signing or storage failure can also prevent a receipt from being delivered. Such failures **MUST NOT** authorize dispatch or output release that requires the missing evidence. If an effect might already have occurred, the kernel **MUST** preserve an incomplete operation for reconciliation. It **MUST NOT** fabricate a successful receipt or use an untrusted emergency signing key.

These rules provide durable execution authority, not a generic guarantee of exactly-once external effects. An external executor needs an authenticated operation binding and durable duplicate suppression, or a provider-specific idempotency and recovery contract. A client **MUST NOT** infer that a timed-out operation is safe to repeat from its transport status alone.

3.5. Protocol Surfaces

Capability tokens and receipts use the same object formats across the three surfaces. Their delivery differs:

Surface	Carries	Encoding	Defined in
Native transport	Calls from agent to kernel	Length-prefixed canonical JSON	Section 10
MCP binding	Calls from MCP clients to a hosted kernel	JSON-RPC over HTTP with server-sent events	Section 11
Trust-control interface	Issuance, delegation, receipt query, revocation	JSON over HTTP	Section 12

Table 1: Protocol Surfaces

The native transport has no initialization exchange. A native agent obtains its capability tokens out of band, for example from the trust-control service, and then sends calls.

4. Encoding and Cryptography

Chio signs typed JSON projections. A verifier reconstructs the projection specified for that artifact, canonicalizes it, and verifies its signature. It does not sign the original transport bytes, whitespace, or unknown members discarded during typed parsing. Consumers therefore distinguish authenticated fields from other data in the incoming envelope.

4.1. Canonical JSON

Canonical bytes are UTF-8. Object members are ordered lexicographically by their UTF-16 code-unit sequences; a proper prefix sorts first. Arrays retain order. The encoding emits no whitespace between tokens. It escapes quotation marks, reverse solidus, and U+0000 through U+001F, using the short escapes for backspace, form feed, line feed, carriage return, and tab. Other control escapes use four lowercase hexadecimal digits. Other Unicode characters are emitted without normalization.

Floating-point values use the shortest round-trip decimal representation. Negative zero becomes 0. Values with magnitude at least 0.000001 and less than 1e21 use decimal notation; other nonzero values use exponent notation, with + on a non-negative exponent. Non-finite values cannot be encoded. These rules follow [\[RFC8785\]](#).

The v1 typed encoding additionally preserves signed 64-bit and unsigned 64-bit integer values as exact decimal integers. It does not round those values to binary64. For example, 9007199254740993 remains those decimal digits. This compatibility rule differs from unrestricted application of RFC 8785's I-JSON number model. An implementation using only binary64 numbers cannot verify such an artifact after rounding its integer fields. For interoperable inputs within the shared I-JSON domain, the encodings agree. Producers can keep integers within the safe binary64 range when interacting with consumers that cannot preserve full-width integers.

Receivers **MUST** validate the original UTF-8 input before constructing a typed object. They **MUST** reject duplicate object member names at every nesting level, including duplicates written with different JSON escape sequences. Parsing into a map and checking afterward is insufficient. Receivers **MUST** reject invalid Unicode, non-finite numbers, out-of-range integers, and conversions that round or truncate a signed field. Signed integer fields **MUST** use decimal integer tokens without a fraction or exponent; their full declared range **MUST** be preserved. An unsigned field **MUST NOT** accept a negative spelling, including negative zero. These requirements apply to network input, persisted signed records, imports, and nested signed envelopes.

Unsigned application parameters may contain ordinary JSON floating-point values. Their canonical representation, rather than their source spelling, is hashed. A receiver **MUST NOT** apply an integer-only restriction to an arbitrary JSON field. Where a protocol specifies exact canonical bytes, the receiver **MUST** additionally compare the original bytes with the canonical encoding.

Each artifact specifies the fields covered by its signature. Unknown members **MUST NOT** confer authority or satisfy a required feature. A closed schema rejects them; an explicitly extensible metadata object retains them in the signed projection. A verifier **MUST NOT** silently drop a recognized security constraint to make an artifact verify.

4.2. Hashing

Unless a structure specifies a domain prefix or Merkle construction, Chio uses SHA-256 over the specified bytes [[FIPS180-4](#)]. Content-addressed receipt identifiers and scope hashes are lowercase hexadecimal strings without 0x. Capability and other application-assigned identifiers are strings; they are not required to be hashes. Merkle hashes are serialized as 0x followed by 64 lowercase hexadecimal digits. A hash of canonical JSON means the hash of its UTF-8 bytes, rather than a hash of an application language's object storage.

4.3. Signature Suites

This document defines Ed25519 [[RFC8032](#)], ECDSA P-256 with SHA-256, ECDSA P-384 with SHA-384 [[FIPS186-5](#)], and a composite of each classical suite with ML-DSA-65 [[FIPS204](#)]. ECDSA signatures use ASN.1 DER encoding [[RFC3279](#)]. ML-DSA-65 uses an empty context string. Composite verification requires both components over identical message bytes and matching encoded algorithm sets.

A verifier dispatches from the parsed key and signature material. It **MUST** reject a key/signature algorithm mismatch. A composite verifier **MUST** reject a different algorithm set on the key and signature, an invalid component pairing, or either failed component signature. An unsupported suite fails verification. Capability and receipt v1 verification uses ordinary Ed25519 verification; it does not imply the additional weak-key checks of a strict-verification profile.

Cryptographic floors are `allow_classical`, `allow_hybrid`, and `pq_required`. They respectively accept classical suites only, classical or composite suites, and composite suites only. A floor-aware verifier **MUST** reject an artifact disallowed by the configured floor. On artifacts with an `algorithm` envelope hint, it also rejects a present hint inconsistent with the signature encoding. A missing hint does not override a self-describing signature unless the artifact explicitly defines a

legacy default. Approval tokens and threshold proposals interpret an absent hint as Ed25519. A verifier **MUST** apply the rules of the artifact it is verifying rather than infer a different default from another artifact.

An execution profile **MUST** select its accepted suites through trusted configuration. That policy applies to every required signature, including capabilities, approval proposals, votes, receipts, and recovery evidence. The allowed sets can differ by signer role, but an artifact cannot select its own lower floor. A signer **MUST** check that the backend's returned key and algorithm match the requested signing identity and that the produced signature verifies. It **MUST NOT** fall back to a weaker backend after a signing failure. Receipt signing, error paths, and recovery **MUST** use the configured receipt authority. Persisted terminal receipts are replayed as original signed envelopes, even when a suite can produce multiple valid signatures for the same message.

4.4. Key and Signature Encodings

Ed25519 public keys are 32 octets encoded as 64 lowercase hexadecimal characters; signatures are 64 octets encoded as 128 such characters. These encodings have no algorithm prefix. ECDSA keys use uncompressed SEC1 points [SEC1]: p256 : followed by hexadecimal of 04 || X || Y (65 octets), or p384 : followed by the corresponding 97-octet point. ECDSA signatures use the same prefix followed by hexadecimal of DER signature bytes.

The composite format for keys and signatures is:

```
hybrid:<classical-encoding>:<pq-hex>:<algorithm-set>
```

The classical component retains its own prefix when it is ECDSA. The algorithm set is exactly ed25519+mldsa65, p256+mldsa65, or p384+mldsa65. ML-DSA-65 public keys occupy 1952 octets and signatures 3309 octets. Components are separated from the right, since an ECDSA classical encoding contains a colon. Composites cannot nest. This version defines no standalone ML-DSA wire encoding.

Producers emit lowercase hexadecimal and no 0x on classical material. Classical decoders also accept uppercase hexadecimal and an optional 0x; typed serialization normalizes those spellings before verification.

4.4.1. Bounded Cryptographic Decoding

A decoder **MUST** bound encoded lengths before allocation and decoding. Ed25519 keys and signatures contain exactly 32 and 64 octets. P-256 and P-384 public keys contain exactly 65 and 97 octets; their DER signatures are at most 72 and 104 octets respectively. An ECDSA verifier **MUST** reject non-DER encodings, trailing data, and invalid scalar or point values. ML-DSA-65 components contain exactly 1952 public-key octets or 3309 signature octets. A composite contains exactly one permitted classical component and one ML-DSA-65 component; recursive composites are invalid. A length-valid encoding still requires full cryptographic verification.

4.5. Artifact Signing Inputs

The projection is part of the protocol. Capability tokens include their schema and exclude their signature and algorithm hint. Receipts sign a wrapper containing the computed identifier and identity projection. Delegation links, approvals, child receipts, and checkpoints use their own body projections. Aggregate budget roots add a domain prefix. Copying a signing rule from one artifact to another is not valid. The relevant sections define the exact bytes.

4.6. Chio Key Identifiers

The `did:chio` form in this version is `did:chio:` followed by the 64 lowercase hexadecimal characters of an Ed25519 public key. It does not encode an ECDSA or composite key. Resolution constructs a local key document; it does not perform an online trust lookup. The key method identifier is the DID plus `#key-1`, with type `Ed25519VerificationKey2020`. Its `publicKeyMultibase` is `z` followed by base58btc of `0xed 0x01` and the raw key. Authentication and assertion references identify that method. Knowing the DID does not establish permission to issue capabilities or sign receipts for an operator.

5. Capability Tokens

A capability token is an issuer-signed grant to a subject key for a bounded time. The issuer's trust relationship, the grant's scope, local policy, and the runtime's enforcement profile jointly determine whether the kernel can dispatch a request.

5.1. Capability Structure

Field	Type	Presence
schema	<code>chio.capability.v1</code> string	Defaulted on input
id	String	Always
issuer, subject	Encoded public keys	Always
scope	Scope object	Always
issued_at, expires_at	Unsigned 64-bit Unix seconds	Always
delegation_chain	Array of links	When nonempty
aggregate_invocation_budget	Budget object	When present
caveats	Array	When nonempty
scope_attenuations	Array	When nonempty
attenuation_proof	Proof object	When present
budget_share_bps	Unsigned 16-bit integer	When present
algorithm	Algorithm hint	When non-default
signature	Encoded signature	Always

Table 2: Capability Token Fields

The signing input is one flattened object containing every typed member above except `algorithm` and `signature`, with the same omission rules. The `schema` member is included and equals `chio.capability.v1`, even when omitted by an older wire producer. There is no byte prefix. The issuer **MUST** reject a signing operation whose actual key differs from issuer.

[Appendix A.1](#) gives exact generated bytes.

For compatibility, verification can retry the body without `schema` only for a plain v1 token: no caveats, no nonempty `scope_attenuations`, no attenuation proof, no budget share, no aggregate budget, and no cumulative-approval constraint. A delegation chain alone does not disable this fallback. New signing uses the schema-aware projection. Other schema values fail validation.

5.2. Scope and Grants

A scope has grants (tool grants), `resource_grants`, and `prompt_grants`. Each array defaults to empty and is omitted when empty. An empty scope is `{}`. Operations are `invoke`, `read_result`, `read`, `subscribe`, `get`, and `delegate`. An operation name is not proof that a runtime implements an enforcement point for it.

5.2.1. Tool Grants

A tool grant requires `server_id`, `tool_name`, and `operations`. `constraints` is omitted when empty. `max_invocations`, `max_cost_per_invocation`, `max_total_cost`, and `dpop_required` are included when present. [Section 7](#) defines the ceilings.

A grant matches an invocation when the server and tool each equal the request value or the literal `*`, the operations contain `invoke`, and all argument constraints match. Tool names have no prefix-glob rule. Matches are ordered by server exactness, tool exactness, then constraint count, each descending; ties keep the original grant index. A constraint evaluation error on a target-matching grant aborts resolution rather than allowing a less specific grant to hide that error. The portable tool profile selects the first match. The hosted kernel can try matched grants in order through governance, policy, runtime, and budget admission.

5.2.2. Resource Grants

A resource grant requires `uri_pattern` and `operations`. Resource reading checks `read`; subscription checks `subscribe`. Patterns use raw-string equality, `*` for all values, or a trailing `*` for a prefix. No URI normalization is implied by that comparison.

5.2.3. Prompt Grants

A prompt grant requires `prompt_name` and `operations`. Prompt retrieval checks `get` using the same exact, universal, or trailing-prefix pattern rules. Resource and prompt matching are outside the portable tool-only profile.

5.3. Constraints

Constraints use `{"type":name,"value":payload}` with case-sensitive snake_case names. A unit constraint omits `value`. Unknown variants or unknown members of the constraint object fail parsing. The portable argument profile evaluates these constraints:

- `path_prefix` (string): identify path-like string leaves, normalize slash and reverse-solidus separators, remove empty and `.` segments, resolve `..` without escaping the root, and require matching absolute or relative form and the stated segment prefix for every candidate.
- `domain_exact` and `domain_glob` (strings): extract host-like values from string leaves, remove scheme, user information, port, path, query, and fragment, trim dots, and lowercase. Exact compares hosts; the glob's `*` matches any run including dots. At least one candidate is needed, and every candidate has to match.
- `max_length` (non-negative integer): limit the UTF-8 byte count of each argument string leaf.
- `max_args_size` (non-negative integer): limit the bytes in compact JSON of the entire arguments value, rather than its canonical form.
- `custom` (two-string array): require a recursively found object member whose name and string value match. Reserved `delivery` and `recovery` spellings are errors, not custom predicates. They are `output_digest_sha256`, `require_finding_purchase`, `require_finding_recovery`, `recovery_of_receipt_id`, and `recovery_of_capability_id`.

- `audience_allowlist` (string array): check members named `recipient`, `recipients`, `audience`, `to`, `channel`, or `channels`, ignoring case in the names. If present, each value must contain at least one string and contain only strings or recursively nested arrays. Every string value is allowlisted, including an empty string only when explicitly allowlisted. Absent audience members pass.
- `memory_store_allowlist` (string array): the same rule for `store`, `memory_store`, `collection`, and `namespace`.

Path candidates have a case-insensitive key containing path or equal to `file`, `filepath`, `dir`, `directory`, `root`, or `cwd`; path-like values without `://` are also candidates when they begin with `/`, `./`, `../`, or `~/`, or contain a path separator. A missing path candidate does not match. Host candidates are `localhost` or strings containing a dot with only ASCII alphanumerics, dots, and hyphens after extraction.

Governed constraints are defined in [Section 8.3](#). Other constraint families require an explicitly supported runtime profile. When constraint evaluation encounters an unsupported constraint, the portable kernel **MUST** fail grant resolution. A marker that a hosted matcher defers to a guard is not evidence that the guard enforced it. [Section 18](#) identifies that profile boundary.

5.4. Sender Constraint

The Chio invocation proof object below is distinct from the hosted HTTP DPoP proof. The native `tool_call_request` has no field carrying it; the ordinary session bridge supplies no invocation proof. Hosted HTTP sender proof authenticates the edge session and is not forwarded as this kernel invocation proof. A grant requiring invocation-level DPoP therefore needs a surface that carries the object; neither transport silently supplies it.

When any matching tool grant has `dpop_required: true`, the hosted kernel requires a valid proof before invocation. The native proof is `{body, signature}`. Its body has schema (`chio.dpop_proof.v1`), `capability_id`, `tool_server`, `tool_name`, `action_hash`, `nonce`, `issued_at` (Unix seconds), and `agent_key`. The subject signs canonical JSON of that body, without a prefix. `action_hash` hashes canonical arguments only. This is capability/action proof of possession; it is distinct from the HTTP-method and URI JWT binding in [\[RFC9449\]](#).

The kernel checks schema, subject key, capability/server/tool/action bindings, freshness, signature, and nonce replay state. The default freshness window is 300 seconds with 30 seconds future-clock allowance; the expiration comparison is inclusive. The nonce's replay identity is (`nonce`, `capability_id`) and is retained through its signed expiry. Preview and dispatch revalidation can be stateless; dispatch credential reservation reserves the nonce. A required proof with unavailable replay state fails closed. Hosted HTTP sender constraint is a separate binding specified in [Section 11.2](#).

5.5. Delegation

A delegation link requires `capability_id`, `delegator`, `delegatee`, `timestamp`, and `signature`. It includes attenuations when nonempty, and `scope_hash`, `aggregate_budget`, and `cumulative_approval` when present. The delegator signs canonical JSON of those typed fields except `signature`; there is no schema member or domain prefix. The signer has to match `delegator`.

A chain verifier checks each link's signature, adjacent key connectivity, nondecreasing timestamps, and final delegatee equal to the token subject. A configured maximum depth is also enforced. A portable verifier has no universal default depth limit. The hosted kernel additionally validates stored parent snapshots, delegator authority, validity windows, expiry narrowing, and delegation permissions. It checks revocation for the token and every named ancestor.

5.6. Attenuation

Signed attenuation arrays use internally tagged objects. Members below are beside type, rather than nested inside a value member:

type	Additional required members
<code>remove_tool</code>	<code>server_id</code> , <code>tool_name</code>
<code>remove_operation</code>	<code>server_id</code> , <code>tool_name</code> , <code>operation</code>
<code>add_constraint</code>	<code>server_id</code> , <code>tool_name</code> , <code>constraint</code>
<code>reduce_budget</code>	<code>server_id</code> , <code>tool_name</code> , <code>max_invocations</code> (unsigned 32-bit integer)
<code>shorten_expiry</code>	<code>new_expires_at</code> (unsigned 64-bit integer)
<code>reduce_cost_per_invocation</code>	<code>server_id</code> , <code>tool_name</code> , <code>max_cost_per_invocation</code> (monetary amount)
<code>reduce_total_cost</code>	<code>server_id</code> , <code>tool_name</code> , <code>max_total_cost</code> (monetary amount)

Table 3: Signed Attenuation Variants

Target names are strings; `operation`, `constraint`, and monetary amounts use the types defined above. A receiver **MUST** reject unknown variants and unknown members within a variant.

A cumulative-approval constraint in a child scope **MUST** preserve the parent's approval-budget identifier, epoch, currency, and canonically equal root binding, or have both root bindings absent. Its threshold **MUST NOT** exceed the parent's threshold. A child **MUST NOT** introduce a cumulative-approval marker absent from its covering parent grant.

Every child grant fits a parent grant of the same kind. For tools, the parent covers the child server and tool, child operations are contained, parent constraints remain present, invocation and monetary caps remain or narrow, and `dpop_required: true` remains true. Monetary caps preserve currency. Constraint preservation uses equality except the supported cumulative-approval narrowing rule in [Section 8](#). Resource and prompt narrowing use their pattern-coverage rules. Delegating authority additionally requires a covering parent grant with `delegate`.

The attenuation proof uses camelCase members `parentScopeHash`, `childScopeHash`, and `normalizedSubsetProof`. The witness contains `normalizedParentScope` and `normalizedChildScope` (JSON strings), `subsetRelations` and `restrictedPredicates` when nonempty, and `aggregateBudget` and `cumulativeApproval` when present. A relation has `grantKind`, `childIndex`, `parentIndex`, and boolean `subset`. Scope hashes are SHA-256 of the canonical typed scope bytes. Array order is preserved; normalization does not sort grants or operations.

The verifier hashes the supplied scope strings, parses both scopes, and requires both strings to be exact canonical JSON and recomputes actual grant subset relationships. It rejects a false declared subset relation, child-hash mismatch, or a widened actual scope. Relation indices and textual restricted-predicate descriptions are not independent authorization evidence. The token's `scope_attenuations` is signed but does not substitute for recomputing the subset. The only supported caveat in this document is the security-context binding below. A verifier **MUST** reject other caveat kinds and a token containing more than one security-context binding.

5.6.1. Authenticated Security Context

A capability can bind its authority to an authenticated runtime context through a caveat whose kind is `bind_security_context`. Its predicate is a string containing the exact canonical JSON of the following object. The caveat **MUST NOT** carry a detached sig; the capability signature covers the complete caveat.

Member	Type and meaning
schema	chio.capability-security-binding.v1
tenantId	Tenant identity
lineageId	Capability lineage identity
sessionId	Authenticated session identity
principalId	Authenticated principal identity
isolationEpochId	Isolation epoch identity
contextGeneration	Positive unsigned 64-bit context generation
workloadId	Workload identity
serverId	Tool-server identity
workloadSignerPublicKey	Workload signing key (Section 4.4)

Table 4: Capability Security Context

All members are **REQUIRED**. All identities are nonempty strings without leading or trailing whitespace or control characters. Unknown members are invalid. The receiver **MUST** validate the predicate's original bytes, require exact canonical encoding, validate the key, and compare every member with independently authenticated runtime context. A caller's unsigned assertion of the same identifiers is insufficient.

An execution kernel **MUST** reject a context-bound call if it cannot obtain that trusted context or if any member differs. Protocol adapters **MUST** preserve the binding through dispatch; they **MUST NOT** remove the caveat to obtain compatibility with a weaker evaluator. A new session, principal, tenant, workload, or isolation epoch does not inherit authority merely because it presents an old token. Delegation requiring a different binding needs explicit authorization from the trusted issuer and a new valid token.

5.7. Chain Binding

A proof, nonempty scope_attenuations, or a budget share triggers chain binding. The verifier **MUST** require an attenuation proof and an issuer-resolved trust-root scope hash for such a token. For an empty chain, the proof's parent hash equals that root hash. For a delegated token, the last link's hash equals the proof's parent hash, the first link's hash equals the root hash, and every link carries a scope hash. This proof-bearing profile accepts at most one link. A plain chain alone does not trigger this rule; aggregate and cumulative profiles impose their own authenticated root-snapshot checks.

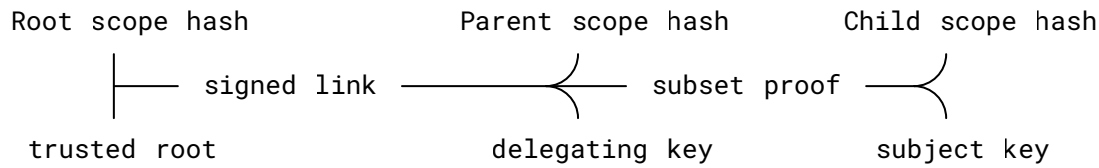


Figure 3: One-Hop Scope-Bound Delegation

5.8. Verification

Portable full verification checks, in order: peer profile validity; issuer trust; optional-feature negotiation; algorithm-hint consistency and cryptographic floor; schema and signature; time; supplied direct-root capability, if present; aggregate and cumulative budget bindings; chain shape; chain binding; and sibling-share admission. Time is valid exactly when `issued_at <= now < expires_at`, with no capability clock-skew allowance.

Failures deny admission. They distinguish untrusted issuer, invalid signature, rejected cryptographic floor, not-yet-valid, expired, attenuation violation, and budget split rejection. Some schema or proof errors occur inside signature reconstruction and surface as internal verification failures. The wire mapping is specified in [Section 14](#).

Hosted admission adds token/ancestor revocation, stored delegation lineage, exact subject binding (`agent_id` equals normalized subject-key encoding), scope resolution, proof of possession, governance, guards, runtime and budget admission, credential reservation, and immediate dispatch revalidation. Checks can repeat across that boundary. A token verified earlier is not exempt from later revocation or changed state.

5.9. Feature Negotiation

A capability-feature profile is `{"schema": "chio.capabilities.v1", "features": {"name": true}}`. schema defaults to this value on parsing and is always serialized; an empty features map is omitted. Unknown envelope members are rejected. Feature names contain 1 to 96 ASCII bytes from `[a-z0-9_.-]`; values are booleans. The flags relevant here are `delegation_chain_binding`, `aggregate_invocation_budget`, `cumulative_approval_budget`, and `threshold_governed_approvals`, `governed_active_response_plan`, and `opaque_supplemental_authorization`. The last two select a response-plan validator or an installed supplemental-authorization verifier; preserving an opaque object does not verify it or authorize execution.

The intersection of two valid profiles contains true only when both sides declare true, false when either explicitly declares false, and omits a flag declared true by only one side. The chain-binding rule's absent-flag compatibility default still applies to that intersection.

Before an invocation crosses an adapter, the adapter **MUST** check support for every required feature against the host-authenticated peer profile. Invocation metadata **MUST NOT** supply that profile. The adapter **MUST NOT** strip approval artifacts, aggregate limits, or supplemental

authorization to make a request acceptable to an unsupported receiver. An absent chain-binding flag defaults to enabled; explicitly disabling it rejects tokens that require it. A feature flag describes supported validation behavior, not permission to skip a constraint whose evaluation is unsupported. The local kernel configures this profile through trusted deployment state; the separately specified federation handshake carries signed capability negotiation and a pinned intersection. MCP initialization advertises its own transport features, not the complete `chio.capabilities.v1` negotiation object.

6. Receipts

A receipt binds a kernel key to a recorded outcome. Its signature permits a verifier to detect changes to the signed record. The verifier separately decides whether to trust that key and what authority the recorded boundary supports. A trace of an observed call does not establish that the kernel authorized that call.

6.1. Receipt Structure

The receipt is one JSON object. It has no top-level schema member. `chio.receipt.v1` names the artifact and signing projection, rather than a field to insert into the wire object. All names below are case-sensitive.

Field	Type	Presence
id	Lowercase SHA-256 hex string	Always
timestamp	Unsigned 64-bit Unix seconds	Always
capability_id	String	Always
tool_server, tool_name	Strings	Always
action	Action object	Always
receipt_kind	Kind string	Always
boundary_class	Boundary string	Always
tool_origin	Origin string	Always
redaction_mode	Redaction string	Always
decision	Decision object	Mediated kind
observation_outcome	Outcome string	Other kinds
content_hash	SHA-256 hex string	Always
policy_hash	Policy identifier string	Always
actor_chain	Array of actor objects	When nonempty
evidence	Array of guard evidence	When nonempty
trust_level	Trust string	Always
metadata	JSON value	When present
tenant_id	String	When present
kernel_key	Encoded public key	Always
algorithm	Algorithm hint	When non-default
signature	Encoded signature	Always

Table 5: Receipt Fields

The action has parameters (any JSON value) and `parameter_hash`. The latter is lowercase hexadecimal SHA-256 of the canonical parameters. An actor has `actor_id` and, when supplied, `actor_kind`. Guard evidence has `guard_name`, `verdict` (boolean), and, when supplied, `details`. `policy_hash` identifies the evaluated policy; a symbolic policy identifier is also representable, so this field is not necessarily a SHA-256 digest.

`tool_origin` is `caller_executed`, `host_executed_provider_reported`, `host_executed_unmediated`, or `chio_internal`. The last class identifies an operation executed within Chio's trusted runtime; it does not confer additional authority or change the receipt's decision semantics. `redaction_mode` is `none`, `summary`, or `redacted`. The origin describes the execution path; it does not change the boundary's authority. Redaction describes the producer's representation of the record, not permission to modify an already signed receipt.

6.2. Receipt Kinds and Boundaries

The kernel or other producer **MUST** check the following coherence rules before signing. A receipt verifier **MUST** reject a record that violates them.

Kind	Boundary	Trust	Outcome member
<code>mediated_decision</code>	<code>prevent</code>	<code>mediated</code>	<code>decision</code>
<code>trace_observation</code>	<code>detect_only</code>	<code>verified</code>	<code>observation_outcome</code>
<code>advisory_evaluation</code>	<code>advisory_only</code>	<code>advisory</code>	<code>observation_outcome</code>

Table 6: Receipt Coherence

A mediated receipt carries a decision and no observation outcome. A non-mediated receipt carries an observation outcome and no decision. Observation outcomes are observed, evaluated, and dropped. `cannot_see` is not a signable receipt boundary in this version. The word `verified` in a trust label describes the observation class; it does not prove preventive enforcement.

6.3. Decisions

Decisions are JSON objects tagged by `verdict`:

- `allow`: no additional members.
- `deny`: `reason` and `guard`, both strings.
- `cancelled`: `reason`, a string.
- `incomplete`: `reason`, a string.

An `allow` decision establishes authorization only inside a coherent `mediated_decision` receipt, under a trusted kernel key. Denied, canceled, and incomplete outcomes do not authorize the call. An incomplete outcome preserves uncertainty about execution; it is not a statement that nothing happened. An observation or advisory receipt never substitutes for an `allow` decision at a preventive boundary.

6.4. Content Binding

For a mediated value result, the kernel hashes canonical JSON of the evaluated output. For an ordinary, non-redacted call with no output, it hashes the four UTF-8 bytes `null`. For a stream, it first hashes each chunk's canonical data; then it hashes the concatenation of those lowercase hexadecimal digests, in chunk order, without separators. The concatenated values are ASCII hexadecimal strings, not raw 32-octet digests. Stream metadata records chunk hashes, count, and canonical byte count. A delivery-mismatch denial uses the distinct redacted commitment `H(UTF8("chio.delivery-mismatch.redacted.v1") || 0x00 || ASCII(expected_digest))` and omits public stream metadata. A verifier **MUST** select that producer profile before comparing the content binding.

When the kernel holds the evaluated output, it **MUST** recompute its content hash before signing. A pre-filled hash that does not match is rejected. A trusted-body relay can sign a record supplied by a trusted producer when it does not hold that output. That seam preserves producer trust; it does not provide an independent check of the content.

Different observation producers can bind different content preimages. A verifier therefore needs the producer's content profile and the actual content to check `content_hash`; the signature alone does not perform this check.

6.5. Receipt Identifier

Let $J(x)$ be the canonical JSON UTF-8 encoding in [Section 4.1](#) and let $H(x)$ be lowercase hexadecimal SHA-256. Construct the identity projection I from all typed receipt fields except `id`, `algorithm`, `signature`, and the selective-disclosure extension signature. Preserve the omission rules of [Section 6.1](#). The identifier is:

```
id = H(J(I))
```

Thus the kernel key, decision, content binding, action, evidence, attribution, metadata, and tenant are bound into the identifier. Field order in an incoming JSON object does not affect it. Unknown members discarded by a typed parser are outside this projection; consumers cannot infer that such members were authenticated. Extensions that change the projection need a separately defined profile. A base-profile consumer **MUST** reject `bbs_projection_version` or `bbs_signature`, or route the object to the separately defined selective-disclosure profile. This document does not define that extension's projection or signing wrapper.

6.6. Signing

Preparation proceeds in order: validate semantic coherence; bind the signing nonce if the pre-binding identifier is nonempty; compute the content-addressed identifier; then sign the wrapper below. The nonce is the trimmed pre-binding identifier. It is placed in `metadata.chio_receipt_signing_nonce`, replacing an existing value. Absent metadata becomes

an object. Non-object metadata is preserved under `original_metadata` before the nonce is inserted. An empty pre-binding identifier does not insert a nonce. A verifier cannot assume every receipt carries one.

Without selective-disclosure extension material, the signature input is exactly the canonical UTF-8 encoding of:

```
{"id": <computed identifier>, "body": <identity projection I>}
```

The angle-bracket notation describes a projection, not a JSON example. The kernel **MUST** reject a signing operation whose actual key differs from the embedded `kernel_key`. The signature covers the wrapper, not the wire envelope or the identity projection by itself. The optional `algorithm` hint is outside the signed projection. [Section 4.4](#) defines the self-describing signature encodings. [Appendix A](#) contains a generated receipt and its exact signing input.

6.7. Receipt Lineage

An ordinary receipt has no top-level DAG ordinal, parent set, or hybrid logical clock. A governed call carries pairwise continuation information inside signed metadata, as specified in [Section 8.7](#). A separate lineage statement can connect parent and child receipts. Multiple verified statements can describe multiple parents; this does not make generic receipt signature verification a DAG validator.

The lineage statement uses camelCase names. Its signed body contains schema (`chio.receipt_lineage_statement.v1`), `id`, `parentReceiptId`, `childReceiptId`, `parentRequestId`, `childRequestId`, `parentSessionAnchor`, `childSessionAnchor`, `relationKind`, `evidenceClass`, `issuedAt`, and `kernelKey`; `continuationTokenId` is included when present. The session-anchor reference and governed continuation are defined in [Section 8.7](#). Relations in this document are `local_child` and `continued`. The signature covers the canonical body, including its identifier; it does not use the ordinary receipt wrapper or nonce rule. A lineage statement verifier checks the schema and signature. The endpoints' authenticity, authority, and relationship need separate verification.

A parent-set hash helper hashes canonical JSON of a sorted, deduplicated array of receipt identifier strings. That helper does not bind a chain identifier or enforce DAG topology. This document defines no common top-level multi-parent receipt envelope and makes no acyclicity or child-ordinal claim for ordinary receipt verification.

6.8. Child Receipts

A child operation can end without being a mediated tool decision. Its receipt carries `id`, `session_id`, `parent_request_id`, `request_id`, `operation_kind`, `timestamp`, `terminal_state`, `outcome_hash`, `policy_hash`, `kernel_key`, and `signature`. `metadata` and `algorithm` use the ordinary presence rules. Its identifier is a producer-assigned string, not the content address of [Section 6.5](#).

Operation kinds are `tool_call`, `create_message`, `create_elicitation`, `list_roots`, `list_resources`, `read_resource`, `list_resource_templates`, `list_prompts`, `get_prompt`, `complete`, `list_capabilities`, and `heartbeat`. The `terminal_state` is tagged by state: `completed`, `cancelled` with a string `reason`, or `incomplete` with a string `reason`.

The child signature covers canonical JSON of its body, including `id` and excluding `algorithm` and `signature`. It has no ordinary receipt wrapper or signing nonce. A successful child outcome hashes canonical JSON of `{"outcome": "result", "result": value}`; an error hashes `{"outcome": "error", "message": text}`. An ordinary operation error can be a completed child operation. Child completion therefore does not mean that its result succeeded or was authorized. A child verifier **MUST** verify the canonical child body under its embedded signing key, and a consumer accepting it as evidence **MUST** separately authorize that signer and apply any configured cryptographic floor. It **MUST NOT** apply the ordinary receipt content-addressed identifier or wrapper to a child receipt. Signature verification does not recompute `outcome_hash`.

6.9. Verification

An ordinary receipt verifier follows this procedure; child receipts use the distinct preimage and checks in [Section 6.8](#):

1. Parse the typed fields and apply the kind, boundary, decision, and trust coherence rules in [Section 6.2](#).
2. Reconstruct `I`, recompute `H(J(I))`, and reject a different `id`.
3. Reconstruct the signing wrapper and verify the signature using the embedded key and signature encoding. Reject an algorithm mismatch between the key and signature. When a cryptographic floor is applied, also check a present hint for consistency and enforce the floor.
4. Check whether the kernel key belongs to the verifier's configured trusted set. A self-consistent record signed by an untrusted key is not accepted as evidence of that verifier's kernel.
5. Recompute `parameter_hash` from `action.parameters` and reject a mismatch. Before claiming content integrity, obtain the output and its producer profile, then recompute its content binding. If the output is unavailable, report content integrity as unchecked.
6. Evaluate the policy identity, time, tenant, attribution, and any lineage evidence against the verifier's own application policy.

A consumer accepting a receipt as evidence **MUST** perform steps 1 through 6 for its intended use. A signature-only helper implements only part of this procedure and **MUST NOT** be presented as complete receipt verification. Freshness, external content, and the operational meaning of metadata require the context identified above. A field named `trust_level` or `evidenceClass` never replaces those checks.

7. Budgets and Metering

Budgets mediate admission and reserve accounting headroom before dispatch. They do not, by themselves, establish the amount an external service billed or stop a provider from reporting more than the reserved exposure. A verifier distinguishes an authorization ceiling, a durable reservation, realized accounting, and an external charge.

7.1. Monetary Amounts

A monetary amount is exactly `{"units":integer,"currency":string}`. `units` is an unsigned 64-bit integer in minor units; zero is representable. `currency` denotes an ISO 4217 currency [ISO4217], such as USD with cents or JPY with yen. The wire type rejects unknown members. The typed grant path does not independently validate ISO spelling or the equality of per-call and total currencies. Delegation narrowing does compare currency strings and rejects a changed currency.

A monetary-amount producer **MUST** include both members and encode units as an integer from 0 through 18446744073709551615. A typed monetary-amount decoder **MUST** reject missing members, unknown members, a non-integer units value, or a units value outside that range. The JSON-schema profile additionally requires a nonempty currency string. The typed grant path does not enforce that string's ISO spelling or compare the per-call denomination with the total denomination; deployments that require those properties apply a separate currency policy.

Budget-store additions **MUST** fail on integer overflow rather than wrap. A store whose integer range is narrower **MUST** reject values outside that range. The reference SQLite store supports nonnegative values through 9223372036854775807. These budget-store rules do not describe the saturating totals in auxiliary metering records.

7.2. Grant Ceilings

A tool grant can carry `max_invocations` (unsigned 32-bit integer), `max_cost_per_invocation` (monetary amount), and `max_total_cost` (monetary amount). Each is independently omitted when absent.

A store enforcing a grant's invocation and monetary ceilings **MUST** authorize an admission only when every applicable check succeeds:

- The grant's reserved plus captured invocation count is less than `max_invocations`. A maximum of zero admits no invocation.
- Requested exposure is no greater than `max_cost_per_invocation`.
- Existing exposed units plus realized spend plus requested exposure is no greater than `max_total_cost`.

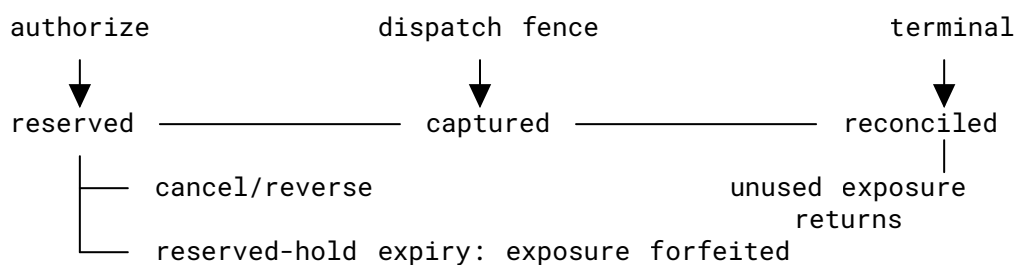
An absent ceiling contributes no check. Equality is permitted for the monetary checks. The kernel requests `max_cost_per_invocation.units` as exposure, or zero when that ceiling is absent. It does not compute exposure from a realized cost. Therefore `max_total_cost` alone does not advance monetary accounting and is not an enforced standalone spending ceiling.

An admission in the durable monetary profile **MUST** have a positive per-call exposure. A kernel enforcing that profile **MUST** deny financial tool dispatch when no durable admission covers the selected grant. An explicitly enabled development escape from that check does not establish durable monetary enforcement.

Monetary rows are scoped to the capability identifier and grant index. A delegated token does not automatically share its parent's monetary row.

7.3. Budget Holds

The admission lifecycle is authorize exposure and reserve invocation quotas, capture before dispatch, then reconcile measured spend. The kernel **MUST** use checked arithmetic and reject any transition that cannot be represented exactly in its authoritative store. It **MUST NOT** admit a value by wrapping, rounding, or saturating a counter.



For a positive monetary exposure, a hold store **MUST** reject reconciliation unless the invocation is captured, the monetary state is exposed, the supplied exposure equals the hold's remaining exposure, and realized units do not exceed that exposure. Successful reconciliation removes that exposure and adds realized units; the unused portion returns to headroom. Realized units can be zero.

A hold store **MUST** reject a monetary release unless the release is positive, does not exceed remaining exposure, the monetary state is exposed, and the invocation remains authorized. It **MUST** reject reversal unless the invocation remains authorized and the reversed amount is the complete remaining exposure. Release and reversal therefore cannot cross invocation capture. Cancellation of a captured admission before dispatch is a separate operation.

For an explicitly caller-reserved hold that is still open and whose reserved_until is at or before the reaper's time, the reaper **MUST** capture the invocation if needed and realize the full remaining exposure. This conservative legacy v1 accounting rule forfeits an abandoned reservation; it is not evidence that an external effect occurred. An operation-owned caller profile can instead compensate an uncommitted reservation when it proves that no dispatch authority was released. It **MUST NOT** compensate committed or uncertain execution merely because a deadline expired. Neither rule applies indiscriminately to every open inline hold. A zero-exposure invocation reservation settles through capture alone.

On the measured-cost path, the kernel reconciles at no more than the authorized exposure. A missing cost report on that path defaults to the full exposure. If reported cost exceeds a positive exposure, accounting closes at exposure and `settlement_status` is failed; the financial reported amount can retain the larger cost and the call can remain allowed. The receipt does not prove the external charge was limited. If an adapter cannot measure realized cost or provide the required final authorization, the kernel **MUST** either retain conservative exposure or reject a call requiring authoritative spend. An explicitly advisory profile can record a provisional outcome, but **MUST NOT** present zero reported cost as proof of zero liability or release committed exposure without authority.

7.4. Delegated Budget Shares

The signed token member `budget_share_bps` is an unsigned 16-bit value from 0 through 10000, omitted when absent. A token verifier **MUST** reject a value above 10000. A token carrying this member **MUST** also carry the attenuation proof required by [Section 5.7](#).

For delegated-token sibling admission, the parent identifier is the last delegation link's `capability_id`, the child identifier is the token's id, and an absent share is treated as 10000. The registry **MUST** reject an unknown parent, a changed share for an already admitted child, or an existing sibling sum plus the proposed share greater than the registered parent share. Re-admitting the same child and share is idempotent. Each intermediate parent requires its own registration; admitting a child does not automatically register it as a parent.

The registry compares parent and child shares on one basis-point scale. It does not scale `max_invocations`, `max_cost_per_invocation`, or `max_total_cost`, and the share alone does not establish a shared monetary balance. An execution kernel using shares as an admission constraint **MUST** maintain one authoritative registry for the delegation family. It **MUST** preserve admitted allocations across restart, serialize concurrent allocation, and release an allocation only when no live or recoverable holder can exercise it. An offline share calculation does not establish availability in that registry.

7.5. Aggregate Invocation Budgets

The token's `aggregate_invocation_budget` is an object with `scope` (`capability` or `delegation_family`), `max_invocations` (unsigned 32-bit integer), and `root_binding` when needed. `Capability` scope forbids a root binding and delegation. `Delegation-family` scope requires one. Neither this object nor its root body has an epoch member.

The signed root binding has `body`, `signature`, and the non-default `algorithm` hint when supplied. The body has:

- `schema`: `chio.aggregate-budget-root.v1`.
- `root_capability_id`, `root_capability_hash`, `root_scope_hash`: strings committing the root token and scope.
- `root_issuer`, `root_subject`: encoded public keys.
- `max_invocations`: unsigned 32-bit integer.

- `root_expires_at`: unsigned 64-bit Unix seconds.

Let $C(x)$ denote the typed canonical UTF-8 JSON bytes in [Section 4.1](#), and $H(x)$ denote SHA-256 rendered as 64 lowercase hexadecimal characters without 0x. A domain string below is UTF-8 followed by one zero octet. Let R be the authenticated direct root token, $S = H(C(R.scope))$, and N the family's `max_invocations`.

The root commitment projection is exactly:

```
{
  "root_capability_id": R.id,
  "root_issuer": R.issuer,
  "root_subject": R.subject,
  "root_scope_hash": S,
  "root_issued_at": R.issued_at,
  "root_expires_at": R.expires_at,
  "aggregate_scope": "delegation_family",
  "max_invocations": N
}
```

Here the expressions $R.id$ and similar expressions denote copied values, not JSON strings. `root_capability_hash` is:

```
H("chio.aggregate-budget-root-commitment.v1" || 0x00 ||
  C(projection))
```

The binding body B is exactly the following projection:

```
{
  "schema": "chio.aggregate-budget-root.v1",
  "root_capability_id": R.id,
  "root_capability_hash": the commitment hash above,
  "root_issuer": R.issuer,
  "root_subject": R.subject,
  "max_invocations": N,
  "root_expires_at": R.expires_at,
  "root_scope_hash": S
}
```

The commitment's `root_issued_at` and `aggregate_scope` are not members of B . A root-binding signer **MUST** use the key identified by $B.root_issuer$ and sign exactly:

```
"chio.aggregate-budget-root.v1" || 0x00 || C(B)
```

The signature is placed beside body, with a non-default algorithm hint when supplied. A verifier **MUST** reject an unsupported body schema, a present algorithm hint inconsistent with the signature, or a failed signature under B.root_issuer. A verifier comparing the binding with R **MUST** require equality of root_capability_id, root_capability_hash, root_issuer, root_subject, max_invocations, root_expires_at, and root_scope_hash with their recomputed root values.

For the complete binding E, including its signature and any non-default algorithm hint, the descendant marker digest is:

```
H("chio.aggregate-budget-root-binding-digest.v1" || 0x00 || C(E))
```

The shared family-accounting owner is:

```
H("chio.aggregate-budget-family-key.v1" || 0x00 || C(B))
```

The owner hashes the body, while the marker digest hashes the complete binding. Neither the aggregate object nor B has an epoch member.

A capability-scoped aggregate object **MUST** omit root_binding and **MUST NOT** authorize delegation. A delegation-family object **MUST** carry root_binding, and its maximum **MUST** equal the binding body's maximum. An implementation of this aggregate profile **MUST** reject family delegation longer than one hop.

A family-descendant verifier **MUST** authenticate a direct root token with an empty delegation chain and a trusted root issuer. It **MUST** reject the descendant unless all of the following hold:

- The first link's capability_id equals the root id, its delegator equals the root subject, and its scope_hash equals the root scope hash.
- The first link timestamp is at least root.issued_at and less than root.expires_at; the child issued_at is at least the final link timestamp.
- The final delegatee equals the child's subject, the child's expiry is no later than the root expiry, and its scope passes delegable attenuation against the root scope.
- The child has delegation_family scope, the same maximum, and a root binding whose typed canonical JSON is identical to the root's binding.
- Every link carries aggregate_budget with the binding digest and maximum. When an attenuation witness is present, it carries the same marker as aggregateBudget.

An aggregate-feature verifier **MUST** reject aggregate evidence when aggregate_invocation_budget was not negotiated. An enforcement path without the required durable composite admission **MUST** reject the aggregate request rather than infer enforcement from parsing.

Composite quota admission **MUST** check every participating quota before writing any reservation. It **MUST** reject a changed stored maximum, an exhausted reserved-plus-captured count, or a maximum of zero. Its quota set **MUST** be strictly sorted by unique key and contain at most eight entries. Capability aggregate quotas use the capability id as owner under

chio.aggregate-capability-invocation.v1; family quotas use the derived family owner under chio.aggregate-family-invocation.v1. Neither key has a grant index. Monetary rows remain per capability and grant.

An issuer supporting aggregate invocation budgets **MUST** emit all required root, link, and witness bindings. A verifier **MUST** check those bindings and the authoritative quota state before dispatch. An implementation that supports only ordinary per-token limits **MUST** reject a token requiring aggregate enforcement. Syntax validation alone does not establish a family's remaining authority.

7.6. Guarantee Levels

Guarantee identifiers and ranks are `advisory_posthoc` (0), `single_node_atomic` (1), `partition_escrowed` (2), and `ha_linearizable` (3). A guarantee-floor verifier **MUST** reject an unknown requested floor, a missing budget-authority block, an unknown claimed level, or a claimed level ranked below the requested floor.

A producer of budget execution metadata **MUST** obtain the guarantee identifier from its accounting store. The reference local stores return `single_node_atomic`; the remote trust-control accounting client returns `advisory_posthoc`. The reference implementation does not supply a store that returns `partition_escrowed` or `ha_linearizable`. Recognizing those identifiers does not establish those backings. Likewise, a `single_node_atomic` label by itself does not establish durable storage or distributed accounting truth.

Hold authority can carry `authority_id`, `lease_id`, and `lease_epoch`. A budget term is `authority_id:lease_epoch`. Local authority uses a kernel-key identifier, lease `single-node`, and epoch 1. A durable serving store checks its ownership fence within the transaction. An epoch on a hold authority is distinct from an aggregate invocation budget.

7.7. Authoritative Spend

An execution nonce has exactly the envelope members `nonce` and `signature`. The legacy v1 profile signs the body directly. The operation-owned v2 profile binds that body to durable admission context ([Section 7.7.1](#)). Neither profile alone authorizes an external caller to execute. The nonce body and its binding are:

Member	Type	Presence
schema	chio.execution_nonce.v1 or chio.execution_nonce.v2	required
nonce_id	nonempty string	required
issued_at	integer from 0 through 9223372036854775807, Unix seconds	required
expires_at	integer in the same range, Unix seconds	required
bound_to	object in the following table	required
reserved_hold_id	nonempty string	omitted when absent
reserving_request_id	nonempty string	omitted when absent

Table 7

bound_to member	Type	Presence
subject_id	nonempty string	required
request_id	nonempty string	required
capability_id	nonempty string	required
tool_server	nonempty string	required
tool_name	nonempty string	required
parameter_hash	64 lowercase hexadecimal SHA-256 characters	required

Table 8

For this JSON-schema profile, a receiver **MUST** reject missing required members, unknown members at any of the three object levels, and values outside the listed types and ranges. A receiver **MUST NOT** default a missing request binding or infer it from another operation.

A v1 nonce signer **MUST** sign the typed canonical JSON of the complete nonce body, including each optional body member that is present, without a domain prefix. The signature is beside nonce, not inside it. The signer and verifier **MUST** use the kernel's configured nonce signing authority and accepted suite policy. A nonce identifier **MUST** be unique within that authority's lifetime; UUIDv7 is one implementation choice. The issuer **MUST** bound the lifetime by the operation's remaining authority. A 30-second lifetime is suitable only when consistent with the selected execution profile and its clock assumptions.

A v1 nonce verifier for a presented tool-call request **MUST** reject any schema other than `chio.execution_nonce.v1`, a time before `issued_at` or at or after `expires_at`, an empty `request_id`, a mismatch in any of the six binding fields against the independently derived expected request, or an invalid signature under the kernel key. Before accepting that execution, it **MUST** reserve the nonce identifier exactly once in its replay store and reject a replay or replay-store failure. The consumed marker remains needed through the signed expiry.

7.7.1. Operation-Owned Nonces

A `chio.execution_nonce.v2` signer signs the canonical JSON of an object with exactly three members:

Member	Value
<code>schema</code>	<code>chio.admission-execution-nonce-signature.v1</code>
<code>operation_id</code>	Authenticated durable admission operation identifier
<code>nonce</code>	Complete v2 nonce body, including all present optional members

Table 9: Operation-Owned Nonce Signing Input

There is no additional domain prefix. The wire envelope remains nonce and signature; the signing wrapper is reconstructed from trusted admission state and is not supplied by the agent. The operation identifier names a binding that includes the authenticated namespace, capability artifact, request, policy, and effect class. The verifier **MUST** resolve that binding independently and check all six nonce request fields, the trusted signer, issuance interval, and current expiry. An identifier chosen in untrusted metadata cannot select a different operation for verification.

The kernel **MUST** atomically reserve and consume the nonce under its original operation's durable ownership. Fresh operation-owned admission **MUST** reject v1, even if a legacy replay cache reports the nonce unused. A v1 verifier **MUST** reject v2 before consuming any replay marker. Relabeling the schema does not convert a signature between these profiles. Historical v1 reservations and terminal decisions remain evidence, not renewed execution authority; cleanup **MUST** retain any tombstones required to prevent their reuse.

7.7.2. Caller Execution and Spend Evidence

A caller-reserved nonce is an admission artifact, not a dispatch credential. An execution profile that delegates the external effect to a caller **MUST** separately authenticate that executor and bind dispatch to the original operation, capability digest, target, parameters digest, reservation, executor identity and key epoch, and validity interval. The executor **MUST** authenticate the kernel's dispatch authorization and durably claim the operation before performing the effect. Reconciliation **MUST** authenticate the executor's report and bind it to that same authorization. A caller-supplied cost or an unsigned assertion of success is insufficient to release captured exposure or claim verified execution.

The kernel **MUST** retain the original dispatch authorization and report identity for recovery. An uncertain committed operation remains subject to [Section 3.4](#); a missing report does not authorize refund or redispach. The output of a caller-executed operation passes the same release checks as native tool output. The executor's signed report attests to that executor's account, not independently to the provider's external state. The executor-authorization and report wire formats belong to a separately agreed execution profile; the execution nonce defined here does not supply those messages.

A consumer recognizing the legacy `chio.mediated_spend.v1` receipt predicate with a `chio.execution_nonce.v1` nonce **MUST** reject the receipt unless every following condition holds:

1. The receipt signature is valid under its embedded `kernel_key` and that key belongs to the consumer's admitted kernel signer set.
2. `receipt_kind` is `mediated_decision`, `boundary_class` is `prevent`, `trust_level` is `mediated`, the decision is `allow`, and `observation_outcome` is absent.
3. `metadata.budget_authority` contains a nonempty `hold_id` and `mediated_spend.profile` equal to `chio.mediated_spend.v1`.
4. `authorize.exposure_units` is positive. `terminal.disposition` is `reconciled` and `terminal.event_id` is present.
5. `execution_nonce_id` equals the presented signed nonce's `nonce_id`. If that nonce has `reserved_hold_id`, it equals `hold_id`.
6. The nonce's `capability_id`, `tool_server`, `tool_name`, and `parameter_hash` equal the receipt's corresponding call fields.
7. The nonce signature verifies under the same admitted `kernel_key`.

This predicate does not independently check nonce `subject_id`, `request_id`, `schema`, `expiry`, `replay` state, or the guarantee floor. It also does not query a budget store or quorum and does not require a `budget_commit_index`. Its terminal realized accounting amount is `terminal.realized_spend_units`, which can differ from the reported `financial.cost_charged`. Execution-time nonce verification and guarantee-floor checking are separate checks; successful evaluation of this predicate alone does not establish them.

7.8. Metering Metadata

The receipt's financial and `budget_authority` metadata are inside the signed receipt projection. A producer of a financial block **MUST** include the required fields below and omit an optional field that has no value.

financial member	Type	Presence	Meaning
grant_index	unsigned 32-bit integer	required	Matching grant index.
cost_charged	unsigned 64-bit integer	required	Recorded financial amount; can exceed ledger-realized units on a reported overrun.
currency	string	required	Recorded denomination. A zero-exposure invocation reconciliation uses an empty string.
budget_remaining	unsigned 64-bit integer	required	Recorded remaining amount. Reconciled paths subtract committed units from the recorded total, saturating at zero; an exhaustion denial can record the total without reading current usage.
budget_total	unsigned 64-bit integer	required	Recorded grant ceiling or path-specific bounded envelope. The inline path uses the maximum u64 value when no total ceiling exists.
delegation_depth	unsigned 32-bit integer	required	Recorded delegation-chain depth.
root_budget_holder	string	required	Recorded holder identifier; inline receipts use the capability issuer key encoding.
settlement_status	string	required	not_applicable, pending, settled, or failed.
payment_reference	string	omitted when absent	Optional external reference; settlement rails are outside this document.
cost_breakdown	JSON value	omitted when absent	Optional itemized report.
oracle_evidence	object	omitted when absent	Optional conversion evidence; verification of that evidence is outside this section.

financial member	Type	Presence	Meaning
attempted_cost	unsigned 64-bit integer	omitted when absent	Attempted amount on a denial.

Table 10

A producer of a full hold-lineage budget_authority block **MUST** use the member names and presence rules below. A label-only denial can carry only guarantee_level, authority_profile, and metering_profile; that reduced block does not satisfy the authoritative-spend profile.

In the full hold-lineage block, authority_profile is authoritative_hold_event, metering_profile is max_cost_preauthorize_then_reconcile_actual, and guarantee_level is one of the identifiers in [Section 7.6](#).

budget_authority member	Type	Presence in full block
guarantee_level	string	required
authority_profile	string	required
metering_profile	string	required
hold_id	string	required
budget_term	string	omitted when absent
authority	object	omitted when absent
authorize	object	required
invocation_capture	object	omitted when absent
terminal	object	omitted when absent
execution_nonce_id	string	omitted when absent
mediated_spend	object containing profile	omitted when absent; profile is chio.mediatespend.v1 when included

Table 11

authority member	Type	Presence when authority is included
authority_id	string	required
lease_id	string	required
lease_epoch	unsigned 64-bit integer	required

Table 12

authorize member	Type	Presence
exposure_units	unsigned 64-bit integer	required
committed_cost_units_after	unsigned 64-bit integer	required
event_id	string	omitted when absent
budget_commit_index	unsigned 64-bit integer	omitted when absent

Table 13

terminal member	Type	Presence when terminal is included
disposition	string	required; reference emitted values are reconciled, reversed, and cancelled_before_dispatch
exposure_units	unsigned 64-bit integer	required
realized_spend_units	unsigned 64-bit integer	required
committed_cost_units_after	unsigned 64-bit integer	required
event_id	string	omitted when absent
budget_commit_index	unsigned 64-bit integer	omitted when absent

Table 14

A successful invocation_capture object records invocation_count_after and includes event_id and budget_commit_index when available. An ambiguous-capture denial instead records event_id with invocation_capture_ambiguous and admission_retained set to true. These are distinct projections of capture state.

When a producer links a v1 nonce to a `chio.mediated_spend.v1` block, it **MUST** include both `execution_nonce_id` and `mediated_spend.profile`. A v2 nonce **MUST NOT** be labeled as satisfying that legacy predicate merely because it occupies the same transport envelope. Receipt consumers distinguish reported financial amounts from the committed authorize, capture, and terminal lineage. The typed financial and budget_authority decoders do not universally reject unknown members; these metadata blocks are not a strict unknown-member-rejection profile.

Auxiliary metering records can measure compute time, data volume, API cost, warehouse queries, and custom dimensions. They are not automatically attached to kernel receipts and do not substitute for the durable hold contract in this section.

8. Governed Transactions

A governed transaction is a tool call that carries a declared intent: a JSON object that names the call's target and purpose and can bound its cost, name a seller, carry a price quote, and assert the call chain the call belongs to. When a grant requires it, the call also carries an approval, signed by one trusted approver or by a quorum of eligible approvers. The kernel checks the intent against the call and the grant, verifies the approval, classifies the asserted call chain, and records the intent hash and its findings in the receipt.

A `tool_call_request` (Section 10.2) carries these objects in four **OPTIONAL** members, each omitted when absent:

`governed_intent`: The intent (Section 8.2).

`approval_token`: A single approval (Section 8.4).

`approval_tokens`: An array of 1 to 32 approval tokens that vote on a threshold proposal (Section 8.5). It is omitted when empty.

`threshold_approval_proposal`: The proposal that the tokens in `approval_tokens` vote on.

The MCP binding carries the same objects in `_meta` (Section 11.6). An approval binds to the call's request identifier: the id of a native `tool_call_request`, or `_meta.chioRequestId` on the MCP binding.

An agent **MUST NOT** send both `approval_token` and a non-empty `approval_tokens`. An agent that sends `threshold_approval_proposal` **MUST** send a non-empty `approval_tokens`, and an agent that sends a non-empty `approval_tokens` **MUST** send `threshold_approval_proposal`. The kernel denies a call that breaks these rules.

8.1. Applying the Checks

The kernel applies the checks in this section to a grant that matches the call ([Section 5.8](#)) when the grant carries any of the constraints `governed_intent_required`, `require_approval_above`, `seller_exact`, `minimum_runtime_assurance`, or `minimum_autonomy_tier` ([Section 5.3](#)), or when the request carries any of the four members above. When the checks apply, the kernel **MUST** deny a call that carries no `governed_intent`.

The kernel runs the checks for each matching grant in turn, before it runs the guards for that grant. It does not use a grant whose checks fail, and it goes on to the next matching grant. If no matching grant passes, the kernel denies the call. It runs the checks again for the chosen grant immediately before dispatch.

Unless this section says otherwise, a governed validation failure **MUST** prevent dispatch. For authorization artifacts representable by the typed digest structures, the kernel returns a signed denial with decision `deny` and guard `kernel`; the native result is `policy_denied`. Structurally inconsistent approvals can fail receipt-metadata construction and return `internal_error` instead. A fallback error receipt lacks the governed context and is not authenticated under the configured kernel key ([Section 10.4](#)).

8.2. Governed Intent

The intent is a JSON object. Its own members and those of `commerce` have `snake_case` names. The members of `metered_billing`, `call_chain`, and `autonomy` have `camelCase` names. **OPTIONAL** members are omitted when absent.

Field	Type	Presence	Meaning
id	string	REQUIRED	Identifier chosen by the agent. The kernel copies it to the receipt.
server_id	string	REQUIRED	Target tool server.
tool_name	string	REQUIRED	Target tool.
purpose	string	REQUIRED	Free text; can be empty.
max_amount	MonetaryAmount object (Section 7.1)	OPTIONAL	Largest amount the intent authorizes.
commerce	object	OPTIONAL	Seller context (Table 16).
metered_billing	object	OPTIONAL	Settlement mode and quote (Section 8.2.4).
runtime_attestation	object	OPTIONAL	Runtime attestation evidence, in the form defined in Section 12.1 .
call_chain	object	OPTIONAL	Asserted call chain (Table 17).
autonomy	object	OPTIONAL	Autonomy tier (Section 8.2.3).
context	JSON value	OPTIONAL	Free-form context. Two keys are reserved (Section 8.7).
body	object	OPTIONAL	tool_invocation, bound_tool_invocation, or the companion active_response_plan profile. Omission means tool_invocation.

Table 15: Governed Intent Members

Field	Type	Presence	Meaning
seller	string	REQUIRED	Seller or payee that the call pays.
shared_payment_token_id	string	REQUIRED	Reference to the payment credential the seller will use.
settlement_destination_ref	string	OPTIONAL	Reference to the account that receives the payment.

Table 16: Commerce Members

Field	Type	Presence	Meaning
chainId	string	REQUIRED	Identifier of the call chain.
parentRequestId	string	REQUIRED	Request identifier of the parent call.
parentReceiptId	string	OPTIONAL	Receipt identifier of the parent call.
originSubject	string	REQUIRED	Subject that started the chain.
delegatorSubject	string	REQUIRED	Subject that delegated to the caller.

Table 17: Call Chain Members

The autonomy object has a **REQUIRED** tier, one of direct, delegated, and autonomous in increasing order, and an **OPTIONAL** string delegationBondId.

The following intent is illustrative; its values come from the tests of the reference implementation.

```
{
  "id": "intent-1",
  "server_id": "srv-pay",
  "tool_name": "charge",
  "purpose": "pay supplier",
  "max_amount": { "units": 500, "currency": "USD" },
  "commerce": {
    "seller": "merchant.example",
    "shared_payment_token_id": "spt_123",
    "settlement_destination_ref": "acct:merchant-primary"
  },
  "metered_billing": {
    "settlementMode": "allow_then_settle",
    "quote": {
      "quoteId": "quote-1",
      "provider": "meter.chio",
      "billingUnit": "1k_tokens",
      "quotedUnits": 12,
      "quotedCost": { "units": 300, "currency": "USD" },
      "issuedAt": 950,
      "expiresAt": 1300
    },
    "maxBilledUnits": 20
  },
  "call_chain": {
    "chainId": "chain-1",
    "parentRequestId": "req-parent-1",
    "parentReceiptId": "rc-parent-1",
    "originSubject": "origin-subject",
    "delegatorSubject": "delegator-subject"
  }
}
```

Its intent hash is 6584f466fdfd57b71861d268d2ffe45a48b179a5999ab9502cb58e935bdb875d.

8.2.1. Approval of Exact Arguments

An intent requesting approval of exact tool arguments **MUST** carry a body with `kind`: `bound_tool_invocation` and a value object containing exactly `capability_id` and `parameters_hash`. The first is the nonempty identifier of the authorizing capability. The second is SHA-256 over canonical JSON of the call's parameters, encoded as `0x` followed by 64 lowercase hexadecimal digits. The intent hash commits both values.

Before checking approval artifacts or dispatching, the kernel **MUST** compare those bindings with the authenticated capability and actual arguments. A mismatch **MUST** deny the call, including substitution of a different capability belonging to the same subject. Approval interfaces **MUST** show or otherwise bind the approved arguments and target to this intent, rather than relying solely on its free-text purpose.

An omitted body or `tool_invocation` retains the legacy intent-only binding. It **MUST NOT** be described as approval of exact arguments. A host requiring that property **MUST** reject a receiver that does not support the bound body; moving the digest into advisory metadata is not an

equivalent fallback. The body does not itself require approval: the matched grant's approval constraints determine that requirement. Authenticated session ownership remains a separate host responsibility.

An `active_response_plan` body selects the companion response-plan profile and its installed validator. An ordinary tool-call evaluator **MUST** reject that body rather than treat it as a tool invocation.

8.2.2. Intent Hash

The intent hash is the SHA-256 digest ([Section 4.2](#)), as 64 lowercase hexadecimal characters, of the canonical JSON serialization ([Section 4.1](#)) of the intent. It has no domain-separation prefix. It covers context, including any proof or token carried there.

The kernel computes the hash over the intent as it parsed it. It ignores members that this section does not define, so they are not hashed; an undefined member inside `max_amount` or `verifiedOutcome` makes the request malformed instead. It also omits a body whose kind is `tool_invocation`. Approval tokens, threshold proposals, and receipts carry this hash.

8.2.3. Intent Checks

When the checks apply, the kernel **MUST** verify the following and **MUST** deny the call if any item fails:

1. `server_id` and `tool_name` equal the target of the call.
2. body is absent, has kind `tool_invocation`, or is a valid `bound_tool_invocation` whose bindings pass [Section 8.2.1](#). An active-response operation instead requires its separately selected profile ([Section 15.8.4](#)).
3. If `runtime_attestation` is present, it passes the kernel's local verification of attestation evidence. This holds even when no grant requires attestation.
4. If `call_chain` is present:
 - `chainId`, `parentRequestId`, `originSubject`, and `delegatorSubject` each contain a character other than white space, and so does `parentReceiptId` when present;
 - `parentRequestId` differs from the call's request identifier;
 - if the capability token has a delegation chain ([Section 5.5](#)), `delegatorSubject` equals the public key string ([Section 4.4](#)) of the delegator of the last link, and `originSubject` equals that of the first link;
 - if the kernel evaluates the call within a parent request of the same session, `parentRequestId` equals that parent's request identifier;
 - a proof or token in context passes [Section 8.7](#).
5. If `commerce` is present, `seller` and `shared_payment_token_id` each contain a character other than white space, and `max_amount` is present. A `settlement_destination_ref` is not empty, has no leading or trailing white space, has at most 2048 characters, and has no control characters.

6. If autonomy is present, a direct tier carries no non-whitespace delegationBondId. The kernel trims a bond identifier for lookup and treats an empty result as absent. A delegated or autonomous tier requires call_chain, a runtime_attestation that the kernel accepts at tier attested or verified respectively, or higher, and a delegationBondId that the kernel resolves to a delegation bond it accepts for the call. Delegation bonds are outside the scope of this document.
7. If metered_billing is present, it passes [Section 8.2.4](#).
8. If the grant declares max_cost_per_invocation and the intent carries max_amount, the two have the same currency, and max_amount.units is at least the ceiling's units, the amount that the kernel holds before dispatch ([Section 7.3](#)).

8.2.4. Metered Billing

The metered_billing object carries a price quote for a tool that bills by usage:

Field	Type	Presence	Meaning
settlementMode	string	REQUIRED	must_prepay, hold_capture, or allow_then_settle.
quote	object	REQUIRED	The quote (Table 19).
maxBilledUnits	integer	OPTIONAL	Largest number of units the call can be billed.
verifiedOutcome	object	OPTIONAL	Reserved. A kernel denies an intent that carries it.

Table 18: Metered Billing Members

Field	Type	Presence	Meaning
quoteId	string	REQUIRED	Quote identifier.
provider	string	REQUIRED	Party that issued the quote.
billingUnit	string	REQUIRED	Unit of quotedUnits, for example 1k_tokens.
quotedUnits	integer	REQUIRED	Quoted number of units.
quotedCost	MonetaryAmount object	REQUIRED	Quoted price.
issuedAt	integer	REQUIRED	Issue time, Unix seconds.
expiresAt	integer	OPTIONAL	Expiry time, Unix seconds.

Table 19: Quote Members

The kernel **MUST** deny the call unless all of the following hold:

1. `quoteId`, `provider`, and `billingUnit` each contain a character other than white space, and `quotedUnits` is greater than 0.
2. `expiresAt`, when present, is greater than `issuedAt`, and the quote is valid at the current time `now`: `issuedAt <= now`, and `now < expiresAt` when `expiresAt` is present.
3. `maxBilledUnits`, when present, is at least `quotedUnits` and greater than 0.
4. Neither `verifiedOutcome` is present nor `billingUnit` is `verified_outcome`. This version of the protocol does not define outcome-based billing.
5. `quotedCost.currency` equals the currency of `max_amount` when present, and the currency of the grant's `max_cost_per_invocation`, or else of its `max_total_cost`, when the grant declares one.

The modes `hold_capture` and `allow_then_settle` add no checks; the kernel records them in the receipt ([Section 8.8](#)). For `must_prepay`, the kernel **MUST** also deny the call in each of these cases:

- the kernel is not configured to authorize payments;
- the grant declares `max_total_cost` but not `max_cost_per_invocation`;
- `quotedCost.units` exceeds the units of either ceiling that the grant declares, or `max_amount.units` is below `quotedCost.units`.

The quoted cost of a `must_prepay` intent counts toward `require_approval_above` ([Section 8.3](#)). When the kernel answers such a call with an execution nonce instead of executing it ([Section 10.2](#)), the call **MUST** carry a singular `approval_token`.

8.3. Intent Constraints

Three grant constraints ([Section 5.3](#)) govern intents and approvals. They do not affect whether a grant matches a call; the kernel enforces them in the checks of this section. A delegated grant keeps each of them unchanged ([Section 5.6](#)).

```
[
  { "type": "governed_intent_required" },
  { "type": "require_approval_above",
    "value": { "threshold_units": 5000 } },
  { "type": "seller_exact", "value": "merchant.example" }
]
```

`governed_intent_required`: The call **MUST** carry `governed_intent`. The kernel then applies every check in [Section 8.2.3](#).

`require_approval_above`: The call **MUST** carry an approval when its requested amount is greater than or equal to `threshold_units`. If a grant carries several of these constraints, the largest threshold applies. The requested amount, in minor units, is the largest of: the base; `quotedCost.units` when `settlementMode` is `must_prepay`; and 0. The base is the units of the grant's `max_cost_per_invocation` when the grant declares it, and otherwise the units of the

intent's `max_amount` when present. A `threshold_units` of 0 therefore always requires approval. The constraint carries no currency, and a smaller `max_amount` does not lower the requested amount when the grant declares a ceiling.

`seller_exact`: The intent **MUST** carry `commerce`, and `commerce.seller` **MUST** equal the constraint's value exactly, with no case folding or normalization. If a grant carries several of these constraints, the last one applies.

A commerce intent under a grant that charges per call also needs an approval. When the grant's `max_cost_per_invocation` has units greater than 0 and the intent carries `commerce`, the intent **MUST** carry `settlement_destination_ref`, and the call **MUST** carry an approval, whatever `require_approval_above` requires.

An approval is a singular token that passes [Section 8.4](#) or a threshold set that passes [Section 8.5](#). The kernel **MUST** deny a call that needs an approval and carries neither. The kernel verifies a presented approval even when no approval is required, and an approval that fails verification denies the call.

The constraints `minimum_runtime_assurance` and `minimum_autonomy_tier` also trigger the checks. For each, the largest value in a grant applies. The first requires a `runtime_attestation` that the kernel accepts at that tier or higher, in the order none, basic, attested, verified. The second requires an autonomy object whose tier is at least the required tier.

8.4. Approval Tokens

An approval token is a signed JSON object that approves one intent for one request by one subject. All member names are `snake_case`.

Field	Type	Presence	Meaning
id	string	REQUIRED	Token identifier.
approver	string	REQUIRED	Public key of the signer (Section 4.4).
subject	string	REQUIRED	Subject key of the capability token that authorizes the call.
governed_intent_hash	string	REQUIRED	Intent hash (Section 8.2.2).
request_id	string	REQUIRED	Request identifier of the call.
threshold_proposal_hash	string	OPTIONAL	Digest of the proposal that the token votes on (Section 8.5).
issued_at	integer	REQUIRED	Start of validity, Unix seconds.
expires_at	integer	REQUIRED	End of validity, exclusive, Unix seconds.
decision	string	REQUIRED	approved or denied.
algorithm	string	OPTIONAL	ed25519, p256, p384, or hybrid. Informational; omitted when absent or ed25519.
signature	string	REQUIRED	Signature by approver (Section 4.3).

Table 20: Approval Token Members

The approver signs the canonical JSON of the token without algorithm and signature ([Section 4.5](#)), with no domain-separation prefix. The token digest is the lowercase hexadecimal SHA-256 digest of the ASCII bytes `chio.governed-approval-token.v1`, one zero byte, and the canonical JSON of the whole token, signature included.

The following token is taken from the schema fixtures of the reference implementation. Its hashes and signature are placeholders. It is a vote on a threshold proposal; a singular token omits `threshold_proposal_hash`.

[illegible]

The kernel **MUST** verify a singular `approval_token` as follows, at the current time now, and **MUST** deny the call if a step fails:

1. `issued_at` $\leq$ `now` $\leq$ `expires_at`.
2. `request_id` equals the call's request identifier.
3. `governed_intent_hash` equals the intent hash.
4. `subject` equals the subject of the capability token.
5. `decision` is approved.
6. `approver` is the kernel's own key, a configured capability-authority key, or another issuer key trusted by the kernel ([Section 5.8](#)), and the signature verifies.
7. `expires_at` - `issued_at` is at most 3600 seconds.

The kernel does not check `id` or `threshold_proposal_hash` of a singular token.

A singular token authorizes one dispatch. When it reserves the call for dispatch, the kernel records the tuple of the subject key, the request identifier, and the intent hash until the token's `expires_at`. The kernel **MUST** deny the call as a replay if the tuple is already recorded and has not expired. Because the record is keyed by the tuple and not by the token's `id`, a second token for the same subject, request, and intent is also refused. The kernel releases an owned replay reservation only after definite pre-effect rejection with successful rollback. After acknowledged or ambiguous external payment authorization it retains the marker, even when later checks prevent tool dispatch. Uncertain cleanup can also retain it; a denial alone does not authorize replay. If the kernel cannot record the tuple, for example because its record store is full, it **MUST** deny the call. The kernel **MUST** retain these records durably across restarts and enforce uniqueness across all processes that can consume the approval. It **MUST NOT** discard a record while an accepted operation still requires it for recovery, even when the original approval has expired.

8.5. Threshold Approval

A threshold approval authorizes a call with a quorum of votes. A proposal, signed by a policy authority, fixes the request, the intent, the subject, the authorizing capability token, the policy, the quorum, the set of eligible approvers, and a deadline. Each vote is an approval token ([Section 8.4](#)) whose `threshold_proposal_hash` is the proposal's digest.

The kernel accepts a threshold set only when the feature `threshold_governed_approvals` is supported for the request's origin ([Section 5.9](#)). It **MUST** deny the call otherwise.

The kernel takes the threshold requirement for the call from its active policy, by policy hash, tool server, and tool. The policy language is out of scope ([Section 1.2](#)). A requirement has:

- a policy hash: 64 lowercase hexadecimal characters, equal to the kernel's active policy hash;
- an eligible set of 1 to 32 approvers, each a JSON object with the string members `identifier` and `public_key`. Identifiers are unique, not empty, free of leading and trailing white space and of control characters; public keys are unique;
- a threshold from 1 to the number of eligible approvers;
- a timeout from 1 to 3600 seconds.

The eligible set digest is the lowercase hexadecimal SHA-256 digest of the ASCII bytes `chio.approver-set.v1`, one zero byte, and the canonical JSON of the array of eligible approvers, sorted by `identifier` and then by `public_key`.

8.5.1. Proposal

A proposal is a JSON object with `snake_case` members. A receiver rejects a proposal with members that [Table 21](#) does not list.

Field	Type	Presence	Meaning
schema	string	REQUIRED	chio.threshold-approval-proposal.v1.
proposal_id	string	REQUIRED	Proposal identifier.
request_id	string	REQUIRED	Request identifier of the call.
governed_intent_hash	string	REQUIRED	Intent hash (Section 8.2.2).
subject	string	REQUIRED	Subject key of the capability token.
authorizing_capability_digest	string	REQUIRED	SHA-256 digest of the canonical JSON of the capability token, signature included, with no prefix.
policy_hash	string	REQUIRED	Policy hash of the requirement.
threshold	integer	REQUIRED	Quorum size.
eligible_set_digest	string	REQUIRED	Eligible set digest.
proposal_created_at	integer	REQUIRED	Start of the voting window, Unix seconds.
proposal_deadline	integer	REQUIRED	End of the voting window, exclusive, Unix seconds.
policy_authority	string	REQUIRED	Public key of the signer.
algorithm	string	OPTIONAL	As for approval tokens.
signature	string	REQUIRED	Signature by policy_authority.

Table 21: Threshold Proposal Members

The four digest members are 64 lowercase hexadecimal characters. `proposal_id` and `request_id` are not empty and have no leading or trailing white space. `threshold` is greater than 0, and `proposal_created_at` is less than `proposal_deadline`. The policy authority signs the canonical JSON of the proposal without `algorithm` and `signature`, with no prefix. The proposal digest is the SHA-256 digest, in lowercase hexadecimal, of the ASCII bytes `chio.threshold-approval-proposal.v1`, one zero byte, and the canonical JSON of the whole proposal, signature included.

The following proposal is taken from the schema fixtures of the reference implementation; its digests and signature are placeholders.

[illegible]

8.5.2. Verifying a Threshold Set

The kernel **MUST** verify a threshold set as follows, at the current time now, and **MUST** deny the call if a step fails:

1. `approval_tokens` holds 1 to 32 tokens, and the kernel has a threshold requirement for the call.
2. `threshold_approval_proposal` is present, its members are valid as stated above, and `proposal_created_at` $\leq$ `now` $<$ `proposal_deadline`.
3. `policy_authority` is the kernel's own key, a configured capability- authority key, or another issuer key trusted by the kernel ([Section 5.8](#)), and the signature verifies.
4. `request_id`, `governed_intent_hash`, and subject match the call as for a singular token. `authorizing_capability_digest` matches the capability token. `policy_hash`, `threshold`, and `eligible_set_digest` equal those of the requirement.
5. `proposal_deadline` equals the smaller of `proposal_created_at` plus the requirement's timeout and the capability token's `expires_at`.
6. Each token is valid at now; its `id` is not empty and has no leading or trailing white space; its `request_id`, `governed_intent_hash`, and subject match the call; its decision is approved; and its `threshold_proposal_hash` equals the proposal digest.
7. Each token's `issued_at` is at least `proposal_created_at` and less than `proposal_deadline`, and its `expires_at` is at most `proposal_deadline`.

8. Each token's approver is the public key of an eligible approver, and its signature verifies. The kernel's trusted issuer keys do not apply to votes.
9. No two tokens share an id, a token digest, or an approver. The kernel does not remove duplicates; a duplicate denies the call.
10. The number of approvers is at least threshold. Extra valid votes are allowed.

The kernel **MUST** resolve the threshold requirement from trusted policy and authenticated request context, not from the proposal alone. It **MUST** apply the configured signature floor to both the proposal and every vote. A collection service can assemble votes but cannot confer authority by declaring that a quorum was reached. The executing kernel performs the verification itself. On resumption, it **MUST** recheck current policy, revocation, validity, and signer eligibility against the original binding; it **MUST NOT** rewrite a pending proposal to fit changed policy.

A vote whose decision is denied fails step 6, so it denies the call rather than counting against the quorum.

The approval set hash identifies the verified set. Its input is a JSON object with the members `token_digests`, an array of the digests of all the tokens sorted in ascending order with no duplicates, and `threshold_proposal_hash`, the proposal digest, together with these members copied from the proposal: `policy_hash`, `threshold`, `eligible_set_digest`, `request_id`, `governed_intent_hash`, `subject`, `authorizing_capability_digest`, `proposal_id`, `proposal_created_at`, and `proposal_deadline`. The hash is the SHA-256 digest, in lowercase hexadecimal, of the ASCII bytes `chio.verified-approval-set.v1`, one zero byte, and the canonical JSON of that object. The order of the tokens in the request does not change it.

A proposal and its votes authorize one call. Before dispatch, the kernel **MUST** record the proposal digest and the approval set hash with its durable record of the call's admission, and **MUST** deny the call if it keeps no durable admission records. It **MUST NOT** record the same proposal, approval set hash, or token digest for a second call. When the kernel resumes an admission it has already recorded, both hashes **MUST** equal the recorded values.

8.5.3. Kernel-Issued Proposals

A kernel issues a proposal when cumulative budget approval is required ([Section 7.3](#)). It signs with its configured proposal authority, takes `proposal_created_at` from the recorded hold, and durably retains the proposal with the admission. If votes are absent, it **MUST NOT** dispatch. It returns `pending_approval` with the complete signed proposal ([Section 10.3](#)). The receipt records deny, with guard `kernel` and reason `cumulative approval required`; its `metadata.threshold_approval` contains `proposal_id`, `proposal_hash`, `proposal_deadline`, and `state: approval_required`. Here deny records that execution was not authorized in this exchange; the operation can still await approval.

A subsequent request carries the original request identifier, unchanged execution binding, exact retained proposal, and collected votes. The kernel **MUST** resume the original admission or reject the request; it **MUST NOT** create a second reservation for the same pending operation. A pending

response **MUST NOT** carry an execution nonce or tool output. A client **MUST** verify the proposal's signature and binding before presenting it to approvers. The MCP projection is defined in [Section 11.6](#).

8.6. Provenance Classes

A `call_chain` is the agent's assertion. The intent has no member that states its strength; the kernel assigns an evidence class when it writes the receipt:

asserted: The kernel checked only the rules of [Section 8.2.3](#).

observed: At least one local evidence source in [Table 22](#) matched the call chain.

verified: An upstream proof in context passed its checks ([Section 8.7](#)).

Source	Condition
<code>session_parent_request_lineage</code>	The kernel evaluates the call within a parent request of the same session whose request identifier equals <code>parentRequestId</code> .
<code>local_parent_receipt_linkage</code>	<code>parentReceiptId</code> names a receipt that the kernel holds.
<code>capability_delegator_subject</code>	<code>delegatorSubject</code> equals the key of the last delegator in the capability token's delegation chain.
<code>capability_origin_subject</code>	<code>originSubject</code> equals the key of the first delegator.
<code>upstream_delegator_proof</code>	A <code>callChainUpstreamProof</code> passed its checks.

Table 22: Call-Chain Evidence Sources

When a grant has been selected and its validated evidence is retained for the receipt, the kernel **MUST** assign `verified` if and only if a `callChainUpstreamProof` passed its checks. Otherwise it **MUST** assign `observed` if at least one source matched, and `asserted` if none did. No other evidence raises the class. In particular, a `callChainContinuation` that passes every check adds no source of its own: the kernel records its `tokenId`, and the class stays at `observed` or `asserted` as the other sources decide.

The sources are narrow. `local_parent_receipt_linkage` means only that the identifier resolves to a stored receipt. For a delegated capability token, the rules of [Section 8.2.3](#) already force both subject sources to match, so its call chain is at least `observed`. A receipt for a call that the kernel denied because no grant passed its checks carries the class `asserted`.

8.7. Call-Chain Continuation

Two keys of the intent's context carry evidence for a call_chain: `callChainUpstreamProof` and `callChainContinuation`. The kernel reads them only when context is a JSON object and `call_chain` is present. A key whose value is null counts as absent, and a value that does not parse as the object below denies the call. If `callChainContinuation` is present, the kernel validates it and ignores `callChainUpstreamProof`. Both objects have camelCase members, and each is signed by its signer over the canonical JSON of all its members except signature, with no domain-separation prefix.

An upstream proof is a statement by the delegator that handed the call chain to the caller. It has the call-chain members of [Table 17](#) together with `signer` and `subject` (public key strings), `issuedAt` and `expiresAt` (integers, Unix seconds), and `signature`. The kernel **MUST** deny the call unless the signature verifies; `issuedAt` $\leq$ `now` $<$ `expiresAt`; `subject` equals the capability token's subject; the capability token has a delegation chain and `signer` equals the delegator of its last link; and the five call-chain members equal those of the intent's `call_chain`, with `parentReceiptId` absent from both or equal.

A continuation token carries call-chain context from a parent call to a child call:

Field	Type	Presence	Meaning
schema	string	REQUIRED	Continuation schema.
tokenId	string	REQUIRED	Token identifier.
signer	string	REQUIRED	Public key of the signer.
subject	string	REQUIRED	Subject key of the child's capability token.
chainId, parentRequestId, originSubject, delegatorSubject	string	REQUIRED	As in Table 17 .
parentReceiptId	string	OPTIONAL	As in Table 17 .
parentReceiptHash	string	OPTIONAL	SHA-256 digest of the canonical JSON of the parent receipt, signature included, with no prefix.
parentSessionAnchor	object	OPTIONAL	Reference to the parent's session anchor: <code>sessionAnchorId</code> and <code>sessionAnchorHash</code> , both strings.
currentSubject	string	REQUIRED	Public key string of the child's subject.
parentCapabilityId	string	OPTIONAL	<code>capability_id</code> of the last delegation link.
delegationLinkHash	string	OPTIONAL	SHA-256 digest of the canonical JSON of the last delegation link without its signature.
governedIntentHash	string	OPTIONAL	Intent hash of the child call.
audience	object	OPTIONAL	<code>serverId</code> and <code>toolName</code> of the child call, both strings.
nonce	string	OPTIONAL	Not checked.
issuedAt, expiresAt	integer	REQUIRED	Validity interval, Unix seconds.
signature	string	REQUIRED	Signature by signer.

Table 23: Continuation Token Members

The kernel **MUST** verify a continuation token as follows and **MUST** deny the call if a step fails:

1. `schema` is `chio.call_chain_continuation.v1`, and the signature verifies.
2. `issuedAt` $\leq$ `now` $<$ `expiresAt`.
3. `subject` equals the capability token's subject, and `currentSubject` equals its public key string.
4. `signer` is the kernel's own key, one of its trusted issuer keys ([Section 5.8](#)), or the delegator of the last link of the capability token's delegation chain.
5. The five call-chain members equal those of the intent's `call_chain`.
6. Each **OPTIONAL** binding that is present matches: `audience` names the call's server and tool; `governedIntentHash` equals the intent hash; `parentCapabilityId` and `delegationLinkHash` match the last delegation link, and a token that carries either one requires a delegated capability token.
7. If `parentReceiptId` names a receipt that the kernel holds, that receipt's signature verifies under the kernel's algorithm policy ([Section 6.9](#)). If it names no such receipt, the token carries neither `parentReceiptHash` nor `parentSessionAnchor`. A token without `parentReceiptId` carries no `parentReceiptHash`.
8. `parentReceiptHash`, when present, equals the digest of the parent receipt.
9. `parentSessionAnchor`, when present, equals the kernel's reference for the parent session: that of the live parent session when the kernel evaluates the call within a parent request, and otherwise the `sessionAnchorId` and `sessionAnchorHash` that the parent receipt's metadata carries. The kernel denies the call when it has no such reference.

An embedded token's `governedIntentHash` would bind an intent containing that same signed token. This document defines no construction for that self-bound form; senders omit this member on a token embedded in context. The kernel does not use `nonce` or `tokenId` to detect replay. It records `tokenId` in the receipt, and the anchor's `sessionAnchorId` when step 9 applied.

This document specifies how a kernel verifies these objects, not when a kernel issues them. Carrying call chains between operators is outside its scope ([Section 1.2](#)).

8.8. Governed Context in Receipts

For a normal signed result carrying `governed_intent`, the kernel adds `governed_transaction` to receipt metadata, including on denial. The malformed-authorization error path in [Section 8.1](#) is an exception. Its members are:

Field	Type	Presence	Meaning
intent_id, purpose, server_id, tool_name	string	REQUIRED	Copied from the intent.
intent_hash	string	REQUIRED	Intent hash (Section 8.2.2).
max_amount, commerce	object	OPTIONAL	Copied from the intent.
metered_billing	object	OPTIONAL	settlementMode, quote, and maxBilledUnits, copied from the intent.
approval	object	OPTIONAL	The approval the call carried.
runtime_assurance	object	OPTIONAL	Present when the kernel accepted runtime_attestation.
call_chain	object	OPTIONAL	The classified call chain.
autonomy	object	OPTIONAL	Copied from the intent.
economic_authorization	object	OPTIONAL	Payment and settlement terms; outside the scope of this document.

Table 24: Governed Transaction Receipt Members

The approval object has snake_case members. For a singular token, token_id is its id, approver_key its approver, approval_artifact_digest its token digest, and approved is true when its decision is approved. For a threshold set, token_id is the proposal_id, approver_key the policy_authority, approval_artifact_digest the approval set hash, and approved is true. The object describes what the call carried. On a receipt whose decision is allow, the approval passed verification; on a denial, it might not have.

The runtime_assurance object has the camelCase members schema, tier, verifier, evidenceSha256, and the **OPTIONAL** verifierFamily and workloadIdentity.

The call_chain object has camelCase members: evidenceClass ([Section 8.6](#)); evidenceSources, an array of the matched sources, omitted when empty; upstreamProof, the proof that passed its checks; continuationTokenId and sessionAnchorId from a continuation token; and the five call-chain members, copied from the intent unchanged.

When the class is asserted, the kernel also adds the member governed_transaction_diagnostics beside governed_transaction. Its member assertedCallChain holds a copy of the call_chain object. The kernel also adds the diagnostics member when a continuation token supplied a session anchor; its member lineageReferences then holds sessionAnchorId.

When the grant charges per call and the intent carries commerce, the kernel binds the seller, the settlement destination, the intent hash, and the approval's digest together in `economic_authorization`.

9. Receipt Checkpoints

A checkpoint signs a commitment to a local receipt batch and, for v2, can commit the sequence of checkpoint batch roots. It supports audit evidence and checked continuity over supplied statements. It does not prove that a publisher disclosed every view it signed.

9.1. Checkpoint Statement

The signed object has two required members, body and signature. A verifier **MUST** reject any unknown member in either the envelope or the body. New issuance uses `chio.checkpoint_statement.v2`. Legacy `chio.checkpoint_statement.v1` remains accepted for verification.

Body field	Type	Presence
schema	String	REQUIRED
checkpoint_seq	Unsigned 64-bit integer	REQUIRED
batch_start_seq	Unsigned 64-bit integer	REQUIRED
batch_end_seq	Unsigned 64-bit integer	REQUIRED
tree_size	Positive integer	REQUIRED
merkle_root	32-octet hash	REQUIRED
issued_at	Unsigned 64-bit integer	REQUIRED
kernel_key	Public-key string (Section 4.4)	REQUIRED
previous_checkpoint_sha256	String	OPTIONAL
chain_root	32-octet hash	OPTIONAL

Table 25: Checkpoint Statement Body

Body field	Meaning
schema	Exactly <code>chio.checkpoint_statement.v1</code> or <code>chio.checkpoint_statement.v2</code> .
checkpoint_seq	Positive checkpoint number.
batch_start_seq	Positive first claim-log entry in the batch.
batch_end_seq	Last claim-log entry in the batch, not less than its first.
tree_size	Number of leaves in this batch, not a cumulative log size.
merkle_root	Root of the receipt batch tree.
issued_at	Positive Unix timestamp in seconds.
kernel_key	Key used to verify the statement signature.
previous_checkpoint_sha256	SHA-256 of the preceding canonical body, exactly 64 lowercase hexadecimal characters without <code>0x</code> .
chain_root	Commitment to the checkpoint chain, subject to Section 9.2 .

Table 26: Checkpoint Statement Body Meanings

The two **OPTIONAL** fields default to absence. A producer **MUST** omit an absent field, and a verifier **MUST** reject explicit null for either. The body has no `log_id`. Producers **MUST** encode `merkle_root` and `chain_root` as `0x` followed by 64 lowercase hexadecimal characters. The compatibility decoder also accepts typed hashes without the `0x` prefix and accepts hexadecimal digits in either case; typed reserialization restores the producer form. Key and signature input compatibility follows [Section 4.4](#).

A verifier **MUST** reject an unsupported schema, a zero checkpoint number, zero batch start, inverted range, zero tree size, zero timestamp, or `tree_size` unequal to `batch_end_seq - batch_start_seq + 1`. It **MUST** reject a present predecessor digest that does not have the exact form in the table. Count arithmetic that overflows is invalid.

The signer **MUST** sign the canonical typed encoding of body alone, without the envelope signature or an additional domain prefix. A verifier **MUST** reconstruct that typed body, serialize it according to [Section 4.1](#), and verify the signature using `kernel_key` over those bytes. It does not require the received JSON bytes to equal the signed bytes. The reference issuer uses Ed25519; the generic verifier dispatches on matching encoded key and signature suites. Its ordinary Ed25519 check does not imply the additional weak-key checks of a strict-verification profile.

`tree_size` is represented as a machine-sized unsigned integer in the reference implementation. Checkpoint sequence numbers used as chain-tree sizes must also fit that implementation's addressable size. These are implementation bounds; the typed integer compatibility rules in [Section 4.1](#) also apply to checkpoint bodies.

9.2. Merkle Trees

For the local claim-log checkpointing profile, a checkpoint issuer **MUST** select a gap-free inclusive range ordered by claim-log entry sequence. For each `tool_receipt` or `child_receipt`, leaf data **MUST** be the canonical typed JSON of the full verified signed receipt, including its signature and kernel key. Other entry kinds are invalid for this batch projection. Receipt identifiers alone are not the leaf data.

For leaf data `b`, an implementation **MUST** compute its leaf hash as `SHA-256(0x00 || b)`. It **MUST** compute a parent as `SHA-256(0x01 || left || right)`, where each child is its raw 32-octet hash. For a subtree of $n > 1$ leaves, the recursive split is the largest power of two strictly less than n , as in [RFC9162]. An unpaired rightmost node advances unchanged. A single-leaf root is its leaf hash; an empty receipt batch is invalid.

The checkpoint-chain leaf is canonical JSON of exactly `checkpoint_seq`, `batch_start_seq`, `batch_end_seq`, and `merkle_root`. Its canonical member order is `batch_end_seq`, `batch_start_seq`, `checkpoint_seq`, `merkle_root`. It excludes every other checkpoint field. Its hash is `SHA-256(0x00 || canonical_chain_leaf_bytes)`. An issuer carrying `chain_root` **MUST** compute it by combining the already hashed chain leaves for checkpoint 1 through `checkpoint_seq`, in sequence order, with the parent hash above, without hashing them as leaves again. The chain tree's size is the checkpoint count.

A v1 checkpoint producer **MUST** omit `chain_root`. A v2 checkpoint 1 producer **MUST** carry a `chain_root` equal to its own chain-leaf hash. A detached later v2 statement can pass standalone validation without a chain root; a verifier **MUST** reject its use as a successor in a predecessor pair. For the local store profile, the first persisted checkpoint **MUST** have checkpoint number 1, batch start 1, and no predecessor digest. Those store-genesis rules are separate from standalone statement validation.

A local store accepting a checkpoint **MUST** verify the covered receipts, require every receipt's `kernel_key` to equal the checkpoint's `kernel_key`, and require the rebuilt batch's leaf count and root to equal the signed `tree_size` and `merkle_root`. A mixed-signer receipt batch is invalid for this store profile.

9.3. Predecessor and Consistency Proofs

A predecessor-pair verifier **MUST** validate both statements and their signatures. It **MUST** require the successor's checkpoint number to equal the predecessor's plus 1, its batch start to equal the predecessor's batch end plus 1, and its `previous_checkpoint_sha256` to equal the SHA-256 digest of the predecessor's canonical typed body, excluding signature. Overflow is invalid. It

MUST reject a successor that drops an existing chain commitment, and a v2 successor without a chain root. Predecessor linkage alone does not prove Merkle prefix extension or require increasing timestamps.

The consistency record is unsigned. Its fields are:

Field	Type	Parsing presence/default
schema	String	REQUIRED
log_id	String	REQUIRED
from_checkpoint_seq	Unsigned 64-bit integer	REQUIRED
to_checkpoint_seq	Unsigned 64-bit integer	REQUIRED
from_checkpoint_sha256	String	REQUIRED
to_checkpoint_sha256	String	REQUIRED
from_log_tree_size	Unsigned 64-bit integer	REQUIRED
to_log_tree_size	Unsigned 64-bit integer	REQUIRED
appended_entry_start_seq	Unsigned 64-bit integer	REQUIRED
appended_entry_end_seq	Unsigned 64-bit integer	REQUIRED
from_chain_root	Hash or null	OPTIONAL ; absent by default
to_chain_root	Hash or null	OPTIONAL ; absent by default
chain_proof_hashes	Array of hashes	OPTIONAL ; defaults to []
from_leaf_inclusion	Merkle inclusion object or null	OPTIONAL ; absent by default
to_leaf_inclusion	Merkle inclusion object or null	OPTIONAL ; absent by default

Table 27

Field	Valid v2 meaning
schema	chio.checkpoint_consistency_proof.v2.
log_id	Derived log identifier of both endpoints.
from_checkpoint_seq	Earlier checkpoint number.
to_checkpoint_seq	Immediately following checkpoint number.
from_checkpoint_sha256	Earlier canonical body digest, 64 lowercase hex characters without 0x.
to_checkpoint_sha256	Later canonical body digest, in the same form.
from_log_tree_size	Earlier endpoint's batch_end_seq.
to_log_tree_size	Later endpoint's batch_end_seq.
appended_entry_start_seq	Later endpoint's batch_start_seq.
appended_entry_end_seq	Later endpoint's batch_end_seq.
from_chain_root	Earlier signed chain root, required for successful v2 verification.
to_chain_root	Later signed chain root, required for successful v2 verification.
chain_proof_hashes	RFC 9162 consistency path.
from_leaf_inclusion	Earlier endpoint's own chain leaf at its final position.
to_leaf_inclusion	Later endpoint's own chain leaf at its final position.

Table 28

The first ten fields have no parsing defaults. A verifier **MUST** reject unknown members of the outer record. Producers omit absent roots and inclusions and an empty chain_proof_hashes. The compatibility parser accepts explicit null for the four optional roots/inclusions, but not for the array. Its nested inclusion objects have the format in [Section 9.4](#) and accept unknown members. Parsing acceptance does not establish a valid v2 proof.

The log identifier is local-log- followed by 64 lowercase hexadecimal characters: SHA-256 of the raw Ed25519 key bytes, or of the UTF-8 bytes of the normalized prefixed public-key encoding for another suite. This proof profile supports adjacent checkpoints under one derived log identifier. A consistency verifier **MUST** reject a pair with different derived identifiers, and **MUST** compare every metadata field against the values recomputed from the signed bodies.

For v2 verification, both signed statements **MUST** carry chain roots. The verifier **MUST** require the record's roots to equal them and both endpoint inclusion objects to be present. For each endpoint, it **MUST** require an inclusion tree size equal to that endpoint's checkpoint number, an index equal to that number minus 1, and a path proving that endpoint's own chain-leaf hash against its signed chain root.

The verifier **MUST** establish the earlier prefix by one of these methods:

- Genesis: the earlier endpoint is checkpoint 1.
- A chain root previously accepted for the earlier endpoint: it equals the earlier signed root.
- The ordered chain-leaf hashes from checkpoint 1 through the earlier endpoint, including it: their count equals its checkpoint number and their root equals its signed chain root.

An application using a previously accepted root or prefix supplies it as verifier input; the unsigned proof record cannot establish that trust. A mid-chain pair without such input is invalid. The verifier **MUST** then verify the Merkle consistency path using the two checkpoint numbers as tree sizes and the two signed chain roots, following [RFC9162], Section 2.1.4.2. Structural input failures are errors; unsuccessful metadata, root, inclusion, anchor, or path comparisons are failed proofs.

Legacy `chio.checkpoint_consistency_proof.v1` carries metadata continuity only. A verifier **MUST NOT** treat it as a cryptographic prefix proof. Successful legacy verification requires two v1 statements, decoded absence of both chain roots and endpoint inclusions, and an empty decoded `chain_proof_hashes`. Omission and explicit null both represent absence for its four optional fields; omission and `[]` both represent an empty path. Prefix anchors are not evaluated for legacy records.

9.4. Inclusion Proofs

The Merkle inclusion object has three required members:

Field	Type	Meaning
<code>tree_size</code>	Positive integer	Number of leaves in the tree being proved.
<code>leaf_index</code>	Non-negative integer	Zero-based leaf position.
<code>audit_path</code>	Array of 32-octet hashes	Sibling hashes in order from leaf toward root.

Table 29

`audit_path` has no parsing default and remains present when empty. The compatibility decoder accepts unknown members. Counts and indices use machine-sized unsigned integers in the reference implementation. Hash serialization and compatibility parsing follow [Section 9.1](#).

An inclusion verifier **MUST** reject a zero tree size or an index not less than that size. It **MUST** compute the leaf hash from the specified bytes and combine the supplied siblings using [Section 9.2](#). A carried rightmost node consumes no sibling at that level. It **MUST** reject a path with missing or unused hashes and accept membership only when the computed root equals the expected root.

The evidence-export wrapper is unsigned and has no schema identifier. It has these five required members, none with a parsing default:

Field	Type	Meaning
checkpoint_seq	Unsigned 64-bit integer	Referenced checkpoint number.
receipt_seq	Unsigned 64-bit integer	Receipt's claim-log entry sequence claimed by the wrapper.
leaf_index	Non-negative integer	Receipt's claimed position in this batch.
merkle_root	32-octet hash	Advertised batch root.
proof	Merkle inclusion object	Nested path and tree parameters.

Table 30

Its compatibility decoder accepts unknown members. Basic wrapper verification **MUST** require matching wrapper and nested indices and verify the canonical receipt bytes against the expected root supplied by the caller. It does not independently authenticate the wrapper's sequence metadata or advertised root.

An evidence-package verifier **MUST** resolve the referenced checkpoint and receipt, compare the wrapper root to the signed checkpoint root, require the wrapper index to be below the checkpoint's batch size, and require receipt_seq to fall within its inclusive batch range. It **MUST** reject two proofs for the same receipt sequence, verify the canonical receipt bytes against the signed root, and require the number of exported tool receipts without successful proofs to equal the manifest's uncheckpointed count.

The verifier **MUST** also require `proof.tree_size == checkpoint.body.tree_size` and `leaf_index == receipt_seq - checkpoint.body.batch_start_seq`, after checking the sequence range. It **MUST** reject arithmetic overflow and inconsistent batch bounds. These checks bind membership to the claimed position, not merely to an unsigned wrapper containing a valid path.

The current export emits tool-receipt proofs. Child receipts can be batch leaves, while child-receipt inclusion-proof export remains outside this export profile. Uncovered tool receipts are reported explicitly; absence of a proof establishes no membership.

9.5. Checkpoint Verification

A consumer that treats a checkpoint as evidence from an expected kernel **MUST** check its `kernel_key` against a separately configured trusted-key policy. Signature validity under a key carried in the statement does not establish that key's authority. The consumer **MUST** apply its trusted signer roles, accepted suites, and any required signer validity interval before accepting the statement. A checkpoint timestamp by itself cannot prove that a signature preceded a key compromise.

A checkpoint-set verifier for this profile **MUST** validate every statement and signature, reject duplicate checkpoint numbers and observed conflicting statements, and require each predecessor digest to resolve within the supplied set and pass predecessor validation. A checkpoint without a predecessor **MUST** be checkpoint 1 with batch start 1. A cited predecessor absent from the set is invalid. The reference set verifier requires the complete prefix through checkpoint 1 and has no separately pinned-boundary input; the pair-level anchor interface in [Section 9.3](#) is a distinct verification path.

Scoped evidence exports retain the prefix through the newest checkpoint covering the selected receipts. Inclusion and consistency verification uses the roots in the validated signed bodies, rather than relying on an unsigned advertised root. Consumer signer policy determines which of those signed bodies is accepted as evidence from the intended kernel.

9.5.1. Retention and Evidence Availability

A receipt, its sequence, and the checkpoint covering that sequence **MUST** be read from a consistent authenticated state. A verifier **MUST NOT** combine a receipt from one log view with a root or sequence assignment from another. An uncovered tail is uncheckpointed evidence, even if its receipts have valid signatures.

Archival **MUST** preserve original signed bytes, sequence identities, checkpoint links, and the information needed to verify retained proofs. Deleting payloads under a retention policy does not authorize renumbering or rewriting history. A service **MUST** distinguish unavailable evidence from a valid proof of absence. It **MUST NOT** reconstruct a missing history as a new empty log under the old identity.

Replay records, unresolved operations, consumed approvals, and accounting obligations have security lifetimes independent of receipt presentation policies. Retention **MUST NOT** remove state still required to prevent replay or recover an operation. A recovery process **MUST** authenticate retained records and reject rollback to an older authority state before resuming execution.

9.6. Claim Limits

The checkpoint surface provides local signed audit evidence and verified continuity for the supplied, anchored views. It does not establish public append-only publication, complete receipt-family sequencing, or strong non-repudiation. A verifier can detect conflicting statements that it

receives; it cannot detect a hidden competing view from these bytes alone. External witnessing and publication policies need separate protocols and qualification and are outside this document.

10. Native Transport

The native transport carries messages between an agent and a kernel over a reliable, ordered, bidirectional byte stream, such as a pair of pipes, a TCP connection, or a Unix domain socket. The agent sends agent messages, the kernel sends kernel messages, and each message travels in one frame.

A native session lasts as long as the byte stream. The transport has no initialization exchange and no in-band version negotiation: both parties use version `chio-wire-v1`, agreed out of band (Section 13). The kernel binds each session to one agent key, which it establishes when the session opens by means outside this document. It denies a call whose capability token has a different subject (Section 5.8).

The native transport provides no confidentiality, integrity protection, or peer authentication of its own. It relies on the underlying byte stream for them (Section 15).

10.1. Native Framing

Each frame is a length followed by a payload (Figure 4).

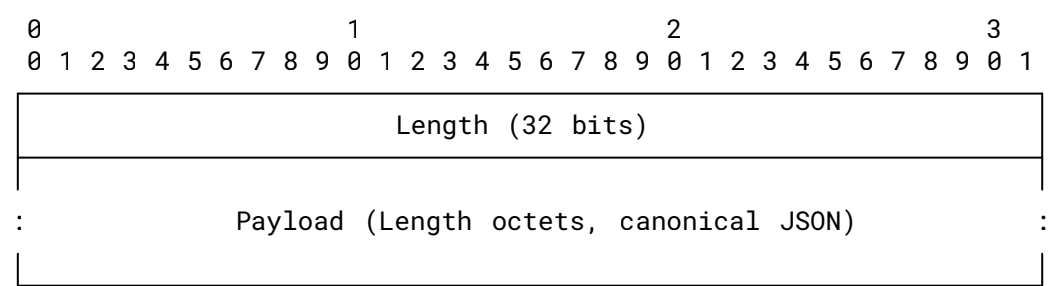


Figure 4: Native Frame

Length: An unsigned 32-bit integer in network byte order that gives the number of octets in Payload. It does not count its own four octets.

Payload: One agent message or kernel message (Section 10.2), encoded as canonical JSON (Section 4.1).

The maximum payload length is 16,777,216 octets. A sender **MUST NOT** send a frame with a longer payload, and a receiver **MUST** reject a frame whose Length exceeds 16,777,216 (Section 10.5). A payload of exactly 16,777,216 octets is permitted.

A sender **MUST** encode each message as canonical JSON and **MUST** put exactly one message in each frame. A sender that fails to encode a message sends no part of its frame. A receiver parses the payload as JSON and does not check that it is canonical. A zero-length payload is valid framing but never a valid message.

For example, the message `{ "type": "heartbeat" }` is 20 octets long, so its frame is the octets 0x00 0x00 0x14 followed by those 20 octets.

10.2. Native Messages

Every message is a JSON object whose `type` member names its kind (Table 31). A sender **MUST NOT** include members that this document does not define for the message. The kernel ignores such members when it receives them.

Sender	type	Other members	Purpose
agent	<code>tool_call_request</code>	Table 32	Request one tool call
agent	<code>list_capabilities</code>	none	Request the session's tokens
agent	<code>heartbeat</code>	none	Liveness probe
kernel	<code>tool_call_chunk</code>	Table 36	One piece of streamed output
kernel	<code>tool_call_response</code>	Table 34	Terminal outcome of a call
kernel	<code>capability_list</code>	<code>capabilities</code>	The session's tokens
kernel	<code>capability_revoked</code>	<code>id</code>	Notice of a revoked capability
kernel	<code>heartbeat</code>	none	Reply to heartbeat

Table 31: Native Message Types

A `tool_call_request` has the members in Table 32. A sender omits an **OPTIONAL** member that has no value and omits `approval_tokens` when it is empty.

Field	Type	Presence
type	string	REQUIRED
id	string	REQUIRED
capability_token	object	REQUIRED
server_id	string	REQUIRED
tool	string	REQUIRED
params	JSON value	REQUIRED
governed_intent	object	OPTIONAL
approval_token	object	OPTIONAL
approval_tokens	array of object	CONDITIONAL
threshold_approval_proposal	object	CONDITIONAL
supplemental_authorization	object	OPTIONAL
execution_nonce	object	OPTIONAL

Table 32: tool_call_request Members

Field	Meaning
type	tool_call_request
id	Request identifier chosen by the agent
capability_token	Capability token (Section 5.1) that authorizes the call
server_id	Identifier of the target tool server
tool	Name of the tool on that server
params	Arguments for the tool
governed_intent	Governed transaction intent (Section 8.2)
approval_token	One approval token (Section 8.4)
approval_tokens	1 to 32 approval tokens; present exactly when threshold_approval_proposal is (Section 8.5)
threshold_approval_proposal	Present exactly when approval_tokens is (Section 8.5)
supplemental_authorization	Opaque authorization extension (Section 7)
execution_nonce	Execution nonce that the kernel returned for this id (Section 7.7)

Table 33: tool_call_request Members Meanings

The agent **MUST NOT** reuse an id within a session except to resume the same operation with a kernel-issued execution nonce or a pending approval proposal and its votes. The target, parameters, subject, and capability binding **MUST** remain unchanged. The kernel **MUST** correlate an accepted resumption with the original admission; other reuse is rejected with `internal_error` ([Section 10.4](#)). A repeated identifier alone does not request or authorize a second execution.

The kernel denies a request that carries both `approval_token` and a non-empty `approval_tokens`, a `threshold_approval_proposal` without `approval_tokens`, or `approval_tokens` without a `threshold_approval_proposal`. It reports `policy_denied` with guard set to `session_authorization`.

No member of `tool_call_request` carries a sender proof. A kernel therefore denies every native call for which a matching grant requires one ([Section 5.4](#)). No member carries model metadata either, so a model constraint that requires metadata ([Section 5.3](#)) is never satisfied on this transport.

The following request is shown with whitespace added and with its capability token abbreviated to the id member. [Appendix A](#) gives a complete frame.

```
{
  "capability_token": {"id": "cap-1"},
  "id": "req-001",
  "params": {"text": "hello"},
  "server_id": "srv",
  "tool": "echo",
  "type": "tool_call_request"
}
```

The kernel answers each `list_capabilities` with one `capability_list`, whose `capabilities` member is an array of the capability tokens that the kernel issued for the session. The kernel does not remove expired or revoked tokens from the list, and it sends an empty array if it cannot produce the list. A kernel **MAY** also send `capability_list` unprompted, for example as its first message, to deliver the tokens it issued when the session opened.

The kernel answers each `heartbeat` with one `heartbeat`. It sends no `heartbeat` unprompted, and this document defines no `heartbeat` interval or timeout.

A kernel **MAY** send `capability_revoked`, whose `id` member is the identifier of a revoked capability. A kernel does not otherwise notify the agent of a revocation: the agent learns of it when a call that depends on the revoked capability is denied ([Section 12.4](#)).

10.3. Tool Call Results

For each `tool_call_request`, the kernel sends zero or more `tool_call_chunk` messages and then one `tool_call_response`. Each carries the request's `id`, and the agent correlates them by that value. [Table 34](#) lists the members of `tool_call_response`.

Field	Type	Presence	Meaning
<code>type</code>	string	REQUIRED	<code>tool_call_response</code>
<code>id</code>	string	REQUIRED	The <code>id</code> of the request
<code>result</code>	object	REQUIRED	Outcome of the call (Table 35)
<code>receipt</code>	object	REQUIRED	Receipt for the call (Section 6.1)
<code>execution_nonce</code>	object	OPTIONAL	Nonce to present when retrying this <code>id</code> (Section 7.7); omitted when absent

Table 34: `tool_call_response` Members

The `result` member is an object whose `status` member selects its other members ([Table 35](#)). The "Decision" column gives the decision that the receipt records ([Section 6.3](#)). The receipt's parameters, target, and any available result binding **MUST** agree with the request and returned result. A valid receipt for a different call is not evidence for this response.

status	Other members	Decision	Meaning
ok	value (JSON value)	allow	The call completed. value is the output, or null if there is none.
stream_complete	total_chunks (integer)	allow	The call completed after streaming total_chunks chunks.
cancelled	reason (string), chunks_received (integer)	cancelled	The call was canceled.
incomplete	reason (string), chunks_received (integer)	incomplete	The call ended before completion.
err	error (object, Section 10.4)	deny	The kernel denied the call or failed to evaluate it.
pending_approval	proposal (object, Section 8.5.1)	deny	Execution awaits approval; no tool was dispatched.

Table 35: Tool Call Results

A pending_approval result contains exactly status and proposal. It ends this request-response exchange but does not finalize the logical operation. The kernel **MUST** send no output chunks and **MUST** omit execution_nonce. A client that does not support this result **MUST** stop; it **MUST NOT** interpret it as an allowed call. Resumption follows [Section 8.5.3](#).

In cancelled and incomplete results, chunks_received is the number of chunks that the kernel sent for the call.

A tool_call_chunk carries one piece of streamed output ([Table 36](#)).

Field	Type	Presence	Meaning
type	string	REQUIRED	tool_call_chunk
id	string	REQUIRED	The id of the request
chunk_index	integer	REQUIRED	Position of the chunk, starting at 0
data	JSON value	REQUIRED	Content of the chunk

Table 36: tool_call_chunk Members

The kernel sends the chunks of a call in order, with `chunk_index` values 0, 1, 2, and so on without gaps, and it sends all of them before the call's `tool_call_response`. A kernel can collect the whole stream before it sends the first chunk, so an agent cannot assume that chunks arrive while the tool runs. Each chunk travels in its own frame, and the frame limit ([Section 10.1](#)) applies to each one. A kernel **MAY** bound the duration, total size, or chunk count of a stream. When it stops a stream at such a bound, it sends only chunks already authorized for release under [Section 3.4](#) and ends the call as incomplete. If the policy requires complete-output inspection, it releases no unchecked partial result.

Every `tool_call_response` carries a receipt signed by the configured kernel receipt authority, including denials, pending approvals, cancellations, and incomplete calls. The receiver **MUST** verify the receipt as specified in [Section 6.9](#) before relying on it. If signing or durable finalization fails, the kernel **MUST NOT** substitute a newly generated, unauthenticated key. It terminates or resets the affected exchange and preserves recoverable operation state. A closed transport does not establish that an external effect was absent.

10.4. Native Errors

An `err` result carries an error object in its `error` member. The object's `code` member names the error, and for some codes a `detail` member adds information ([Table 37](#)). A code without detail is serialized with no `detail` member. The native transport carries no numeric error code; [Section 14.2](#) maps each code to the registry code shown.

code	detail	Meaning	Registry
<code>capability_denied</code>	string	The capability token is invalid for the call, for example because its signature does not verify	2100
<code>capability_expired</code>	none	The token is outside its validity interval	2101
<code>capability_revoked</code>	none	The token, or a capability in its delegation chain, is revoked	2102
<code>policy_denied</code>	object	A check of the kernel or a guard denied the call	3100
<code>tool_server_error</code>	string	The tool server failed	5100
<code>internal_error</code>	string	The kernel failed to evaluate the call	6100

Table 37: Native Error Codes

The detail of `policy_denied` has two string members: `guard`, which names the check that denied the call, and `reason`, a description for people. The kernel uses these guard values:

`kernel`: The kernel's own checks or policy denied the call.

`approval`: The call needs an approval that the request did not carry ([Section 8.4](#)).

session_authorization: The request's approval members conflict ([Section 10.2](#)).

The codes `capability_denied`, `capability_expired`, `capability_revoked`, and `tool_server_error` identify specific failures, and a kernel **SHOULD** use them for those failures. A kernel **MAY** instead report any denial as `policy_denied` with guard set to `kernel` and a reason that names the failure, such as "capability has expired", and any evaluation failure as `internal_error`. An agent can therefore receive either form for the same condition.

The following `err` result reports an expired token in the second form.

```
{
  "error": {
    "code": "policy_denied",
    "detail": {
      "guard": "kernel",
      "reason": "capability has expired"
    }
  },
  "status": "err"
}
```

10.5. Receiver Behavior

A receiver reads a frame in two steps: it reads the four octets of Length, then Length octets of payload. It handles failures as follows:

- If the stream ends before all four octets of Length arrive, the connection is closed and no message is delivered.
- If the stream ends after Length but before the full payload arrives, the connection is closed and the partial payload is discarded.
- If Length exceeds 16,777,216, the receiver **MUST** reject the frame without reading or allocating its payload.
- If the payload is not exactly one JSON object that forms a valid message of the kind the receiver expects (for example, it is empty, it is not JSON, a **REQUIRED** member is missing, a member has the wrong type, or type is unknown), the receiver **MUST** reject the frame.

The transport defines no marker from which a receiver could find the next frame boundary, and a receiver **MUST NOT** attempt to resynchronize. After any of these failures, and after a failure to send, the kernel ends the session: it reads no further frames, sends no further messages, and closes the session. It sends the peer no error message first. Neither party resumes a session. A peer that wants to continue opens a new transport, which begins a new session.

Signed objects travel inside messages, including capability tokens (in `tool_call_request` and `capability_list`) and receipts (in `tool_call_response`), approval artifacts, and execution nonces. Each artifact defines its own authenticated projection and signing input ([Section 4.5](#)). A

party that forwards an artifact **MUST** preserve that projection and its signature. It **MAY** re-encode the same typed values when this produces the same signing input. Unknown JSON members are not thereby authenticated.

11. Hosted MCP Binding

A hosted edge exposes the kernel's execution services as an MCP server [MCP]. It uses the MCP Streamable HTTP transport: the client sends each JSON-RPC 2.0 message [JSON-RPC] in an HTTP POST [RFC9110], and the edge answers requests with server-sent events [SSE]. The edge supports MCP version 2025-11-25, which it selects as Section 13 specifies.

A client never handles capability tokens. For each MCP session the edge opens a kernel session, creates an agent key, and obtains capability tokens for that key under its issuance policy. For each tool call it selects one of those tokens (Section 11.6).

11.1. Endpoints and Headers

The edge serves one endpoint path, `/mcp`, with the methods in Table 38.

Method	Use	Success response
POST	Send one JSON-RPC request, notification, or response	200 with an event stream for a request; 202, or 200 with an event stream, otherwise
GET	Open the notification stream (Section 11.5)	200 with an event stream
DELETE	End the session	204

Table 38: Hosted Endpoint Methods

Table 39 lists the request headers that the edge examines.

Header	Sent with	Rule
Authorization	every request	A Bearer credential (Section 11.2)
Origin	any request	If present, its host is localhost, 127.0.0.1, or ::1
Accept	POST, GET	POST lists application/json and text/event-stream; GET lists text/event-stream
Content-Type	POST	Begins with application/json
MCP-Session-Id	every request after initialize	The session identifier
MCP-Protocol-Version	any request in a session	If present, equals the session's version
Last-Event-ID	GET	Replay cursor (Section 11.5)
DPoP	any request	Sender proof, when the access token requires one (Section 11.2)

Table 39: Hosted Request Headers

The edge limits the request rate of each client address and answers a request over the limit with 429 and a Retry-After header. It then processes a POST in this order, stopping at the first failure:

1. A present Origin that is malformed or absent from the deployment's explicit allowed-origin set: 403. Matching compares the complete origin, including scheme, host, and effective port. A loopback host name alone is not authorization for an arbitrary browser origin.
2. Admission fails ([Section 11.2](#)): 401.
3. Accept does not accept both required media types: 406. The parsed Content-Type media type is not application/json: 415. Parameters are parsed as HTTP parameters; a prefix such as application/json-extra is not a match.
4. Content-Length is not an integer: 400, with JSON-RPC code -32600. The body exceeds 8,388,608 octets: 413, with JSON-RPC code -32600. The body is not JSON, or reading it fails: 400, with JSON-RPC code -32700.
5. If method is initialize, the edge continues as [Section 11.3](#) describes.
6. MCP-Session-Id is missing, empty, or has leading or trailing whitespace: 400, with JSON-RPC code -32600. The session is unknown: 404.
7. For a session in a terminal state ([Section 11.4](#)), the edge answers 404, as required by the MCP session lifecycle.
8. Otherwise, a MCP-Protocol-Version header that differs from the session's version: 400. Credential continuity fails: 403. The session is not ready: 409.

Where a step gives a JSON-RPC code, the response body is a JSON object with members `jsonrpc`, `error`, and `id`. The `id` equals the request identifier when it can be determined safely; otherwise it is `null`, as specified by [JSON-RPC]. HTTP-layer rejection of a notification uses an HTTP error status rather than a JSON-RPC response to that notification. Other error responses carry a plain-text body.

A POST that carries a request, meaning a message with both `id` and `method`, receives an event stream. The stream starts with a priming event that has an event identifier, a `retry` field of 1000 milliseconds, and empty data. It ends after the event that carries the response to the request. The edge handles the requests of a session one at a time: a request POST waits until the stream of the previous request has ended. An accepted POST carrying a notification or response **MUST** receive 202 with no body. The edge **MUST** process the accepted message; an occupied request stream is not permission to silently discard it. Any resulting server messages use an appropriate open stream under the MCP transport rules.

A DELETE is checked for Origin and admission as a POST is. It then needs an MCP-Session-Id of a known session (400 or 404 otherwise, with plain-text bodies) and credential continuity (403). The edge answers 404 for a session already in a terminal state. Otherwise it moves the session to deleted and answers 204.

11.2. Session Admission

Every request, initialize included, **MUST** carry an Authorization header with the Bearer scheme and a non-empty token. The edge answers a request without one, or with a token that it does not accept, with 401 and a WWW-Authenticate challenge. If the edge publishes OAuth protected resource metadata [RFC9728], the challenge is Bearer with a `resource_metadata` parameter that gives the metadata URL, and a `scope` parameter when scopes are configured. Otherwise the challenge is Bearer alone.

The edge accepts tokens in one configured mode:

- a static token, which has to equal the configured value;
- a JWT [RFC7519], signed with EdDSA, RS256, RS384, RS512, PS256, PS384, PS512, ES256, or ES384 under a configured key or key set. The edge **MUST** configure an allowed algorithm set independently of the token, require and validate `iss`, `aud`, and `exp`, and reject a token before a present `nbf` or at or after `exp`. It **MUST** bind issuer keys to the expected issuer, validate its own audience and configured resource and scopes, and reject a token intended for a different use. JWT processing follows [RFC8725];
- a token that the edge validates by OAuth token introspection [RFC7662].

Introspection **MUST** use an authenticated, protected connection to the configured authorization server and require an active token for the expected resource, principal, scope, and validity interval. Static bearer credentials are an explicitly configured shared-principal profile; they **MUST NOT** be described as distinct authenticated users or sender-bound credentials.

The edge has no unauthenticated mode.

A JWT or introspected token can bind the request to a sender through members of its `cnf` claim. For `chioSenderKey`, the request **MUST** carry a DPoP header whose value is the base64url encoding without padding [RFC4648] of a Chio sender proof (Section 5.4) signed by that key. The proof binds `capability_id` to the token's `jti` claim, `tool_server` to the edge's resource identifier (its protected resource, or `/mcp`), `tool_name` to the HTTP method, and `action_hash` to the SHA-256 hash of the empty octet string. The edge rejects a stale proof, a proof issued too far in the future, and a proof whose nonce it has already seen for the same `jti`. For `x5t#S256` and `chioAttestationSha256`, the request **MUST** carry the header `x-chio-mtls-thumbprint-sha256` or `x-chio-runtime-attestation-sha256`, respectively, with a value equal to the claim. A failed check is 401. These checks authenticate HTTP requests; the edge does not pass the proof to the kernel (Section 11.6).

The Chio object carried in this binding's DPoP header is not the JWT proof format of [RFC9449]. Implementations **MUST NOT** advertise this binding as OAuth DPoP interoperability. Deployments needing that format require a separately specified binding.

Certificate and attestation headers **MUST** originate from an authenticated trusted ingress that strips client-supplied copies and validates the underlying evidence. An attestation digest comparison alone is not sender authentication. Tokens with an attestation binding **MUST** also have a verified sender-key or certificate binding on the same request. The edge **MUST** reject the request if the required trusted ingress context is unavailable.

The edge binds the authentication context of `initialize` to the session. Every later request of the session **MUST** present the same context: the same `Origin` value or none, and either the same static token or, for a JWT or introspected token, the same principal, issuer, subject, audience, scopes, and identity claims. When the edge recorded a fingerprint of the token, the request **MUST** carry the same token. The edge answers a mismatch with 403.

11.3. Initialization

A client opens a session as follows:

1. The client sends an `initialize` request in a POST without an `MCP-Session-Id` header. The edge answers 400, with JSON-RPC code -32600, if the message has no `id` or if the POST carries an `MCP-Session-Id` header with any value. It creates no session in either case.
2. The edge creates a session in state `initializing`, with a new session identifier, a new kernel session, an agent key, and capability tokens for that key. It generates the session identifier from fresh randomness; clients treat it as opaque.
3. The edge answers with an event stream that holds a priming event and then the `initialize` response. If the response is a result, the edge sets `MCP-Session-Id` on the HTTP response, records `result.protocolVersion` as the session's version, and moves the session to `ready`. If initialization fails, the HTTP response has no `MCP-Session-Id` and the edge discards the session.

4. The client sends notifications/initialized in a POST with the session identifier. The edge then activates the kernel session. Until it does, the edge answers tools/list, tools/call, and the other operations of a session with JSON-RPC error -32002. It answers ping at any time.

Every initialize creates a new session, and the edge never re-initializes an existing one. A client that sets toolCallChunkNotifications within capabilities.experimental.chioToolStreaming to true in initialize receives streamed tool output as notifications (Section 11.6). The result's capabilities.experimental member advertises that feature as chioToolStreaming and describes version selection as chioProtocol (Section 13).

11.4. Session States

Figure 5 shows the states of a hosted session.

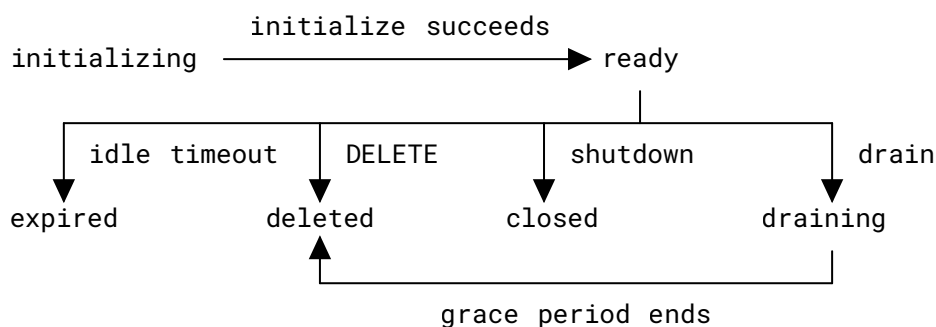


Figure 5: Hosted Session States

A ready session becomes expired when no request has been accepted for the idle period, which is 15 minutes by default. Each POST or GET that the edge accepts restarts the idle period; an open stream does not. An operator can drain a ready session, which then becomes deleted when a grace period ends, 5 seconds by default. A DELETE moves any session that is not in a terminal state to deleted, and an operator shutdown moves it to closed.

The states deleted, expired, and closed are terminal, and a terminal session never changes state. The edge answers a request to an initializing or draining session with 409 and a request to a terminal session with 404. Retaining an internal tombstone for replay protection does not change that externally visible status.

The edge does not resume a terminal session. A client continues after 404 only by opening a new session. While a session is ready, a client that loses a connection can keep sending requests with the same session identifier and credentials. A response lost with its stream cannot be recovered through the notification replay buffer (Section 11.5). The client **MUST** resolve the original operation's outcome through an authenticated recovery path before retrying a call with possible side effects. A new session does not make a new execution of that call safe.

11.5. Notification Stream and Replay

A client opens the notification stream of a session with a GET. The GET needs an MCP-Session-Id header (400, with JSON-RPC code -32600, otherwise) and an Accept header that lists text/event-stream (406 otherwise). The edge applies the Origin, admission, version, continuity, and state checks of a POST. A session has at most one notification stream: the edge answers a second GET with 409 while the first stream is open. The stream carries only notifications, meaning messages with method and without id. It has no priming event and ends when its output channel closes or the client disconnects. A logical session-state transition does not guarantee immediate closure of an already open stream.

While a notification stream is open, the edge sends notifications only on it. While none is open, it sends them on the stream of the current request POST. A request stream also carries the edge's requests to the client, which have both method and id.

Each event has an identifier of the form <session-id>-<n>, where <n> is a decimal counter. The counter starts at 1 and counts every event of the session on every stream, priming events included. The counter is allocated when the edge creates an event, so its value does not establish delivery order across streams or buffered events. For a non-priming event, data is one JSON-RPC message; a priming event has empty data. The edge sets no event field. The edge retains the last 64 notifications for replay and retains no other events. Delivery on an open stream is best effort: the edge drops events, without notice, for a reader that falls too far behind.

A GET with a Last-Event-ID header asks the edge to replay the notifications that followed that event. The edge splits the value at its last hyphen. The part before it has to equal the session identifier, and the part after it, the cursor, has to be an unsigned decimal integer. The edge answers 409 and opens no stream if the value is malformed or names another session, if it retains no notifications, or if the cursor is below one less than the counter of the oldest retained notification or above that of the newest. Otherwise it sends each retained notification with a larger counter and then continues with live notifications, without duplicates. After a 409, a client can open a stream without Last-Event-ID and accept that it missed notifications.

An edge can serve several sessions through one connection to an upstream tool server. In that configuration it delivers every notification from the upstream server into every live session's queue. Session workers filter resource updates by subscription and elicitation completion by session context, and ignore unsupported notifications.

11.6. Tool Calls

A tools/call request names the tool in params.name and passes params.arguments, which defaults to an empty object. The edge answers JSON-RPC error -32602 if name is absent or names a tool it does not expose. Its tools/list lists only the tools that the session's capability tokens authorize.

The client sends no capability token. The edge selects the first of the session's tokens whose grants match the tool's server and name, the arguments, and the model metadata ([Section 11.7](#)). If no token matches, the edge returns a result with `isError` set to `true` and does not submit the call to the kernel, so no receipt records the call. Otherwise the edge submits the call to the kernel with the selected token, and the kernel evaluates it as it evaluates a native call ([Section 5.8](#)). The submission carries no sender proof: the kernel denies a call for which a matching grant requires one ([Section 5.4](#)).

The members of `params._meta` in [Table 40](#) carry the other inputs of a native request ([Table 32](#)). When a member and its alias are both present, the edge uses the member. `_meta` has to be an object, and each member has to have the structure of its native equivalent; the edge answers -32602 otherwise.

Member	Native equivalent
<code>chioRequestId</code>	<code>id</code>
<code>modelMetadata</code>	none (Section 11.7)
<code>executionNonce</code>	<code>execution_nonce</code>
<code>governedIntent</code>	<code>governed_intent</code>
<code>approvalToken</code>	<code>approval_token</code>
<code>approvalTokens</code>	<code>approval_tokens</code>
<code>thresholdApprovalProposal</code>	<code>threshold_approval_proposal</code>
<code>supplementalAuthorization</code>	<code>supplemental_authorization</code>
<code>routeSelection</code>	none

Table 40: Chio Members of `_meta` in `tools/call`

Member	Accepted alias
chioRequestId	none
modelMetadata	chioModelMetadata
executionNonce	chioExecutionNonce
governedIntent	chioGovernedIntent
approvalToken	chioApprovalToken
approvalTokens	chioApprovalTokens
thresholdApprovalProposal	chioThresholdApprovalProposal
supplementalAuthorization	chioSupplementalAuthorization
routeSelection	route_selection, chioRouteSelection

Table 41: Accepted Chio Metadata Aliases

chioRequestId is a non-empty string of at most 2048 UTF-8 octets, with no leading or trailing whitespace and no control characters. The kernel uses it as the request identifier. Without it, the edge uses the request identifier bound in a presented execution nonce, or assigns a new identifier to each call. A request that carries approval members or supplementalAuthorization **MUST** carry chioRequestId, and the approval members have to follow the combination rules of [Section 10.2](#); the edge answers -32602 otherwise. The kernel records routeSelection, an object, in the receipt's metadata as route_selection.

The following request is illustrative.

```
{
  "jsonrpc": "2.0",
  "id": 7,
  "method": "tools/call",
  "params": {
    "name": "echo",
    "arguments": {"text": "hi"},
    "_meta": {
      "chioRequestId": "req-7",
      "modelMetadata": {
        "model_id": "m-1",
        "safety_tier": "standard",
        "provider": "p"
      }
    }
  }
}
```

The edge returns the kernel's outcome as an MCP tool result:

- For an allowed call, `isError` defaults to `false`. A tool output already shaped as an MCP result can retain an existing `isError: true`. An object value becomes `structuredContent` plus a text rendering in `content`. A string value becomes one text item. A value that already has the members of a tool result keeps its existing fields. Missing `isError` defaults to `false`; missing `content` is synthesized only when `structuredContent` is present.
- For a denied, canceled, or incomplete call, and for a failure of the kernel to evaluate the call, `isError` is `true`. An ordinary error has one text item that states the reason. A retained streamed error instead describes the stream delivery in `content` and records the reason in `structuredContent.chioToolStream.reason`.
- For pending approval, `content` is an empty array, `isError` is `false`, and `structuredContent` is the native pending result: `status: pending_approval` and the complete signed proposal. This is a workflow state, not permission to execute. The next call uses the original `chioRequestId`, `proposal`, and collected votes.
- For a call that needs a URL elicitation, the edge returns JSON-RPC error -32042 with `data.elicitations`.

For a kernel outcome, the edge includes the signed receipt in `result._meta.chio.receipt`, its identifier in `receiptId`, the projected decision in `decision`, and diagnostic lifecycle information in `terminalState`. The projected decision is `allow`, `deny`, or `pending_approval`; it **MUST NOT** replace the signed receipt's decision. In particular, a pending result's receipt records denied execution for that exchange ([Section 8.5.3](#)).

The edge also returns `result._meta.chioEvidence`, with schema `chio.mcp.execution-evidence.v1`, the signed receipt, the kernel `requestId`, `outputKind`, `output`, and diagnostic `terminalState`. For an allowed scalar or object result, `outputKind` is `value` and `output` contains the original value whose canonical bytes the kernel bound. A projected MCP wrapper is not a substitute for that original value. For a stream, `outputKind` is `stream` and `output` is `null`; stream content verification requires the ordered chunks and their binding profile. When no releasable value is available, `outputKind` is `none` and `output` is `null`. The evidence wrapper is not separately signed; consumers **MUST** verify the enclosed receipt and all claimed bindings. They **MUST NOT** infer permission from an unsigned diagnostic field or from MCP's `isError` alone.

The edge **MUST** construct these metadata namespaces from the actual kernel response, replacing conflicting tool-supplied metadata. It **MUST** preserve the original signed receipt. Authorized consumers can also query retained receipts through trust-control ([Section 12.3](#)). An admission or transport failure can occur before a receipt exists; such an error **MUST NOT** carry fabricated execution evidence. When issued, an execution nonce is returned in `result._meta.chioExecutionNonce`; it **MUST** be absent from a pending approval response.

If the client enabled chunk notifications at initialization, the edge sends one `notifications/chio/tool_call_chunk` notification per chunk before the result. Its `params` members are `requestId` (the JSON-RPC id of the call), `chunkIndex`, `totalChunks`, and `chunk`. The result's `structuredContent.chioToolStream` then has `mode` set to `notification_stream`. Otherwise a retained stream's `chioToolStream` has `mode` set to `collapsed_result` and carries every chunk in

chunks. In both cases `chioToolStream` also carries `totalChunks` and `terminalState`, which is completed or incomplete in retained streamed results. Cancellation is an exception: the edge returns a reason-only error result and omits the stream summary and collapsed chunks. Chunk notifications can already have been queued before that cancellation result.

11.7. Model Metadata

A client can describe the model on whose behalf it calls a tool, in the `modelMetadata` member of `_meta` (Table 42).

Field	Type	Presence	Meaning
<code>model_id</code>	string	REQUIRED	Model identifier
<code>safety_tier</code>	string	OPTIONAL	low, standard, high, or restricted
<code>provider</code>	string	OPTIONAL	Provider name
<code>provenance_class</code>	string	OPTIONAL	Replaced by the edge

Table 42: Model Metadata

The edge has not verified this metadata, so it **MUST** set `provenance_class` to `asserted`, whatever value the client sent. The other provenance classes, `observed` and `verified`, are not available to a client. The kernel evaluates model constraints (Section 5.3) against the metadata; those constraints do not consider the provenance class. The kernel records the metadata in the receipt's `metadata.model_metadata` member, where `provenance_class` is always present. For the request above, the receipt records:

```
{
  "model_metadata": {
    "model_id": "m-1",
    "provenance_class": "asserted",
    "provider": "p",
    "safety_tier": "standard"
  }
}
```

12. Trust-Control Interface

The trust-control service offers an HTTP interface [RFC9110] whose paths begin with `/v1` (Section 13). Request and response bodies are JSON objects with camelCase member names. Nested capability tokens, receipts, and attestation statements keep their own member names. This section specifies four endpoints: issuance, delegated issuance, receipt query, and revocation.

A caller authenticates with an `Authorization` header that carries the Bearer scheme and the service token. The service compares the token with its configured value in constant time and answers a missing or wrong token with 401 and `WWW-Authenticate: Bearer`. The receipt query

also accepts tenant read tokens ([Section 12.3](#)). The nodes of a clustered service can authenticate issuance requests to each other by a mechanism outside the scope of this document; a clustered service also adds members that describe the node that handled a request.

An error response has the body `{"error": "<message>"}`, and its status code gives the class of error. The service rejects a body or query string that does not parse into the structure defined here with a 4xx status, which can precede authentication and can carry a different body. It answers 503 when it sheds load.

12.1. Capability Issuance

POST `/v1/capabilities/issue` issues a direct capability token, one with an empty delegation chain. [Table 43](#) lists the request members. A 200 response has one member, `capability`, which holds the token ([Section 5.1](#)).

Field	Type	Presence	Meaning
<code>subjectPublicKey</code>	string	REQUIRED	Subject key (Section 4.4)
<code>scope</code>	object	REQUIRED	Requested scope (Section 5.2)
<code>ttlSeconds</code>	integer	REQUIRED	Requested lifetime in seconds
<code>runtimeAttestation</code>	object	OPTIONAL	Runtime attestation statement; omitted when absent

Table 43: Issuance Request Members

The service signs the token with its authority key. The token's `subject` is `subjectPublicKey`, its `issued_at` is the current time, and its `expires_at` is `issued_at` plus a policy-bounded `ttlSeconds`. The lifetime **MUST** be positive. The service **MUST** reject overflow or an unrepresentable deadline instead of saturating it. Its `scope` is the requested scope, to whose grants the service's issuance policy can add constraints. The service answers:

- 400 if `subjectPublicKey` is not a valid key, or if `runtimeAttestation` has a malformed `workload_identity` or one that conflicts with a SPIFFE URI in `runtime_identity`;
- 403 if its issuance policy denies the request, including when it rejects the runtime attestation;
- 409 if the authority backend is not configured or its configuration conflicts with the issuance mode;
- 500 for any other failure.

A runtime attestation statement is a normalized result from an attestation verifier, with members `schema`, `verifier`, `tier` (none, basic, attested, or verified), `issued_at`, `expires_at`, `evidence_sha256`, and the **OPTIONAL** `runtime_identity`, `workload_identity`, and `claims`. The service rejects a statement outside its validity interval (`issued_at` ≤ now < `expires_at`). Configured trust and assurance policies determine acceptance and effective tier; without trust

rules, untrusted evidence can be downgraded to none. That effective tier selects any configured scope and lifetime ceilings. Attestation evidence formats and their appraisal are outside the scope of this document.

12.2. Delegated Issuance

POST /v1/federation/capabilities/issue issues a capability token to the subject of an Agent Passport presentation, optionally within a delegation policy signed by another party. Passports and presentations are outside the scope of this document; this section specifies the delegation policy and the checks that bound the issued token. Only the service token authenticates this endpoint.

Field	Type	Presence	Meaning
presentation	object	REQUIRED	Passport presentation; its format is outside this document
expectedChallenge	object	REQUIRED	Challenge that the presentation answers; its format is outside this document, except its verifier member, a string that names the verifying service
capability	object	REQUIRED	Requested token: scope (object, Section 5.2) and ttl (integer, seconds)
admissionPolicy	object	OPTIONAL	Admission policy for enterprise identities; outside this document
enterpriseIdentity	object	OPTIONAL	Enterprise identity of the subject; outside this document
delegationPolicy	object	OPTIONAL	Signed delegation policy (Table 45)
upstreamCapabilityId	string	OPTIONAL	Parent capability that the delegation policy names

Table 44: Delegated Issuance Request Members

A delegationPolicy has two members: body ([Table 45](#)) and signature, the signature of signerPublicKey over the canonical encoding of body ([Section 4.5](#)).

Field	Type	Presence	Meaning
schema	string	REQUIRED	chio.federated-delegation-policy.v1
issuer	string	REQUIRED	Party that grants the delegation
partner	string	REQUIRED	Party that receives it
verifier	string	REQUIRED	Service that may issue under the policy
signerPublicKey	string	REQUIRED	Key that signs the policy (Section 4.4)
createdAt	integer	REQUIRED	Start of validity, in Unix seconds
expiresAt	integer	REQUIRED	End of validity, in Unix seconds
ttlSeconds	integer	REQUIRED	Largest lifetime of an issued token
scope	object	REQUIRED	Largest scope of an issued token
purpose	string	OPTIONAL	Stated purpose
parentCapabilityId	string	OPTIONAL	Parent capability to which the policy is bound

Table 45: Delegation Policy Body Members

The service **MUST** perform these checks, in this order, and reject the request at the first that fails:

1. If the service has a configured public URL, expectedChallenge's verifier equals it (400).
2. If delegationPolicy is present: its schema is the value above, createdAt ≤ expiresAt, ttlSeconds is greater than 0, the signature verifies, createdAt ≤ now ≤ expiresAt, and its verifier equals the challenge's verifier and the configured public URL, if any (400 for each).
3. If delegationPolicy is present, the requested token is within its ceiling: the requested scope is a subset of the policy's scope ([Section 5.6](#)), ttl ≤ ttlSeconds, and now plus ttl, saturating at the unsigned 64-bit maximum, is at most expiresAt (403).
4. upstreamCapabilityId, if present, equals the policy's parentCapabilityId, and a policy parentCapabilityId is present only with upstreamCapabilityId (400).
5. The presentation verifies against the challenge and the service's verifier policy, by rules outside this document (400, 403, or 409).
6. If upstreamCapabilityId is present, the service holds a record of that capability imported from the operator that issued it (404), the policy's signerPublicKey is the key that shared the record, and the requested scope and expiry fit within the parent's (403). Importing such records is outside the scope of this document.
7. If delegationPolicy is present, signerPublicKey is one of the keys that the service's capability authority trusts (403).

The service then issues a direct token from its authority key to the presentation's subject, with the requested scope and `ttl`, under its issuance policy (403 if that policy denies it). With a delegation policy, the service records the policy as the lineage parent of the new token (409 if it has no receipt store) and returns the identifier of that record. The delegation appears in the service's records, not in the token's delegation chain ([Section 5.5](#)).

A 200 response has the members in [Table 46](#).

Field	Type	Presence	Meaning
subject	string	REQUIRED	The presentation's subject as a <code>did:chio</code> identifier (Section 4.6)
subjectPublicKey	string	REQUIRED	The subject key
verification	object	REQUIRED	Result of verifying the presentation; outside this document
capability	object	REQUIRED	The issued token
enterpriseIdentityProvenance	object	OPTIONAL	Outside this document
enterpriseAudit	object	OPTIONAL	Outside this document
delegationAnchorCapabilityId	string	CONDITIONAL	Present when <code>delegationPolicy</code> was: identifier of the recorded lineage parent

Table 46: Delegated Issuance Response Members

12.3. Receipt Query

GET `/v1/receipts/query` returns the stored receipts that match its query parameters ([Table 47](#)). Every parameter is **OPTIONAL**, and the filters that are present all apply.

Parameter	Type	Selects receipts
capabilityId	string	whose capability_id equals the value
toolServer	string	whose tool_server equals the value
toolName	string	whose tool_name equals the value
outcome	string	whose decision is the value: allow, deny, cancelled, or incomplete
since	integer	whose timestamp is at or after the value (Unix seconds)
until	integer	whose timestamp is at or before the value
minCost	integer	whose charged cost, in minor units (Section 7.1), is at least the value
maxCost	integer	whose charged cost is at most the value
costCurrency	string	whose charged cost is in this currency (ISO4217)
agentSubject	string	whose subject key equals the value; the capability's subject key is used only when the receipt subject is absent
cursor	integer	after this position (Section 12.3.1)
limit	integer	up to this many; default 50, clamped to the range 1 to 200

Table 47: Receipt Query Parameters

The service **MUST** reject the query with 400 if minCost exceeds maxCost, if minCost or maxCost is present without costCurrency, or if costCurrency is not exactly three uppercase ASCII letters. It **MUST** also reject an outcome outside the four values with 400. A receipt without cost data matches no cost filter.

A caller authenticated with the service token reads all receipts. The service **MAY** also accept read tokens that are each bound to one tenant; a query authenticated with such a token returns only receipts whose signed tenant_id is that tenant. The service **MUST** derive tenant context from authenticated credentials, apply it to every lookup and page, and verify that returned signed bodies match that context. An identifier or query parameter is not read authority. Receipts without authenticated tenant attribution **MUST NOT** be included in a tenant query. Privileged administrative access is an explicitly separate authorization role.

12.3.1. Paging

The service orders matching receipts by their position in its store, ascending, and returns those whose position is greater than cursor. The response has three members:

totalCount: The number of receipts that match the filters, regardless of cursor and limit.

nextCursor: The position of the last receipt returned when the page is full, and null otherwise.

receipts: An array of receipt objects ([Section 6.1](#)).

A client pages by passing `nextCursor` as `cursor` until `nextCursor` is null; the last page can be empty. Positions are not otherwise exposed. The service verifies the signature of each receipt before it returns it, and it fails the query with 500 if a stored receipt does not verify. A query that matches nothing returns:

```
{"nextCursor": null, "receipts": [], "totalCount": 0}
```

12.4. Revocation

POST `/v1/revocations` revokes a capability. Its request body has one **REQUIRED** member, `capabilityId`, a string that identifies the capability. Only the service token authenticates this endpoint. The service records the identifier as revoked without checking that it names an issued capability, so a caller can revoke an identifier before any token uses it. It answers 200 only after the revocation is visible in its store, 409 if it has no revocation store, and 500 if the store fails. A 200 response has these members:

capabilityId: The identifier from the request.

revoked: true.

newlyRevoked: false if the identifier was already revoked, and true otherwise.

```
{"capabilityId": "cap-1", "newlyRevoked": true, "revoked": true}
```

GET `/v1/revocations`, authenticated by the service token, reports revocation state. With the parameter `capabilityId`, its response includes `capabilityId` and a boolean `revoked` for that identifier. The response also has `configured` (boolean), `backend` (string), `count` (integer), and `revocations`, an array of up to `limit` objects with members `capabilityId` and `revokedAt` (Unix seconds).

A kernel learns of a revocation only when it checks. Before it dispatches a call, the kernel checks the revocation state of the token's `id` and of each capability in the token's delegation chain ([Section 5.8](#)). A kernel configured with a trust-control service makes each check a GET `/v1/revocations` request with `capabilityId` set to the identifier and `limit` set to 1, reads `revoked`, and keeps no cache. The kernel **MUST** deny the call if any identifier is revoked or a check fails. It records a denial receipt when its signing and storage authorities are available, subject to [Section 3.4](#). The trust-control service does not notify kernels of revocations. The reference native driver does not notify agents; the optional `capability_revoked` message remains a permitted wire allocation ([Section 10.2](#)).

A successful revocation response acknowledges durable visibility at the responding authority. It does not by itself acknowledge observation by every kernel or cancellation of an already-committed external effect. Deployments with replicated authorities **MUST** define the freshness bound and admission fence for their revocation profile. A kernel **MUST NOT** continue using a stale negative revocation result after that bound. Unavailable or indeterminate revocation state denies new dispatch.

13. Versioning and Negotiation

Each wire surface has its own version selector. A version match on one surface does not establish a match on another surface.

A schema identifier fixes its field meanings, canonicalization, and signing input. An incompatible change requires a new schema or binding version. A registry allocation alone does not negotiate an extension. A peer **MUST** reject an unsupported required feature rather than remove it and retry under weaker semantics. In particular, peers using threshold approval **MUST** agree support for the `pending_approval` result before opening an execution session.

For the native transport, this document defines `chio-wire-v1`. The agent and kernel establish that value out of band. Native messages carry no in-band version field. A kernel that cannot accept the agreed version closes or resets the transport. Registry code 1000 classifies that out-of-band incompatibility; there is no native numeric-error frame or implemented in-band version exchange. There is no native downgrade exchange.

The hosted binding supports MCP version 2025-11-25 and follows MCP's initialization negotiation. The client **MUST** send `params.protocolVersion`. If the requested version is supported, the edge returns that version; otherwise it returns a version it supports. A client that cannot use the returned version **MUST** end initialization without submitting operations. A missing or malformed selector is invalid parameters, not implicit agreement to a default.

The client **MUST** send the selected version in `MCP-Protocol-Version` on subsequent HTTP requests. The edge rejects an invalid or unsupported header with HTTP 400. When the header is absent but an authenticated session identifies the negotiated version, the edge can use that version for compatibility; it **MUST NOT** infer another version from request data. Session admission and state handling are specified in [Section 11](#).

The trust-control interface uses the path prefix `/v1`. A client selects that prefix explicitly. There is no downgrade negotiation for that interface. Future prefixes are separate interface versions.

14. Error Model

Errors describe a failed operation, not proof that a side effect did not occur. In particular, an incomplete outcome records uncertainty about the tool's terminal state. Before repeating an operation with side effects, an agent determines whether it completed or uses the tool's own idempotency contract.

14.1. Error Registry

The numeric registry has eight categories: `protocol`, `auth`, `capability`, `guard`, `budget`, `tool`, `internal`, and `transaction`. [Section 17.1](#) gives every initial allocation. The transaction entries reserve identifiers used by related evidence profiles; this document does not define those profiles.

A transient flag describes whether a changed condition can permit a later attempt. It is not permission to repeat the same request without change. The retry strategy is part of each allocation. For example, `auth_missing_or_invalid` requires fresh credentials, and `budget_exhausted` requires a budget change or reconciliation. A revoked token stays revoked even though a newly issued token can authorize a later call. For tool and internal failures, bounded backoff controls load only after the client establishes that retry is safe under [Section 3.4](#). It does not resolve an uncertain external effect.

14.2. Surface Mappings

A native error is a `tool_call_response` whose result has `status: err`. The nested error has code and, for variants carrying diagnostic text, `detail`. The native names are `capability_denied`, `capability_expired`, `capability_revoked`, `policy_denied`, `tool_server_error`, and `internal_error`. They map to registry codes 2100, 2101, 2102, 3100, 5100, and 6100 respectively. A registry allocation does not add a variant to the closed native error union.

Hosted errors use JSON-RPC and, where supplied, a Chio code in error data. Invalid request shape uses -32600, invalid method parameters use -32602, and a session that is not initialized uses -32002. Version selection follows [Section 13](#); an unsupported later HTTP version header is an HTTP-layer error. Authentication failures can occur at HTTP session admission before a JSON-RPC response exists. A client keeps HTTP status, JSON-RPC code, and Chio code distinct.

15. Security Considerations

This section describes the protocol's threat model, controls, and residual risks using the approach of [\[RFC3552\]](#). The security boundary includes authority issuance, kernel admission, dispatch, resource accounting, output release, and evidence verification.

15.1. Assets and Trust Boundaries

The protected assets are execution authority, confidential data and credentials, resource budgets, isolation state, signed evidence, and the availability of the services that enforce them.

An adversary can control an agent, its prompts and requested parameters, a tool's returned data, and an untrusted network. It can steal bearer tokens, replay or reorder messages, submit malformed signed objects, collude across sessions, race concurrent admissions, interrupt processes, and present stale storage snapshots. A compromised tool server can try to escape its permitted files, destinations, or credential scope.

The trusted computing base includes the kernel's enforcement code, trusted issuers and policy configuration, signing authorities, durable authority stores, authenticated adapters, and the host isolation mechanisms selected by the execution profile. A secret broker is also trusted for the provider credentials and actions entrusted to it. Protocol signatures do not compensate for compromise of those components.

Chio constrains the authority available to an agent even when its instructions or model behavior are adversarial. It does not infer user intent from the semantic safety of a prompt. A policy that authorizes an undesired operation can still produce a correctly signed allow receipt. Likewise, a compromised kernel key can authenticate false statements; a verifier checks the signer's statement and the evidence supporting each additional claim, rather than treating a signature as proof of truth.

An execution profile **MUST** identify the components on which its claims depend and prevent alternative paths around them. An adapter that omits authenticated context, a tool with ambient credentials, or a recovery path that recreates consumed authority can defeat otherwise valid protocol checks.

15.2. Transport Security

[Table 48](#) states the transport requirements for each surface. TLS supplies channel protection; [\[RFC8446\]](#) defines TLS 1.3.

Surface	TLS	Mutual TLS	Without transport security
Native transport	REQUIRED across hosts or untrusted networks	REQUIRED when the peer's identity is part of the authorization decision	Conformant only over a same-host socket or loopback, for development
MCP binding	REQUIRED for any non-loopback deployment	REQUIRED when the session's sender binding is an mTLS certificate thumbprint	Conformant only on loopback or in a test harness
Trust-control interface	REQUIRED for any non-loopback deployment	REQUIRED for service-to-service deployments that rely on transport identity	Conformant only on loopback
Kernel to tool server	Provided by mutual TLS when networked	REQUIRED over TCP across processes or hosts	Not conformant for production over a network

Table 48: Transport Security Requirements

A deployment that is missing a transport property that [Table 48](#) requires **MUST** deny the request or restrict itself to a local development configuration. A deployment **MUST NOT** claim confidentiality, peer authentication, or replay resistance for traffic it carries in plaintext across a network.

A hosted edge can run behind a proxy that terminates TLS and passes the client certificate's thumbprint to the edge in a request header. The edge accepts that value only in authenticated ingress context. The proxy **MUST** remove any client-supplied copy, and the edge **MUST** be unreachable by untrusted clients through a path that bypasses the proxy. Same-host IPC used for authority decisions **MUST** authenticate peer identity and socket ownership; a pathname or loopback address alone is insufficient.

15.3. Token Theft and Replay

A capability token that is not sender-constrained is a bearer token for its validity interval: anyone who obtains it can present it. Signatures, validity intervals, and revocation bound the damage. A correctly bound sender proof ([Section 5.4](#)) prevents possession of the token alone from authorizing a call; compromise of the sender key remains a compromise of that authority.

The native transport carries no sender proof ([Section 10.2](#)), and it has no anti-replay marker of its own: a captured frame that carries a bearer token can be replayed until the token expires or is revoked. The MCP binding can bind a session to a sender key or a certificate thumbprint ([Section 11](#)). Operators **SHOULD** issue tokens with the shortest validity interval that the task allows, **SHOULD** require sender constraint for sensitive or cross-host flows where the surface supports it, and **SHOULD** revoke a token as soon as its holder is suspected of compromise ([Section 12.4](#)).

A kernel **MUST** durably record consumed sender-proof nonces for their full acceptance interval. All processes accepting proofs in the same authority domain **MUST** share atomic replay protection. An unavailable, corrupt, or rolled-back replay store **MUST** deny admission. Expiry-based collection **MUST** account for the profile's allowed clock skew and unresolved operations; restart is not a new replay epoch.

15.4. Kernel Impersonation and Receipt Trust

A receipt carries the public key that signed it (`kernel_key`), so a valid signature proves only that the holder of that key signed the receipt. A verifier **MUST** check that `kernel_key` is a key it trusts for the kernel that it expects to have made the decision, using a key it obtained out of band, before it treats a receipt as evidence. Receipt verification therefore depends on how a deployment distributes kernel keys; this document does not define a public transparency system for them ([Section 9.6](#)).

In the same way, a kernel accepts a capability token only from an issuer in its configured set of trusted issuer keys ([Section 5.8](#)). Distributions **SHOULD** pin or securely provision kernel keys, issuer keys, and certificates, rather than learning them from the traffic they protect.

Capability tokens carry no audience. Any kernel that trusts a token's issuer accepts the token for the grants it names. Operators that run kernels in different trust domains **SHOULD** use a distinct issuer key for each domain, so that a token issued for one domain is not accepted in another.

15.4.1. Signing Roles and Key Lifecycle

Trust is role-specific. Configuring a key to sign receipts **MUST NOT** make it a capability issuer, manifest publisher, approver, credential broker, or active-response authority. Deployments **MUST** authorize those roles explicitly and **SHOULD** use separate keys for independent trust domains.

A key lifecycle profile **MUST** specify activation, retirement, compromise handling, and historical verification. Rotation **MUST** have one unambiguous active signing authority per role and epoch; concurrent or stale workers **MUST NOT** continue signing as that active authority after its fence changes. Activation **MUST** follow the profile's authenticated transition and durable-state requirements. A failed rotation **MUST NOT** silently select an older or weaker signer.

An old public key retained for historical verification does not authorize new signatures. A verifier **MUST** apply its own trusted key history and compromise policy. An artifact's self-asserted timestamp cannot establish that it predates compromise. When a profile requires independent key-log witnesses, the roster and quorum **MUST** come from trusted configuration, and missing or conflicting witness evidence **MUST** block activation. Key-log and witness wire formats are companion-profile concerns; a receipt checkpoint alone does not supply a key-rotation protocol.

15.5. Delegation Abuse

An attacker who holds a delegated token can try to widen its scope, truncate its lineage, or claim a parent it does not hold. The attenuation proof and the chain-binding rule ([Section 5.6](#), [Section 5.7](#)) make a claimed parent scope verifiable against the trust root or the previous delegation link, and delegated issuance at the trust-control service cannot exceed its signed delegation policy ([Section 12.2](#)).

The kernel checks the revocation state of a token and of each ancestor that its delegation chain names ([Section 5.8](#)), so revoking a token denies every descendant whose chain names it. Operators **SHOULD** revoke the parent token when they suspect a compromise anywhere below it.

The kernel enforces delegated budget shares against the state it has recorded about sibling tokens ([Section 7.4](#)). Two kernels that do not share that state each admit children up to the full share, so a deployment that spreads one delegation family across kernel instances needs a single authority for that state to keep the bound.

15.6. Signature Algorithm Downgrade

Keys and signatures identify their own algorithm ([Section 4.4](#)), and a verifier rejects an algorithm hint that disagrees with the encoded signature. A verifier that requires post-quantum protection **MUST** enforce a minimum accepted signature suite and **MUST** reject a classical signature where that minimum requires a hybrid one ([Section 4.3](#)). A verifier that enforces no minimum accepts classical and hybrid signatures alike, and so does not protect against an attacker who can produce a classical signature for a key the verifier trusts.

15.7. Canonicalization

Signatures cover canonical bytes ([Section 4.1](#)), so two parties that parse the same bytes differently can disagree about what was signed. JSON objects with duplicate member names are the usual source of such disagreement. Verifiers **MUST** apply the original-byte parsing and numeric preservation requirements in [Section 4.1](#). Intermediaries **MUST** preserve every member and value in the artifact's typed signing projection when forwarding it. They **MAY** change insignificant JSON whitespace and member order, since verification reconstructs canonical bytes ([Section 10.5](#)). They **MUST NOT** add unauthenticated members and represent them as part of the signed statement.

15.8. Runtime Enforcement Profiles

Kernel admission controls entry to an operation. Confinement, credential custody, and information-flow policies control the authority exercised while that operation runs and the data released afterward. When a profile requires these controls, the kernel **MUST** establish them before dispatch and **MUST** deny service if any required control is unavailable. It **MUST NOT** fall back to a less restrictive execution path.

15.8.1. Tool Server Confinement

A confined execution profile **MUST** bind the selected tool to an authenticated manifest and an independently configured operator ceiling. The effective authority is their intersection. A signed manifest is accepted only from a publisher authorized for that workload; signature validity alone does not authorize its requested privileges.

The launcher **MUST** establish the required process, filesystem, syscall, network, environment, and descriptor restrictions before untrusted code runs. It **MUST** prevent executable, directory, or socket substitution between verification and use. Child processes **MUST** inherit restrictions that prevent escape from the operation's authority. A failed isolation setup **MUST** prevent launch. A pathname constraint on a tool argument is not an operating-system filesystem boundary.

A destination policy **MUST** cover the connection actually made, including name resolution, redirects, proxying, and endpoint changes. A hostname string comparison alone is insufficient. Profiles that restrict network destinations **MUST** use a trusted enforcement point capable of enforcing those restrictions. Portable artifacts do not establish that a particular host installed its promised controls; claims about confinement require authenticated evidence from that host and its stated trust assumptions.

15.8.2. Credential Custody

A credential-custody profile **MUST** keep provider secrets outside the agent and untrusted tool processes. A trusted broker performs the authorized provider operation without returning reusable credentials. Secrets **MUST NOT** be exposed through tool parameters, inherited environment variables, command-line arguments, IPC responses, logs, or receipts.

The broker **MUST** authenticate the invoking kernel and bind each operation to the verified capability, caller and tenant, target, parameters, resource reservation, and executor identity. It **MUST** reject destination substitution, stale authority, and replay. Broker-specific quota checks **MUST** participate in the same committed accounting decision as the operation's other required budgets; independent local counters do not establish a composite spend bound. An upstream timeout preserves an uncertain committed operation rather than restoring spend authority.

15.8.3. Information-Flow Control

A flow-enforced profile **MUST** bind the authenticated principal, tenant, lineage, session, isolation epoch, and context generation through [Section 5.6.1](#). The kernel **MUST** derive input sensitivity from trusted retained state and authenticated tool declarations, not from agent-selected labels. Unknown history **MUST** be treated as the profile's most restrictive label rather than as public data.

The kernel **MUST** propagate accumulated sensitivity through the operation and check the destination's policy-owned clearance before egress. Closing a session or changing an untrusted identifier **MUST NOT** clear principal or lineage sensitivity. A less restrictive successor requires an authenticated isolation transition or authorized declassification.

Declassification **MUST** bind the exact operation, data or label transition, permitted purpose and destination, approving authority, and expiry. Single-use declassification authority **MUST** be consumed durably and **MUST NOT** be restored after an uncertain dispatch. A tool or adapter that cannot preserve the required context **MUST** be rejected for that profile. These requirements constrain explicit flows; they do not establish elimination of timing, traffic-volume, or other covert channels.

15.8.4. Active Response

An active-response profile treats containment actions as governed kernel operations. Detection evidence is an input to policy, not authority to act. The kernel **MUST** authenticate its provenance and causal scope and verify the response plan, required capabilities, and approvals before applying a response. Untrusted tool output **MUST NOT** impersonate an internal security event or approval.

Simulation and live execution **MUST** be distinct authenticated modes. A dry-run result **MUST NOT** be accepted as evidence that a live action was applied. Reversible responses **MUST** retain durable ownership of their individual contributions so expiry or rollback removes only the state that response owns. Overlapping responses **MUST NOT** undo each other's restrictions. An uncertain rollback **MUST** remain unresolved and **MUST NOT** be recorded as successful restoration.

Permanent revocation requires separately authorized action; an automatic containment policy does not implicitly acquire that authority. Recovery **MUST** reconcile pending responses and stale ownership before the affected execution domain becomes ready. These requirements use the governance model in [Section 8](#) without defining a separate quorum or approval protocol.

15.9. Resource Exhaustion

A kernel rejects native frames larger than the maximum payload length before allocating a buffer for them ([Section 10.1](#)). The MCP binding limits each session to one notification stream and never resumes a session that has reached a terminal state ([Section 11.4](#)). Deployments **MUST** bound message size, nesting depth, array and string lengths, signature work, concurrency, queues, and operation duration at each untrusted boundary. They **MUST** apply limits before the corresponding allocation or expensive work where possible. Streaming and decompression need limits on cumulative decoded size as well as individual frames. Overload **MUST** reject or defer work without bypassing authorization, accounting, or durable finalization. An authenticated caller can still spend its own budget and queue share; budgets bound the cost of that, not its occurrence.

15.10. Budgets and Approvals

A budget hold is reserved before dispatch and captured once ([Section 7.3](#)); a kernel denies a request that tries to capture a hold a second time. Approval tokens and threshold approvals are single use ([Section 8.4](#), [Section 8.5](#)), and a kernel records the ones it has accepted. Both singular and threshold approvals require durable replay protection. Threshold approval **MUST** use the admission reservation in [Section 8.5.2](#) and **MUST** deny admission when that authority is unavailable. Reconstructing a request after restart **MUST NOT** produce a new approval or reservation for an already committed operation.

Threshold approval counts distinct keys. Distinct keys are not distinct people or organizations: an operator who configures an eligible key set decides how independent the approvers are.

15.11. Checkpoint Equivocation

A kernel signs its own checkpoints, so a kernel whose key is compromised, or a dishonest operator, can sign two checkpoints that disagree and show each to a different verifier. Consistency proofs let a verifier detect the disagreement when it holds both checkpoints ([Section 9.3](#)); nothing in this document makes a verifier hold both. [Section 9.6](#) states what a verified checkpoint establishes.

15.12. Provenance Inflation

Callers can supply context that the kernel cannot check, such as model metadata and call-chain information. The kernel records such context as asserted ([Section 8.6](#), [Section 11.7](#)). A report or export **MUST NOT** present asserted context as observed or verified.

15.13. Attestation

Attestation evidence can inform issuance ([Section 12.1](#)), but it does not authorize a call by itself. A profile that binds an attestation digest to a token **MUST** also bind the request to the token's sender, by a sender proof or by mutual TLS continuity, on the same request.

15.14. Time and Deadlines

Validity intervals are half-open: `issued_at <= now < expires_at` unless an artifact explicitly states a different rule. Remote timestamp acceptance can use only the skew allowance specified by that artifact's profile. An allowance for remote clocks **MUST NOT** excuse regression or failure of the kernel's own trusted clock.

The kernel **MUST** use checked time arithmetic and reject an unreadable, regressed, or unrepresentable clock value when it affects authority. It **MUST NOT** replace a failed time read with zero or extend a deadline through integer saturation. Issuers and kernels need sufficiently synchronized wall clocks for the selected validity intervals.

For an admitted operation, the effective remaining lifetime **MUST** be bounded by both its signed wall-clock deadline and its original elapsed-time budget. Retries, queue transfers, and restarts **MUST NOT** reset that budget. A monotonic timestamp from another process or boot is not a portable time authority. If recovery cannot conservatively establish the remaining lifetime, it **MUST** refuse new dispatch under that authorization.

16. Privacy Considerations

Blocking dispatch does not remove attempted arguments from a denial receipt. A pre-invocation guard can deny an attempt while its raw arguments remain in signed audit evidence; protect receipt storage and read access independently of tool admission.

Receipts are designed to be kept and shown to other parties, and they contain more than a verdict. A receipt records the tool server and tool, the capability identifier, the call's parameters as the agent sent them together with their hash, a hash of the output, the guards that were evaluated and their evidence, the chain of actors that the kernel attributes the call to, the time, and, in multi-tenant deployments, the tenant ([Section 6.1](#)). A receipt has no dedicated output field. Free-form metadata can still contain output or other sensitive data.

Parameters are the most sensitive part. Because a receipt signs the parameters that the agent sent, anyone who can read the receipt can read them, including secrets or personal data that the agent placed there. Agents and tool designers **SHOULD** keep secrets and personal data out of tool arguments where the tool allows it. Deployments that handle regulated data **SHOULD** evaluate a guard that detects and redacts or blocks sensitive values before the call, and **SHOULD** restrict who can query receipts.

Receipts are linkable. The capability identifier and the subject key connect every call made under one token, and a delegation chain reveals who delegated to whom. A party that sees many receipts can build a profile of an agent's activity and of the organization behind it.

The `redaction_mode` field records how much of a receipt's detail was redacted when it was signed or exported. It describes the receipt; it does not by itself remove anything.

A trust-control service limits receipt queries by tenant ([Section 12.3](#)). A tenant sees its own receipts; an administrator sees all of them. Checkpoints commit to batches of receipts from every tenant that a kernel serves, so the size and timing of a batch can reveal aggregate activity across tenants to anyone who can see the checkpoint.

When MCP logging is enabled, a hosted edge can send the client log messages that name denied tools and carry error text ([Section 11](#)). Operators **SHOULD** review what those messages reveal before enabling them for untrusted clients.

17. IANA Considerations

This document requests four registries under a new "Chio Protocol" registry group. Each uses Specification Required as defined in [[RFC8126](#)]. Registration takes effect through IANA; the initial allocations below are requests, not assertions that IANA has assigned these values.

String values in the message and artifact registries are case-sensitive ASCII names of 1 to 96 octets using lowercase letters, digits, underscore, hyphen, and period. Signature-suite names additionally permit plus. These are proposed registration rules; compatibility decoders can accept other key spellings as specified in [Section 4.4](#).

A registration identifies its value, name, description, and a stable specification. The designated experts check uniqueness, complete encoding and verification rules, and whether the specification states its security and interoperability consequences. A registration does not establish cryptographic suitability or implementation support. An update retains the meaning of previously allocated values; a change that breaks that meaning receives a new value.

17.1. Chio Error Codes

The registration template is: numeric code; symbolic name; category; transient flag; retry strategy and guidance; native error name, if any; JSON-RPC code, if any; and reference. Codes are decimal integers from 0 through 2147483647. Negative JSON-RPC codes belong to their separate namespace and are not values in this registry. Experts check that the retry guidance distinguishes a changed condition from repeating an unsafe side effect. The initial allocations follow. Core allocations reference this document. Values 7100 through 7113 are reserved for related transaction profiles, rather than allocated by this document. A future Specification Required registration supplies a permanent public specification before assigning any reserved value.

1000 (protocol_version_unsupported): Category protocol; transient false; retry do_not_retry_until_version_change. Select a mutually supported binding version before a new operation. This allocation does not replace MCP initialization negotiation.

1001 (session_not_initialized): Category protocol; transient false; retry reinitialize. JSON-RPC code: -32002. Open a new session and rerun initialize before retrying the operation.

1002 (invalid_request_shape): Category protocol; transient false; retry do_not_retry. JSON-RPC code: -32600. Correct the request shape before retrying.

1100 (auth_missing_or_invalid): Category auth; transient true; retry retry_after_refresh. Acquire valid credentials or refresh the token before retrying.

2100 (capability_denied): Category capability; transient false; retry do_not_retry. Native name: capability_denied. Do not retry the same capability without correcting signature, scope, or subject binding.

2101 (capability_expired): Category capability; transient false; retry retry_after_reissue. Native name: capability_expired. Issue or obtain a fresh capability before retrying.

2102 (capability_revoked): Category capability; transient false; retry retry_after_reissue. Native name: capability_revoked. Obtain a newly issued capability; the revoked capability must not be retried.

3100 (guard_denied): Category guard; transient false; retry do_not_retry. Native name: policy_denied. Adjust the request or policy inputs before retrying.

4100 (budget_exhausted): Category budget; transient true; retry retry_after_budget_change. Retry only after budget replenishment, grant widening, or billing reconciliation.

5100 (tool_server_error): Category tool; transient true; retry retry_with_backoff. Native name: tool_server_error. Establish safe recovery or idempotency before a bounded retry.

6100 (internal_error): Category internal; transient true; retry retry_with_backoff. Native name: internal_error. Resolve the original operation's state before a bounded retry.

Values 7100 through 7113 are Reserved. All other values not allocated above are Unassigned. Reservation does not define a wire error or imply support for a companion transaction profile.

17.2. Chio Signature Suites

The registration template is: suite name; public-key encoding; signature encoding; message preparation and hash; verification rule; and reference. Experts check that key and signature encodings are unambiguous and that composite suites specify every component and require all components to verify. [Section 4.3](#) and [Section 4.4](#) define the initial values:

- ed25519: Ed25519, bare hexadecimal key and signature.
- p256: ECDSA P-256 with SHA-256; SEC1 uncompressed key and DER signature.
- p384: ECDSA P-384 with SHA-384; SEC1 uncompressed key and DER signature.
- ed25519+mldssa65: Ed25519 and ML-DSA-65, composite encoding.
- p256+mldssa65: ECDSA P-256 and ML-DSA-65, composite encoding.
- p384+mldssa65: ECDSA P-384 and ML-DSA-65, composite encoding.

The envelope hint `hybrid` identifies a composite. It is not a seventh suite and does not select its components. The composite's encoded algorithm set selects them.

17.3. Chio Native Message Types

The registration template is: type value; direction (agent to kernel or kernel to agent); payload fields and types; terminal or streaming behavior; and reference. Experts check that state transitions, correlation, and failure behavior are defined. Initial values are:

Value	Direction	Reference
<code>tool_call_request</code>	Agent to kernel	Section 10.2
<code>list_capabilities</code>	Agent to kernel	Section 10.2
<code>heartbeat</code>	Both directions	Section 10.2
<code>tool_call_chunk</code>	Kernel to agent	Section 10.2
<code>tool_call_response</code>	Kernel to agent	Section 10.2
<code>capability_list</code>	Kernel to agent	Section 10.2
<code>capability_revoked</code>	Kernel to agent	Section 10.2

Table 49: Initial Native Message Types

The shared `heartbeat` allocation defines a payload in each direction. Registering a new type does not make old decoders accept it. A future wire version or explicit extension agreement defines its availability.

17.4. Chio Signed Artifact Schemas

The registration template is: schema identifier; artifact role; exact signed projection; canonical encoding; domain separation, if any; verification procedure; and reference. Experts check that a verifier cannot confuse artifacts with different authority or trust roles. A registry identifier can name a projection without appearing as a member in its wire envelope. In particular, receipt records carry no top-level schema member.

Initial values in the scope of this document are:

- `chio.capability.v1`: capability token ([Section 5](#)).
- `chio.receipt.v1`: receipt signing projection ([Section 6](#)).
- `chio.aggregate-budget-root.v1`: aggregate budget root ([Section 7.5](#)).
- `chio.execution_nonce.v1`: legacy execution nonce ([Section 7.7](#)).
- `chio.execution_nonce.v2`: operation-owned execution nonce ([Section 7.7.1](#)).
- `chio.dpop_proof.v1`: Chio sender proof ([Section 5.4](#)).
- `chio.receipt_lineage_statement.v1`: signed receipt relationship ([Section 6.7](#)).

- `chio.call_chain_continuation.v1`: call-chain continuation ([Section 8.7](#)).
- `chio.threshold-approval-proposal.v1`: threshold proposal ([Section 8.5](#)).
- `chio.checkpoint_statement.v2`: signed receipt checkpoint ([Section 9.1](#)).

Approval tokens, delegation links, and child receipts do not gain a new schema member from this registration. Unsigned proof records do not become signed artifacts through inclusion in a registry.

18. Implementation Status

This section is to be removed before publishing as an RFC.

This section follows [[RFC7942](#)] and is removed before publication as an RFC. It describes the implementation supporting this specification; implementation availability is distinct from IETF endorsement or independent interoperability results.

Chio is implemented as a modern Rust kernel for agentic operating systems. Its source is published under Apache-2.0 at <https://github.com/backbay-labs/chio>. The contact is Connor Whelan, connor@backbay.io.

The implementation provides the signed capability and receipt formats, framed native transport, hosted MCP binding, trust-control interfaces, authenticated attenuation, budget accounting, governed approvals, and receipt checkpoints specified here. The runtime composes these with authenticated process context, durable execution and recovery, confined tool processes, brokered credentials, information-flow enforcement, and governed active response. Cryptographic suites and host mechanisms are selected through explicit build and deployment profiles.

The production execution profile uses a single operator authority domain with durable stores and fenced ownership. Its resource guarantee is `single_node_atomic`; it does not advertise `ha_linearizable` merely because multiple clients or processes use that authority. Native confinement is a platform profile, not a property inferred from an artifact verifier or transport adapter. Public checkpoint witnessing and cross-operator federation have separate trust and availability requirements.

Conformance artifacts exercise capability validation, attenuation, receipt integrity, revocation, sender binding, governed transactions, and the native framing contract. Binding examples and cryptographic vectors are given in [Appendix A](#) and [Appendix B](#). Runtime qualification additionally exercises concurrency, process failure, restart, confinement, credential custody, and retention. These test categories distinguish wire interoperability from deployment assurance; no independent multi-implementation interoperability result is claimed by this section.

19. References

19.1. Normative References

-
- [FIPS180-4]** National Institute of Standards and Technology, "Secure Hash Standard (SHS)", FIPS PUB 180-4, DOI 10.6028/NIST.FIPS.180-4, August 2015, <<https://doi.org/10.6028/NIST.FIPS.180-4>>.
- [FIPS186-5]** National Institute of Standards and Technology, "Digital Signature Standard (DSS)", FIPS PUB 186-5, DOI 10.6028/NIST.FIPS.186-5, February 2023, <<https://doi.org/10.6028/NIST.FIPS.186-5>>.
- [FIPS204]** National Institute of Standards and Technology, "Module-Lattice-Based Digital Signature Standard", FIPS PUB 204, DOI 10.6028/NIST.FIPS.204, August 2024, <<https://doi.org/10.6028/NIST.FIPS.204>>.
- [ISO4217]** International Organization for Standardization, "Codes for the representation of currencies", ISO 4217:2015, August 2015, <<https://www.iso.org/iso-4217-currency-codes.html>>.
- [JSON-RPC]** JSON-RPC Working Group, "JSON-RPC 2.0 Specification", January 2013, <<https://www.jsonrpc.org/specification>>.
- [MCP]** Model Context Protocol, "Model Context Protocol Specification, Version 2025-11-25", 25 November 2025, <<https://modelcontextprotocol.io/specification/2025-11-25>>.
- [RFC2119]** Bradner, S., "Key words for use in RFCs to Indicate Requirement Levels", BCP 14, RFC 2119, DOI 10.17487/RFC2119, March 1997, <<https://www.rfc-editor.org/rfc/rfc2119>>.
- [RFC3279]** Bassham, L., Polk, W., and R. Housley, "Algorithms and Identifiers for the Internet X.509 Public Key Infrastructure Certificate and Certificate Revocation List (CRL) Profile", RFC 3279, DOI 10.17487/RFC3279, May 2002, <<https://www.rfc-editor.org/rfc/rfc3279>>.
- [RFC4648]** Josefsson, S., "The Base16, Base32, and Base64 Data Encodings", RFC 4648, DOI 10.17487/RFC4648, October 2006, <<https://www.rfc-editor.org/rfc/rfc4648>>.
- [RFC7519]** Jones, M., Bradley, J., and N. Sakimura, "JSON Web Token (JWT)", RFC 7519, DOI 10.17487/RFC7519, May 2015, <<https://www.rfc-editor.org/rfc/rfc7519>>.
- [RFC7662]** Richer, J., Ed., "OAuth 2.0 Token Introspection", RFC 7662, DOI 10.17487/RFC7662, October 2015, <<https://www.rfc-editor.org/rfc/rfc7662>>.
- [RFC8032]** Josefsson, S. and I. Liusvaara, "Edwards-Curve Digital Signature Algorithm (EdDSA)", RFC 8032, DOI 10.17487/RFC8032, January 2017, <<https://www.rfc-editor.org/rfc/rfc8032>>.
- [RFC8126]** Cotton, M., Leiba, B., and T. Narten, "Guidelines for Writing an IANA Considerations Section in RFCs", BCP 26, RFC 8126, DOI 10.17487/RFC8126, June 2017, <<https://www.rfc-editor.org/rfc/rfc8126>>.
-

- [RFC8174] Leiba, B., "Ambiguity of Uppercase vs Lowercase in RFC 2119 Key Words", BCP 14, RFC 8174, DOI 10.17487/RFC8174, May 2017, <<https://www.rfc-editor.org/rfc/rfc8174>>.
- [RFC8259] Bray, T., Ed., "The JavaScript Object Notation (JSON) Data Interchange Format", STD 90, RFC 8259, DOI 10.17487/RFC8259, December 2017, <<https://www.rfc-editor.org/rfc/rfc8259>>.
- [RFC8446] Rescorla, E., "The Transport Layer Security (TLS) Protocol Version 1.3", RFC 8446, DOI 10.17487/RFC8446, August 2018, <<https://www.rfc-editor.org/rfc/rfc8446>>.
- [RFC8725] Sheffer, Y., Hardt, D., and M. Jones, "JSON Web Token Best Current Practices", BCP 225, RFC 8725, DOI 10.17487/RFC8725, February 2020, <<https://www.rfc-editor.org/rfc/rfc8725>>.
- [RFC8785] Rundgren, A., Jordan, B., and S. Erdtman, "JSON Canonicalization Scheme (JCS)", RFC 8785, DOI 10.17487/RFC8785, June 2020, <<https://www.rfc-editor.org/rfc/rfc8785>>.
- [RFC9110] Fielding, R., Ed., Nottingham, M., Ed., and J. Reschke, Ed., "HTTP Semantics", STD 97, RFC 9110, DOI 10.17487/RFC9110, June 2022, <<https://www.rfc-editor.org/rfc/rfc9110>>.
- [RFC9162] Laurie, B., Messeri, E., and R. Stradling, "Certificate Transparency Version 2.0", RFC 9162, DOI 10.17487/RFC9162, December 2021, <<https://www.rfc-editor.org/rfc/rfc9162>>.
- [RFC9449] Fett, D., Campbell, B., Bradley, J., Lodderstedt, T., Jones, M., and D. Waite, "OAuth 2.0 Demonstrating Proof of Possession (DPoP)", RFC 9449, DOI 10.17487/RFC9449, September 2023, <<https://www.rfc-editor.org/rfc/rfc9449>>.
- [RFC9728] Jones, M.B., Hunt, P., and A. Parecki, "OAuth 2.0 Protected Resource Metadata", RFC 9728, DOI 10.17487/RFC9728, April 2025, <<https://www.rfc-editor.org/rfc/rfc9728>>.
- [SEC1] Standards for Efficient Cryptography Group, "SEC 1: Elliptic Curve Cryptography, Version 2.0", May 2009, <<https://www.secg.org/sec1-v2.pdf>>.
- [SSE] WHATWG, "HTML Living Standard, Section 9.2: Server-sent events", <<https://html.spec.whatwg.org/multipage/server-sent-events.html>>.

19.2. Informative References

- [A2A] Linux Foundation A2A Project, "Agent2Agent (A2A) Protocol Specification", <<https://a2a-protocol.org/latest/specification/>>.
- [BISCUIT] Eclipse Biscuit, "Biscuit, a bearer token with offline attenuation and decentralized verification", <<https://doc.biscuitsec.org/reference/specifications>>.

- [I-D.ietf-wimse-arch-08]** Salowey, J. A., Rosomakho, Y., and H. Tschofenig, "Workload Identity in a Multi System Environment (WIMSE) Architecture", Work in Progress, Internet-Draft, draft-ietf-wimse-arch-08, 6 July 2026, <<https://datatracker.ietf.org/doc/html/draft-ietf-wimse-arch-08>>.
- [MACAROONS]** Birgisson, A., Politz, J. G., Erlingsson, U., Taly, A., Vrabie, M., and M. Lenczner, "Macaroons: Cookies with Contextual Caveats for Decentralized Authorization in the Cloud", Network and Distributed System Security Symposium NDSS 2014, February 2014, <<https://www.ndss-symposium.org/ndss2014/programme/macaroons-cookies-contextual-caveats-decentralized-authorization-cloud/>>.
- [OID4VCI]** OpenID Foundation, "OpenID for Verifiable Credential Issuance 1.0", 16 September 2025, <https://openid.net/specs/openid-4-verifiable-credential-issuance-1_0.html>.
- [OID4VP]** OpenID Foundation, "OpenID for Verifiable Presentations 1.0", 9 July 2025, <https://openid.net/specs/openid-4-verifiable-presentations-1_0.html>.
- [RFC3552]** Rescorla, E. and B. Korver, "Guidelines for Writing RFC Text on Security Considerations", BCP 72, RFC 3552, DOI 10.17487/RFC3552, July 2003, <<https://www.rfc-editor.org/rfc/rfc3552>>.
- [RFC7942]** Sheffer, Y. and A. Farrel, "Improving Awareness of Running Code: The Implementation Status Section", BCP 205, RFC 7942, DOI 10.17487/RFC7942, July 2016, <<https://www.rfc-editor.org/rfc/rfc7942>>.
- [RFC8693]** Jones, M., Nadalin, A., Campbell, B., Ed., Bradley, J., and C. Mortimore, "OAuth 2.0 Token Exchange", RFC 8693, DOI 10.17487/RFC8693, January 2020, <<https://www.rfc-editor.org/rfc/rfc8693>>.
- [RFC8792]** Watsen, K., Auerswald, E., Farrel, A., and Q. Wu, "Handling Long Lines in Content of Internet-Drafts and RFCs", RFC 8792, DOI 10.17487/RFC8792, June 2020, <<https://www.rfc-editor.org/rfc/rfc8792>>.
- [RFC9334]** Birkholz, H., Thaler, D., Richardson, M., Smith, N., and W. Pan, "Remote Attestation procedureS (RATS) Architecture", RFC 9334, DOI 10.17487/RFC9334, January 2023, <<https://www.rfc-editor.org/rfc/rfc9334>>.
- [RFC9396]** Lodderstedt, T., Richer, J., and B. Campbell, "OAuth 2.0 Rich Authorization Requests", RFC 9396, DOI 10.17487/RFC9396, May 2023, <<https://www.rfc-editor.org/rfc/rfc9396>>.
- [RFC9635]** Richer, J., Ed. and F. Imbault, "Grant Negotiation and Authorization Protocol (GNAP)", RFC 9635, DOI 10.17487/RFC9635, October 2024, <<https://www.rfc-editor.org/rfc/rfc9635>>.
- [RFC9943]** Birkholz, H., Delignat-Lavaud, A., Fournet, C., Deshpande, Y., and S. Lasker, "An Architecture for Trustworthy and Transparent Digital Supply Chains", RFC 9943, DOI 10.17487/RFC9943, June 2026, <<https://www.rfc-editor.org/rfc/rfc9943>>.

[UCAN] UCAN Working Group, "User Controlled Authorization Network (UCAN) Specification", <<https://github.com/ucan-wg/spec>>.

[VC-DATA-MODEL-2.0] World Wide Web Consortium, "Verifiable Credentials Data Model v2.0", 15 May 2025, <<https://www.w3.org/TR/vc-data-model-2.0/>>.

Appendix A. Examples

The examples in this appendix are built from two cases of the Chio binding vector corpus, `capability/v1.json` case `valid_delegated_capability` and `receipt/v1.json` case `allow_receipt`, so every key, hash, and signature in them is exact. Lines longer than 69 characters are folded as described in [RFC8792], and each folded block begins with the header line that document defines.

A.1. Capability Token

The following case is `capability/v1.json` case `valid_delegated_capability` from the Chio binding vector corpus. It is a token with one delegation link and one tool grant, pretty-printed here for reading. The corpus omits the `schema` member from its tokens. Capability tokens are defined in [Section 5](#).

===== NOTE: '\' line wrapping per RFC 8792 =====

```
{
  "delegation_chain": [
    {
      "capability_id": "cap-bindings-valid",
      "delegatee": "91a28a0b74381593a4d9469579208926afc8ad82c8839b76\
44359b9eba9a4b3a",
      "delegator": "66be7e332c7a453332bd9d0a7f7db055f5c5ef1a06ada66d\
98b39fb6810c473a",
      "signature": "a4c297402d3bef255c8a9648215d9bb9c7fcbfb0a2e4009c\
fc6202459c10b2e4066de4ef60d4118f612993b6ceb2197095483e94c8\
a2ccc658ad7ef5b865be00",
      "timestamp": 1710000250
    }
  ],
  "expires_at": 1710000800,
  "id": "cap-bindings-valid",
  "issued_at": 1710000200,
  "issuer": "66be7e332c7a453332bd9d0a7f7db055f5c5ef1a06ada66d98b39fb\
6810c473a",
  "scope": {
    "grants": [
      {
        "constraints": [
          {
            "type": "path_prefix",
            "value": "/workspace/"
          }
        ],
        "max_invocations": 3,
        "operations": [
          "invoke",
          "read_result"
        ],
        "server_id": "srv-files",
        "tool_name": "file_read"
      }
    ]
  },
  "signature": "a16f5a447da2a5dbd840e44c7c1a5093474e4a1b9826ae80ca85\
f197b18e8ad05607745ac9e0ff063db7b49ff4da637b630cf10d55c1ef1e64\
fb5c27d81ed505",
  "subject": "0b513ad9b4924015ca0902ed079044d3ac5dbec2306f06948c10da\
8eb6e39f2d"
}
```

The signing input of the token is its canonical form without the signature member and with the member "schema":"chio.capability.v1" added, which sorts between issuer and scope. The capability_body_canonical_json value of the case is the same text without the schema member. The signing input is:

```
===== NOTE: '\' line wrapping per RFC 8792 =====  
  
{  
  "delegation_chain":[{"capability_id":"cap-bindings-valid",\  
    "delegatee":\  
    "91a28a0b74381593a4d9469579208926afc8ad82c8839b7644359b9eba9a4b3a",\  
    "delegator":\  
    "66be7e332c7a453332bd9d0a7f7db055f5c5ef1a06ada66d98b39fb6810c473a",\  
    "signature":"a4c297402d3bef255c8a9648215d9bb9c7fcbfb0a2e4009cfc62024\  
59c10b2e4066de4ef60d4118f612993b6ceb2197095483e94c8a2ccc658ad7ef5b86\  
5be00", "timestamp":1710000250}], "expires_at":1710000800, "id":\  
    "cap-bindings-valid", "issued_at":1710000200, "issuer":\  
    "66be7e332c7a453332bd9d0a7f7db055f5c5ef1a06ada66d98b39fb6810c473a",\  
    "schema":"chio.capability.v1", "scope":{"grants":[{"constraints":\  
    [{"type":"path_prefix", "value":"/workspace/"}], "max_invocations":3,\  
    "operations":["invoke", "read_result"], "server_id":"srv-files",\  
    "tool_name":"file_read"}]}, "subject":\  
    "0b513ad9b4924015ca0902ed079044d3ac5dbec2306f06948c10da8eb6e39f2d"}  
}
```

The signature member is an Ed25519 signature over these octets that verifies under the key in issuer.

A.2. Tool Call Request Frame

The following request is built from `capability/v1.json` case `valid_delegated_capability` and `receipt/v1.json` case `allow_receipt` of the Chio binding vector corpus. Its `capability_token` is the token of [Appendix A.1](#), and its `server_id`, `tool`, and `params` are the `tool_server`, `tool_name`, and `action.parameters` members of the receipt of [Appendix A.3](#). Its `id` is illustrative. The request is not a case of the corpus, and its canonical form is:

```
===== NOTE: '\\' line wrapping per RFC 8792 =====

{"capability_token":{"delegation_chain":[{"capability_id":\
"cap-bindings-valid","delegatee":\
"91a28a0b74381593a4d9469579208926afc8ad82c8839b7644359b9eba9a4b3a",\
"delegator":\
"66be7e332c7a453332bd9d0a7f7db055f5c5ef1a06ada66d98b39fb6810c473a",\
"signature":"a4c297402d3bef255c8a9648215d9bb9c7fcbfb0a2e4009cfc62024\
59c10b2e4066de4ef60d4118f612993b6ceb2197095483e94c8a2ccc658ad7ef5b86\
5be00","timestamp":1710000250}], "expires_at":1710000800,"id":\
"cap-bindings-valid","issued_at":1710000200,"issuer":\
"66be7e332c7a453332bd9d0a7f7db055f5c5ef1a06ada66d98b39fb6810c473a",\
"scope":{"grants":[{"constraints":[{"type":"path_prefix","value":\
"/workspace/"}], "max_invocations":3, "operations":["invoke",\
"read_result"], "server_id":"srv-files", "tool_name":"file_read"}]},\
"signature":"a16f5a447da2a5dbd840e44c7c1a5093474e4a1b9826ae80ca85f19\
7b18e8ad05607745ac9e0ff063db7b49ff4da637b630cf10d55c1ef1e64fb5c27d81\
ed505","subject":\
"0b513ad9b4924015ca0902ed079044d3ac5dbec2306f06948c10da8eb6e39f2d"},\
"id":"req-001","params":{"mode":"read","path":\
"/workspace/docs/roadmap.md"},"server_id":"srv-files","tool":\
"file_read","type":"tool_call_request"}
```

In the native transport ([Section 10](#)), a message is sent as a frame: a four-octet length in big-endian order, followed by the canonical form as the payload. The payload here is 1111 octets long, so the length prefix is:

```
00000457
```

The first 32 octets of the payload are the ASCII text `{"capability_token":{"delegation`. In hex they are:

```
7b226361706162696c6974795f7466b656e223a7b2264656c6567617469666e
```

A.3. Receipt

The following case is `receipt/v1.json` case `allow_receipt` from the Chio binding vector corpus. It records an allow decision for a call with the server, tool, and parameters of [Appendix A.2](#), and it is pretty-printed here for reading. Its `capability_id` is `cap-bindings-001`. The capability and receipt cases of the corpus are independent, so that value is not the id of the token in [Appendix A.1](#). Receipts are defined in [Section 6](#).

```
===== NOTE: '\\' line wrapping per RFC 8792 =====
{
  "action": {
    "parameter_hash": "035350b0e6a021a4c924a149111944945e94d12f6d8a5\
e24e4461e7009ebf929",
    "parameters": {
      "mode": "read",
      "path": "/workspace/docs/roadmap.md"
    }
  },
  "boundary_class": "prevent",
  "capability_id": "cap-bindings-001",
  "content_hash": "4062edaf750fb8074e7e83e0c9028c94e32468a8b6f161477\
4328ef045150f93",
  "decision": {
    "verdict": "allow"
  },
  "evidence": [
    {
      "details": "path allowed",
      "guard_name": "ForbiddenPathGuard",
      "verdict": true
    },
    {
      "details": "no secrets detected",
      "guard_name": "SecretLeakGuard",
      "verdict": true
    }
  ],
  "id": "c9909f733d5fb367922293f0167d06db715393be114f212096cf2866f29\
433ea",
  "kernel_key": "ea4a6c63e29c520abef5507b132ec5f9954776aebefe7b92421\
eea691446d22c",
  "metadata": {
    "surface": "bindings-vectors",
    "version": 1
  },
  "policy_hash": "policy-bindings-v1",
  "receipt_kind": "mediated_decision",
  "redaction_mode": "none",
  "signature": "276a4c59dbc5e0c4fbd47beefa33515d207e8f516aca296afee2\
cb95a443bb18d31031cae0b69943e7a930be03b2e0a60156e5a26e732e888e\
32d4653d059f03",
  "timestamp": 1710000200,
  "tool_name": "file_read",
}
```

```
"tool_origin": "caller_executed",
"tool_server": "srv-files",
"trust_level": "mediated"
}
```

The signing input of the receipt is the canonical form of a JSON object with two members: `body`, which is the receipt without its `id` and `signature` members, and `id`, which is the SHA-256 digest of the canonical form of `body`, in lowercase hex. The `receipt_body_canonical_json` value of the case is the canonical form of the receipt without its `signature` member. The signing input is:

```
===== NOTE: '\\' line wrapping per RFC 8792 =====

{"body":{"action":{"parameter_hash":\
"035350b0e6a021a4c924a149111944945e94d12f6d8a5e24e4461e7009ebf929",\
"parameters":{"mode":"read","path":"/workspace/docs/roadmap.md"}},\
"boundary_class":"prevent","capability_id":"cap-bindings-001",\
"content_hash":\
"4062edaf750fb8074e7e83e0c9028c94e32468a8b6f1614774328ef045150f93",\
"decision":{"verdict":"allow"},"evidence":[{"details":\
"path_allowed","guard_name":"ForbiddenPathGuard","verdict":true},\
{"details":"no secrets detected","guard_name":"SecretLeakGuard",\
"verdict":true}], "kernel_key":\
"ea4a6c63e29c520abef5507b132ec5f9954776aebebe7b92421eea691446d22c",\
"metadata":{"surface":"bindings-vectors","version":1},"policy_hash":\
"policy-bindings-v1","receipt_kind":"mediated_decision",\
"redaction_mode":"none","timestamp":1710000200,"tool_name":\
"file_read","tool_origin":"caller_executed","tool_server":\
"srv-files","trust_level":"mediated"},"id":\
"c9909f733d5fb367922293f0167d06db715393be114f212096cf2866f29433ea"}
```

The signature member is an Ed25519 signature over these octets that verifies under the key in `kernel_key`. The `parameter_hash` member of `action` is the SHA-256 digest of the canonical form of its `parameters` member, in the same encoding.

Appendix B. Test Vectors

The cases in this appendix come from the files `canonical/v1.json`, `hashing/v1.json`, `signing/v1.json`, `capability/v1.json`, and `receipt/v1.json` of the Chio binding vector corpus, and all keys in them are test keys derived from seeds that the corpus publishes. Lines longer than 69 characters are folded as described in [RFC8792], and each folded block begins with the header line that document defines.

B.1. Canonical JSON

Each case gives a JSON text and its canonical form, which [Section 4](#) defines.

The following case is `canonical/v1.json` case `object_key_sorting` from the Chio binding vector corpus. Members are sorted by key. The input is:

```
{"z":1, "a":2, "m":3}
```

The canonical form is:

```
{"a":2, "m":3, "z":1}
```

The following case is `canonical/v1.json` case `utf16_key_ordering` from the Chio binding vector corpus. Keys are compared as UTF-16 code units, so the key U+10000, which the input writes as a surrogate pair, sorts before the key U+E000, which is the reverse of their order as UTF-8 octets. The input is:

```
{"\ue000":1, "\ud800\udc00":2}
```

The canonical form holds characters outside ASCII, so it is shown as its 18 octets, in hex:

```
7b 22 f0 90 80 80 22 3a 32 2c 22 ee 80 80 22 3a 31 7d
```

The following case is `canonical/v1.json` case `nested_structures` from the Chio binding vector corpus. Members are sorted at every level of nesting, and the order of array elements is unchanged. The input is:

```
===== NOTE: '\' line wrapping per RFC 8792 =====  
{  
  "tool": "read",  
  "params": {  
    "path": "/tmp/demo",  
    "flags": ["read",  
      "text"]  
  },  
  "enabled": true  
}
```

The canonical form is:

```
===== NOTE: '\' line wrapping per RFC 8792 =====  
{  
  "enabled": true,  
  "params": {  
    "flags": ["read", "text"],  
    "path": "\br/>    /tmp/demo"  
  },  
  "tool": "read"  
}
```

The following case is `canonical/v1.json` case `number_formatting` from the Chio binding vector corpus. Numbers are written as ECMAScript writes them: `1.0` becomes `1`, negative zero becomes `0`, and large and small magnitudes use exponent forms such as `1e+21` and `1e-7`. The input is:

```
{"whole":1.0, "small":1e-7, "big":1e21, "negative_zero":-0.0}
```

The canonical form is:

```
{"big":1e+21,"negative_zero":0,"small":1e-7,"whole":1}
```

B.2. SHA-256 Digests

Each case gives the octets of a UTF-8 string and their SHA-256 digest [[FIPS180-4](#)], written as 64 lowercase hex digits.

The following case is hashing `/v1.json` case `abc_lowercase` from the Chio binding vector corpus. The input is the ASCII string `abc`, which is three octets:

```
61 62 63
```

The digest is:

```
ba7816bf8f01cfea414140de5dae2223b00361a396177a9cb410ff61f20015ad
```

The following case is hashing `/v1.json` case `unicode_utf8` from the Chio binding vector corpus. The input is the word `chio`, a space, and the character `U+1F510`, which is 9 octets in UTF-8:

```
63 68 69 6f 20 f0 9f 94 90
```

The digest is:

```
5c57c7deea3998c13730797e592abe72cae55130e8da2923de6374b75b83101b
```

The following case is hashing `/v1.json` case `json_canonical_form` from the Chio binding vector corpus. The input is a canonical JSON text:

```
{"a":1,"b":[2,3]}
```

The digest is:

```
efbd0040190fb0871831e606c581f8a66db79d8e2bb836745a70051306956070
```

B.3. Ed25519 Signatures

In each case the message is the canonical form of the JSON input, and the signature is an Ed25519 signature [[RFC8032](#)] over the UTF-8 octets of that message.

The following case is `signing/v1.json` case `valid_canonical_json_message` from the Chio binding vector corpus. The input is:

```
{"z":1,"a":2}
```

Its canonical form, which is the message, is:

```
{"a":2,"z":1}
```

The Ed25519 public key is:

```
fd1724385aa0c75b64fb78cd602fa1d991fdebf76b13c58ed702eac835e9f618
```

The signature is:

```
===== NOTE: '\\' line wrapping per RFC 8792 =====  
0b656434a9cf4f1d2f5cd0a2e008d69789251ecde8373781ee24d22f36abe53abf9f\  
d51ee04df5c090143a5d49e14324a2d52f628472f02719bd405d2567590e
```

Verification is expected to succeed.

The following case is `signing/v1.json` case `tampered_canonical_json_message` from the Chio binding vector corpus. It reuses the signature of case `valid_canonical_json_message` for a message whose value of `z` differs. The input is:

```
{"z":2,"a":2}
```

Its canonical form, which is the message, is:

```
{"a":2,"z":2}
```

The Ed25519 public key is:

```
fd1724385aa0c75b64fb78cd602fa1d991fdebf76b13c58ed702eac835e9f618
```

The signature is:

```
===== NOTE: '\' line wrapping per RFC 8792 =====  
  
0b656434a9cf4f1d2f5cd0a2e008d69789251ecde8373781ee24d22f36abe53abf9f\  
d51ee04df5c090143a5d49e14324a2d52f628472f02719bd405d2567590e
```

Verification is expected to fail.

B.4. Capability Token Verification

Each case gives the signing input of a token, the value of its signature member, the Unix time at which the token is verified, and the result that verification is expected to produce. The signing input is the canonical form of the token without its signature member and with the member "schema": "chio.capability.v1" added, as in [Appendix A.1](#). The corpus omits the schema member from the token itself, and its capability_body_canonical_json value is the signing input without that member.

The following case is capability/v1.json case valid_delegated_capability from the Chio binding vector corpus. Its signature verifies, its delegation link verifies, and the verification time falls within its validity period. Its signing input is:

```
===== NOTE: '\' line wrapping per RFC 8792 =====  
  
{ "delegation_chain": [ { "capability_id": "cap-bindings-valid", \  
  "delegatee": \  
    "91a28a0b74381593a4d9469579208926afc8ad82c8839b7644359b9eba9a4b3a", \  
    "delegator": \  
      "66be7e332c7a453332bd9d0a7f7db055f5c5ef1a06ada66d98b39fb6810c473a", \  
    "signature": "a4c297402d3bef255c8a9648215d9bb9c7fcbfb0a2e4009cfc62024\  
59c10b2e4066de4ef60d4118f612993b6ceb2197095483e94c8a2ccc658ad7ef5b86\  
5be00", "timestamp": 1710000250 } ], "expires_at": 1710000800, "id": \  
  "cap-bindings-valid", "issued_at": 1710000200, "issuer": \  
    "66be7e332c7a453332bd9d0a7f7db055f5c5ef1a06ada66d98b39fb6810c473a", \  
  "schema": "chio.capability.v1", "scope": { "grants": [ { "constraints": \  
    [ { "type": "path_prefix", "value": "/workspace/" } ], "max_invocations": 3, \  
    "operations": [ "invoke", "read_result" ], "server_id": "srv-files", \  
    "tool_name": "file_read" } ] }, "subject": \  
  "0b513ad9b4924015ca0902ed079044d3ac5dbec2306f06948c10da8eb6e39f2d" }
```

The value of the signature member is:

```
===== NOTE: '\' line wrapping per RFC 8792 =====  
  
a16f5a447da2a5dbd840e44c7c1a5093474e4a1b9826ae80ca85f197b18e8ad05607\  
745ac9e0ff063db7b49ff4da637b630cf10d55c1ef1e64fb5c27d81ed505
```

Verification at Unix time 1710000400 is expected to produce:

```
{
  "delegation_chain_shape_valid": true,
  "signature_valid": true,
  "time_status": "valid",
  "time_valid": true
}
```

The following case is `capability/v1.json` case `expired_capability` from the Chio binding vector corpus. Its signature verifies, and it expired at 1710000100, before the verification time. Its signing input is:

```
===== NOTE: '\' line wrapping per RFC 8792 =====

{"expires_at":1710000100,"id":"cap-bindings-expired","issued_at":\
1710000000,"issuer":\
"66be7e332c7a453332bd9d0a7f7db055f5c5ef1a06ada66d98b39fb6810c473a",\
"schema":"chio.capability.v1","scope":{"grants":[{"constraints":\
[{"type":"path_prefix","value":"/workspace/"}],"max_invocations":3,\
"operations":["invoke","read_result"],"server_id":"srv-files",\
"tool_name":"file_read"}]},"subject":\
"0b513ad9b4924015ca0902ed079044d3ac5dbec2306f06948c10da8eb6e39f2d"}
```

The value of the signature member is:

```
===== NOTE: '\' line wrapping per RFC 8792 =====

d4f16e2dc56f829a047cf339e7139c0ceb859045f4854b746bdecf6901fb493a1088\
bab6456d2c39b38cc955f84041ebd22d8636bf4cfc45e838423b868de509
```

Verification at Unix time 1710000400 is expected to produce:

```
{
  "delegation_chain_shape_valid": true,
  "signature_valid": true,
  "time_status": "expired",
  "time_valid": false
}
```

The following case is `capability/v1.json` case `broken_delegation_chain_signature` from the Chio binding vector corpus. Its signature verifies, and the signature of its delegation link does not. Its signing input is:

```
===== NOTE: '\\' line wrapping per RFC 8792 =====

{"delegation_chain":[{"capability_id":"cap-bindings-broken-chain",\
"delegatee":\
"91a28a0b74381593a4d9469579208926afc8ad82c8839b7644359b9eba9a4b3a",\
"delegator":\
"66be7e332c7a453332bd9d0a7f7db055f5c5ef1a06ada66d98b39fb6810c473a",\
"signature":"e5031fd5c95e1cb79201c9750831e6604ee65fa451202f0ef540e9e\
30f7bc8e73ca877424e7ca3121db39cbb57e97827878a8656b077a7c084f9bfeeb84\
66f0e", "timestamp":1710000301}], "expires_at":1710000800, "id":\
"cap-bindings-broken-chain", "issued_at":1710000200, "issuer":\
"66be7e332c7a453332bd9d0a7f7db055f5c5ef1a06ada66d98b39fb6810c473a",\
"schema":"chio.capability.v1", "scope":{"grants":[{"constraints":\
[{"type":"path_prefix", "value":"/workspace/"}], "max_invocations":3,\
"operations":["invoke", "read_result"], "server_id":"srv-files",\
"tool_name":"file_read"}]}, "subject":\
"0b513ad9b4924015ca0902ed079044d3ac5dbec2306f06948c10da8eb6e39f2d"}
```

The value of the signature member is:

```
===== NOTE: '\\' line wrapping per RFC 8792 =====

76466629cbe01ac3915bb67af0ff4fa591aec42d8a8f72e7c36731b7b1596cfb39e3\
a82dfc467dd2ed61c7ba7a5e728016d82b1702cbac5b8cc94719b99d0c08
```

Verification at Unix time 1710000400 is expected to produce:

```
{
  "delegation_chain_shape_valid": false,
  "signature_valid": true,
  "time_status": "valid",
  "time_valid": true
}
```

B.5. Receipt Verification

Each case gives the signing input of a receipt, the value of its signature member, and the result that verification is expected to produce. The signing input is the canonical form of a JSON object with two members: `body`, which is the receipt without its `id` and `signature` members, and `id`, which is the SHA-256 digest of the canonical form of `body`, in lowercase hex, as in [Appendix A.3](#). The `receipt_body_canonical_json` value of each case is the canonical form of the receipt without its signature member.

The following case is receipt/v1.json case allow_receipt from the Chio binding vector corpus. It records an allow decision. Its signing input is:

```
===== NOTE: '\\' line wrapping per RFC 8792 =====

{"body":{"action":{"parameter_hash":\
"035350b0e6a021a4c924a149111944945e94d12f6d8a5e24e4461e7009ebf929",\
"parameters":{"mode":"read","path":"/workspace/docs/roadmap.md"}},\
"boundary_class":"prevent","capability_id":"cap-bindings-001",\
"content_hash":\
"4062edaf750fb8074e7e83e0c9028c94e32468a8b6f1614774328ef045150f93",\
"decision":{"verdict":"allow"},"evidence":[{"details":\
"path_allowed","guard_name":"ForbiddenPathGuard","verdict":true},\
{"details":"no secrets detected","guard_name":"SecretLeakGuard",\
"verdict":true}], "kernel_key":\
"ea4a6c63e29c520abef5507b132ec5f9954776aebebe7b92421eea691446d22c",\
"metadata":{"surface":"bindings-vectors","version":1},"policy_hash":\
"policy-bindings-v1","receipt_kind":"mediated_decision",\
"redaction_mode":"none","timestamp":1710000200,"tool_name":\
"file_read","tool_origin":"caller_executed","tool_server":\
"srv-files","trust_level":"mediated"},"id":\
"c9909f733d5fb367922293f0167d06db715393be114f212096cf2866f29433ea"}
```

The value of the signature member is:

```
===== NOTE: '\\' line wrapping per RFC 8792 =====

276a4c59dbc5e0c4fbd47beefa33515d207e8f516aca296afee2cb95a443bb18d310\
31cae0b69943e7a930be03b2e0a60156e5a26e732e888e32d4653d059f03
```

With no trusted signers configured, verification is expected to produce:

```
===== NOTE: '\\' line wrapping per RFC 8792 =====

{
  "authorized": false,
  "boundary_class": "prevent",
  "decision": "allow",
  "ok": false,
  "parameter_hash_valid": true,
  "receipt_id_valid": true,
  "receipt_kind": "mediated_decision",
  "result": "Authorized",
  "signature_valid": true,
  "signer_key_hex": "ea4a6c63e29c520abef5507b132ec5f9954776aebebe7b9\
2421eea691446d22c",
  "signer_trusted": false,
  "trust_level": "mediated"
}
```

The following case is receipt/v1.json case deny_receipt from the Chio binding vector corpus. It records a deny decision by the guard ForbiddenPathGuard for the path /etc/shadow. Its signing input is:

```
===== NOTE: '\\' line wrapping per RFC 8792 =====

{"body":{"action":{"parameter_hash":\
"d6f55d803ff027795f4a4c43fd8f6d3119051f82b7e3e1a13b1713fd37ea7b09",\
"parameters":{"mode":"read","path":"/etc/shadow"}}, "boundary_class":\
"prevent", "capability_id":"cap-bindings-001", "content_hash":\
"4062edaf750fb8074e7e83e0c9028c94e32468a8b6f1614774328ef045150f93",\
"decision":{"guard":"ForbiddenPathGuard", "reason":\
"path is forbidden", "verdict":"deny"}, "evidence":[{"details":\
"path allowed", "guard_name":"ForbiddenPathGuard", "verdict":true},\
{"details":"no secrets detected", "guard_name":"SecretLeakGuard",\
"verdict":true}], "kernel_key":\
"ea4a6c63e29c520abef5507b132ec5f9954776aeb7b92421eea691446d22c",\
"metadata":{"surface":"bindings-vectors", "version":1}, "policy_hash":\
"policy-bindings-v1", "receipt_kind":"mediated_decision",\
"redaction_mode":"none", "timestamp":1710000200, "tool_name":\
"file_read", "tool_origin":"caller_executed", "tool_server":\
"srv-files", "trust_level":"mediated"}, "id":\
"56d5644db5c53d0bb837bf615a60e541724858757aee1b8259a25816b75bb682"}
```

The value of the signature member is:

```
===== NOTE: '\\' line wrapping per RFC 8792 =====

e3778cd741746bf1295ed45d5ffa4d311dd7076d4e8d7c009bdf3283e88e693cc9cd\
19bb07d2f65d8306e04d1c58bdc9c1753d79af4a7c2234758b5dbf8a1c06
```

With no trusted signers configured, verification is expected to produce:

```
===== NOTE: '\\' line wrapping per RFC 8792 =====

{
  "authorized": false,
  "boundary_class": "prevent",
  "decision": "deny",
  "ok": false,
  "parameter_hash_valid": true,
  "receipt_id_valid": true,
  "receipt_kind": "mediated_decision",
  "result": "Denied",
  "signature_valid": true,
  "signer_key_hex": "ea4a6c63e29c520abef5507b132ec5f9954776aeb7b9\
2421eea691446d22c",
  "signer_trusted": false,
  "trust_level": "mediated"
}
```

Appendix C. Relationship to Other Work

This appendix compares Chio with adjacent work. A comparison states where the problems overlap; it does not claim wire compatibility unless it says so.

C.1. Agent Communication Protocols

The Model Context Protocol [[MCP](#)] defines how a client discovers and calls the tools, resources, and prompts of a server. The Agent2Agent protocol [[A2A](#)] defines how agents exchange tasks and messages. Chio does not replace either. The MCP binding in [Section 11](#) makes a Chio kernel accessible as an MCP server. Applications using a compatible admission profile obtain the kernel's authority, accounting, governance, and evidence services through ordinary MCP tool calls. Chio sender constraints and governed metadata require a client that supplies those extensions.

C.2. Delegated Authorization

OAuth 2.0 Token Exchange [[RFC8693](#)], Rich Authorization Requests [[RFC9396](#)], and GNAP [[RFC9635](#)] address aspects of authorization. A Chio capability token plays a related role. Token Exchange conveys delegated authority, RAR describes fine-grained requested authorization, and GNAP negotiates grants. Chio uses signed delegation links and scope witnesses with verifier-owned issuer and root inputs; its current authenticated attenuation profile accepts one hop. Token signing still requires a trusted issuer. A kernel checks that authority and records receipt-bearing outcomes. Chio defines its own capability and receipt objects and does not claim GNAP compatibility.

DPoP [[RFC9449](#)] binds an OAuth access token to a key held by the client. The sender proof in [Section 5.4](#) has the same goal for capability tokens. It uses a Chio-specific proof format and is not the JWT-based proof defined by RFC 9449. The native v1 transport has no sender-proof member; the hosted HTTP binding uses the Chio format as specified in [Section 11.2](#).

C.3. Capability Tokens

Macaroons [[MACAROONS](#)], Biscuit [[BISCUIT](#)], and UCAN [[UCAN](#)] are bearer or key-bound capability tokens that support offline attenuation. Chio shares their model of narrowing authority by adding restrictions. Its authenticated attenuation profile binds a one-hop delegation to scope hashes, with a trusted token issuer and verifier-owned root inputs. Chio connects delegated authority to a stateful execution boundary: resource reservations, approval consumption, dispatch commitment, output release, and signed evidence belong to one recoverable operation. [Section 7](#) specifies the accounting profiles; [Section 3.4](#) defines the lifecycle that a token format alone does not provide.

C.4. Workload Identity and Attestation

The WIMSE architecture [[I-D.ietf-wimse-arch-08](#)] addresses identity for workloads that call one another across systems, and RATS [[RFC9334](#)] defines how a verifier appraises attestation evidence. A Chio subject is a public key. A deployment can bind that key to a workload identity or accept attestation evidence at issuance ([Section 12](#)), but attestation does not by itself authorize a call ([Section 15](#)).

C.5. Signed Evidence and Transparency

SCITT [[RFC9943](#)] defines signed statements and transparency services that record them. Chio receipts and checkpoints address a related problem: they let a party check what a kernel authorized and executed. Chio is not a SCITT profile, and [Section 9](#) states the limits of the claims that checkpoints support.

C.6. Verifiable Credentials

W3C Verifiable Credentials [[VC-DATA-MODEL-2.0](#)], OpenID for Verifiable Credential Issuance [[OID4VCI](#)], and OpenID for Verifiable Presentations [[OID4VP](#)] carry portable claims about a subject. Chio's portable identity credentials for agents, which the reference implementation issues through a bounded OID4VCI-compatible profile and presents through an OID4VP-style profile, are outside the scope of this document.

C.7. Candidate Companion Documents

The reference implementation carries mechanisms that this document does not specify. Each could be specified in its own document:

- federation: standing relationships between operators, including peer handshakes, key pinning, and cross-operator delegation;
- portable identity and reputation credentials for agents;
- settlement of obligations recorded in receipts, and payment channels;
- public anchoring of checkpoints;
- authenticated caller execution, provider evidence, and idempotent recovery across external execution boundaries;
- manifests, process confinement, information-flow labels, and declassification artifacts;
- credential-broker and key-lifecycle protocols, including witness policies and key-rotation evidence;
- active-response artifact and authority-service bindings;
- selective disclosure of receipt fields.

Appendix D. Document History

This section is to be removed before publishing as an RFC.

draft-whelan-chio-protocol-00: Initial version.

Acknowledgments

Chio builds on earlier work on capability-based authorization, in particular macaroons, Biscuit, and UCAN, and on the IETF's work on proof-of-possession tokens, workload identity, and transparency services.

Author's Address

Connor Whelan

Backbay Industries

Email: connor@backbay.io